Ch 1: Advanced Sorting Algorithms

Data Structure and Algorithm Design

Dr. Udaya R. Dhungana
Assistant Professor
Pokhara University, Nepal
Former Guest Faculty, Darmstadt University of Applied Sciences, Germany

CamScanner

Overview

Ch 1: Advanced Sorting Algorithms

- Non-comparison-based sorting
  - Radix sort
  - bucket sort
- Parallel Sorting Algorithms (Multicore and Distributed Systems)
- External sorting and its applications in large-scale data engineering

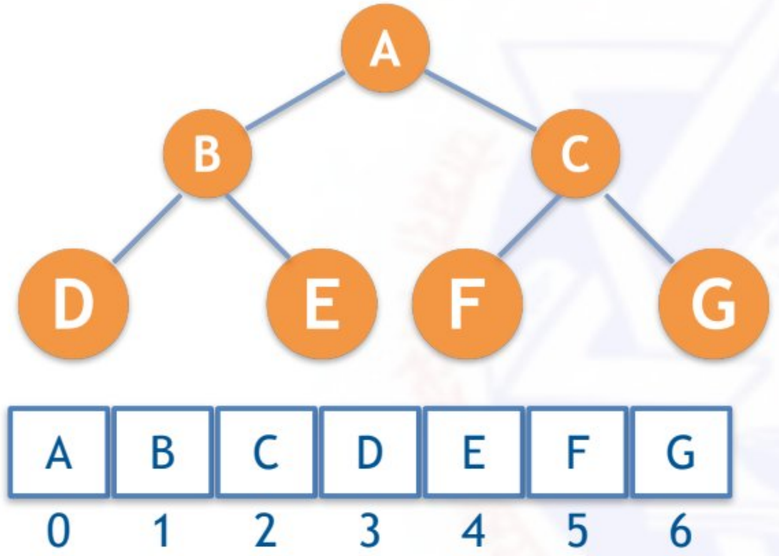
CamScanner

1. Review of analysis of Heap Sort, Merge Sort and Quick Sort
2. Lower Bounds of Sorting Problems and Decision Tree
3. Counting Sort
4. Radix Sort
5. Bucket Sort

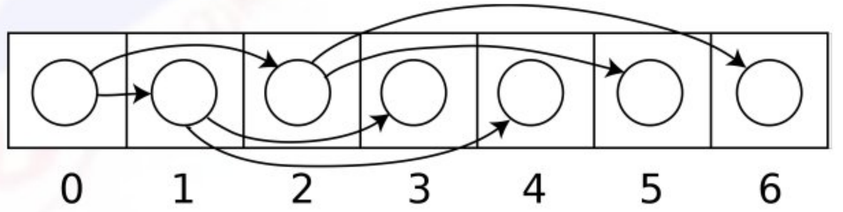
NON-COMPARISON-BASED SORTING

CamScanner

Array Representation of Binary Tree

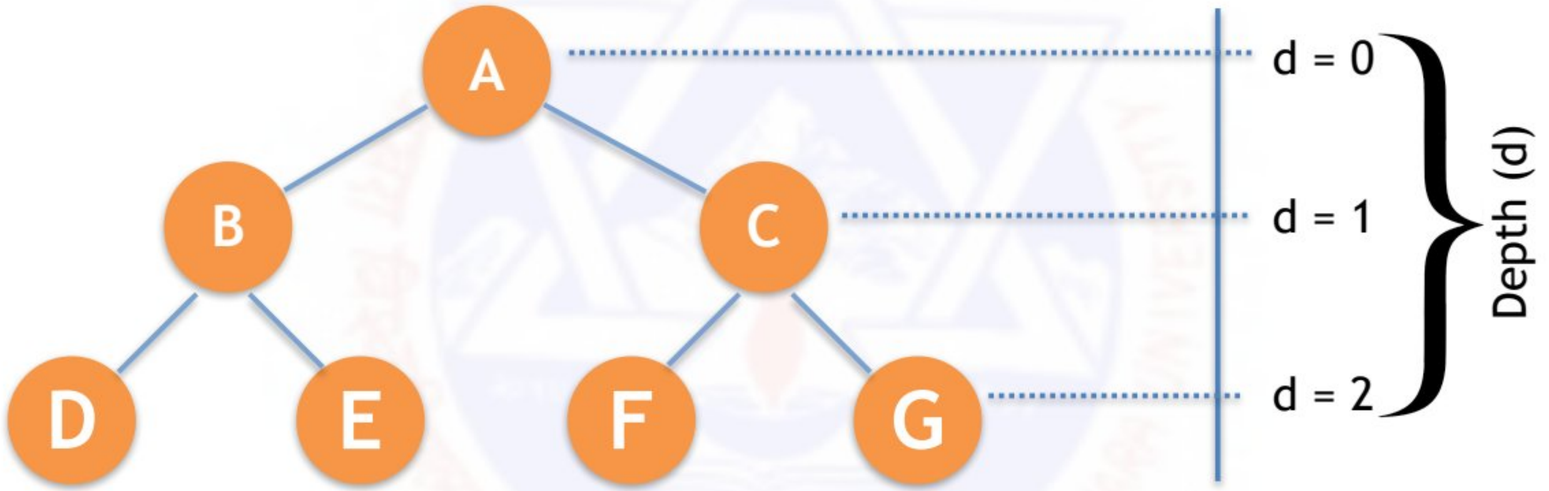


- If a *node* is at index *i* :
  - Its *left child* is at *2*i + 1*
  - Its *right child* is at *2*i + 2*
  - Its parent is at $\left\lfloor \frac{i-1}{2} \right\rfloor$



Perfect Binary Tree

Let us consider a perfect binary tree of depth *d* with *n* nodes:



No of nodes = $2^{d+1} - 1$ nodes.

Depth = $\log(n + 1) - 1$

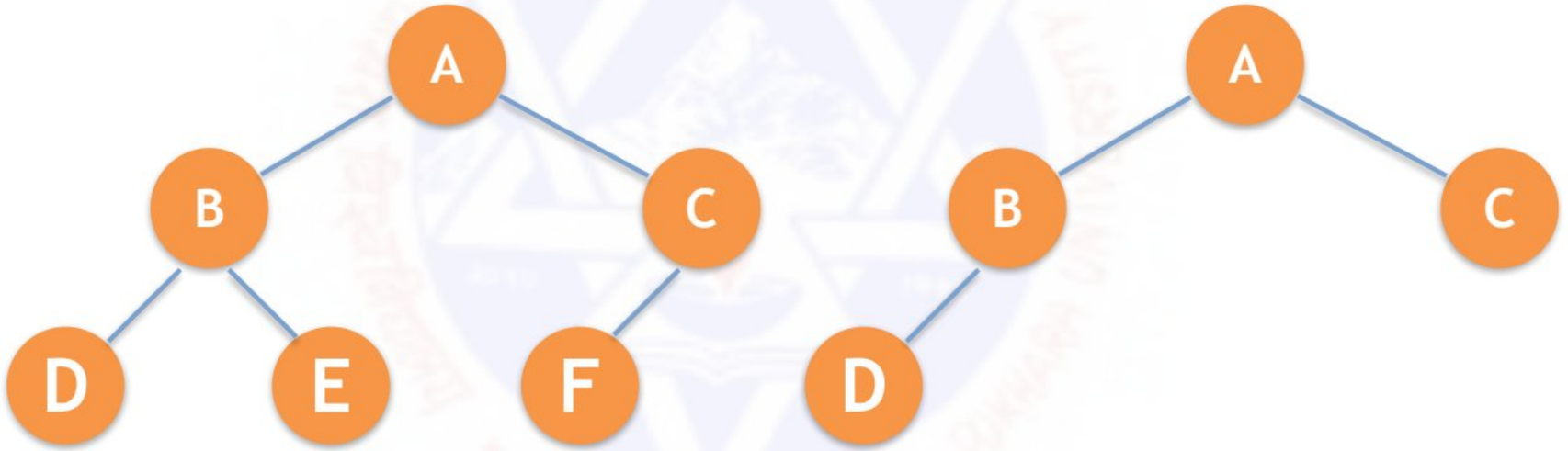
Leaf Nodes = 2^d

Depth d	# nodes at depth d	# of child nodes
0	1 = 2^0	2
1	2 = 2^1	4
2	4 = 2^0	8

CamScanner

Complete Binary Tree

A complete binary tree is a binary tree in which every level, except possibly the last, is completely filled, and all nodes are as far left as possible. (Also called almost complete or nearly complete binary tree)



(a) (b)

It can have between 1 and 2^h nodes at the last level h

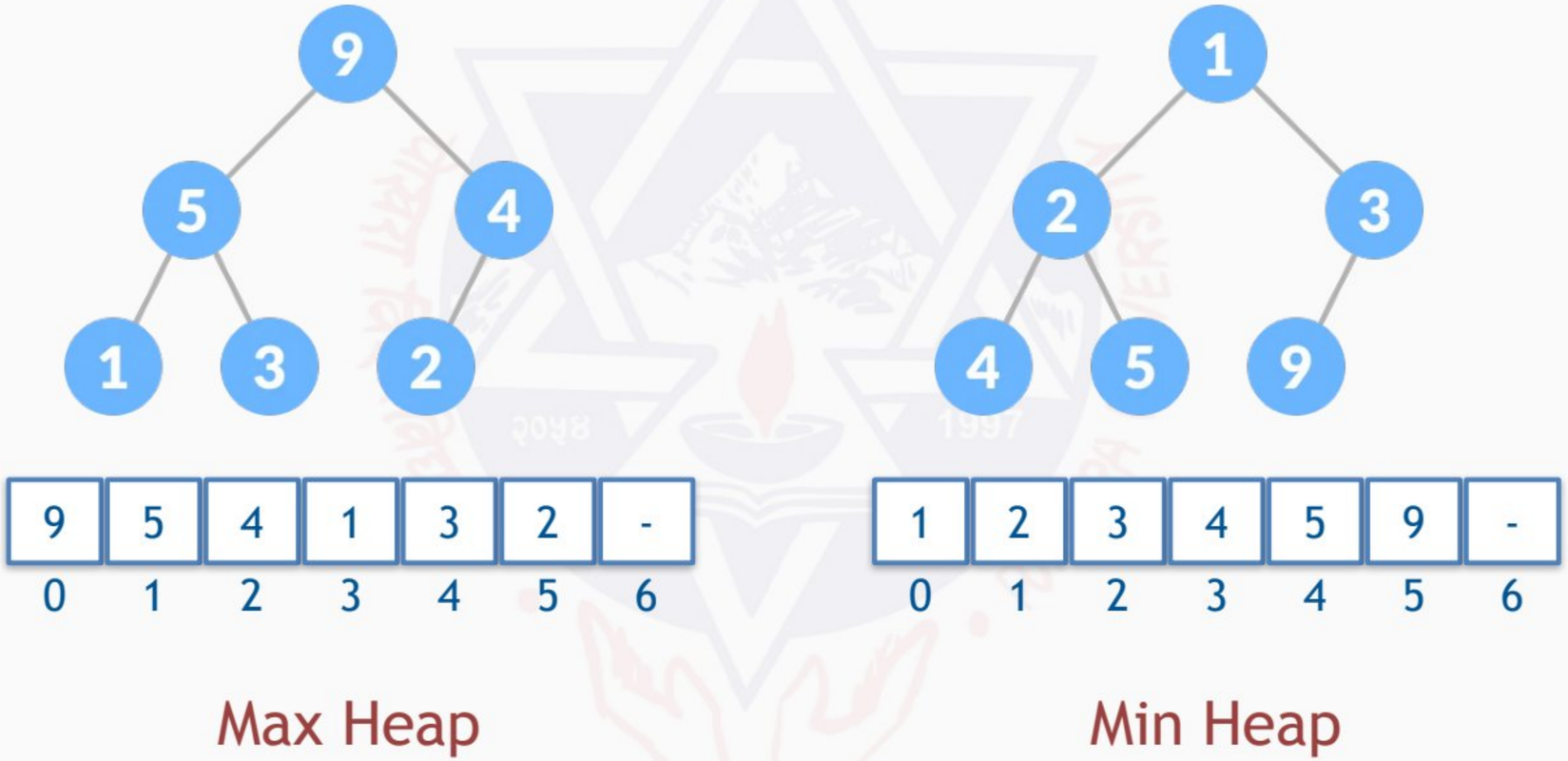
CamScanner

Binary Heap Data Structure

- A heap is a specialized tree-based data structure
- A binary heap is a ***complete binary tree*** that satisfies one of the following the ***heap property***:
  - The value of every node is greater than or equal to the value of its child nodes. Such heap is called ***max heap***. In max heap, the value of root node is the greatest value.
  - The value of every node is smaller than or equal to the value of its child nodes. Such heap is called ***min heap***. In min heap, the value of root node is the smallest value.

CamScanner

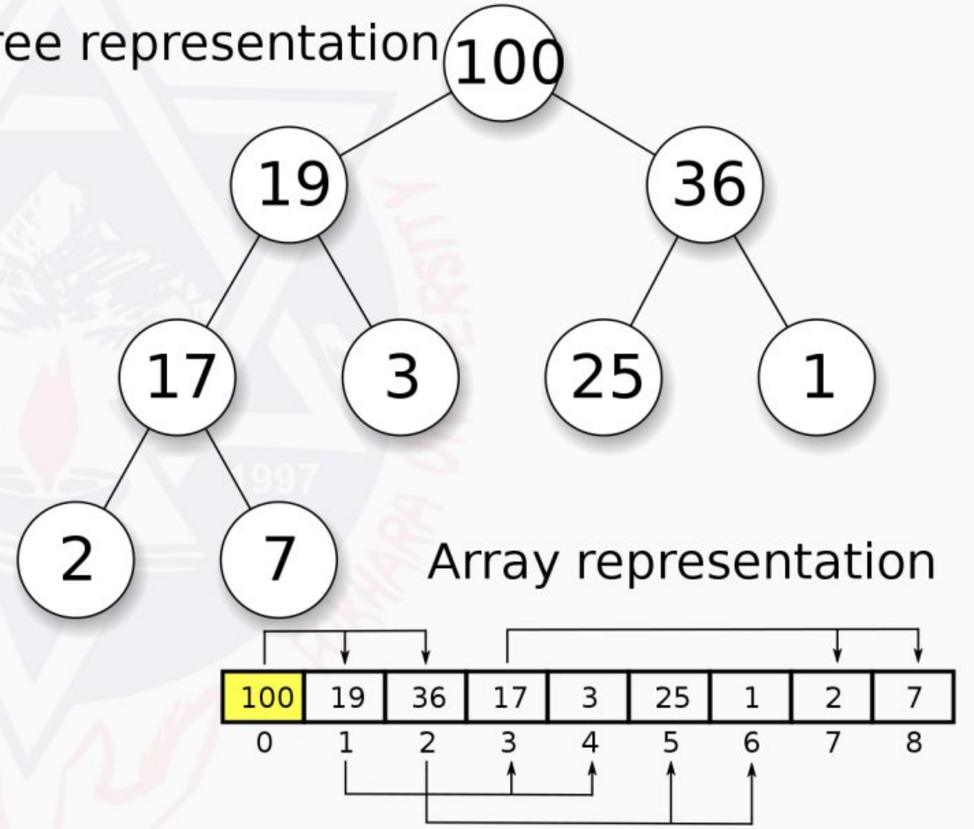
Binary Heap Data Structure



CamScanner

Binary Heap Data Structure

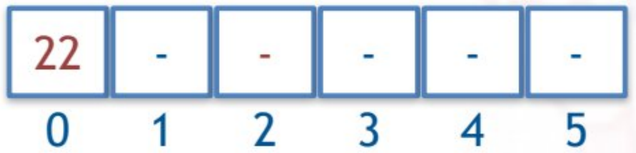
- Heap Sort
- Priority Queue
- Dijkstra's Algorithm



CamScanner

Building a Heap

Build a max heap from an array a[] = {22, 55, 44, 66, 58, 77}.



CamScanner

Building a Heap

Build a max heap from an array a[] = {22, 55, 44, 66, 58, 77}.

22	55	-	-	-	-
0	1	2	3	4	5

CamScanner

Building a Heap

Build a max heap from an array a[] = {22, 55, 44, 66, 58, 77}.

55	22	-	-	-	-
0	1	2	3	4	5

CamScanner

Building a Heap

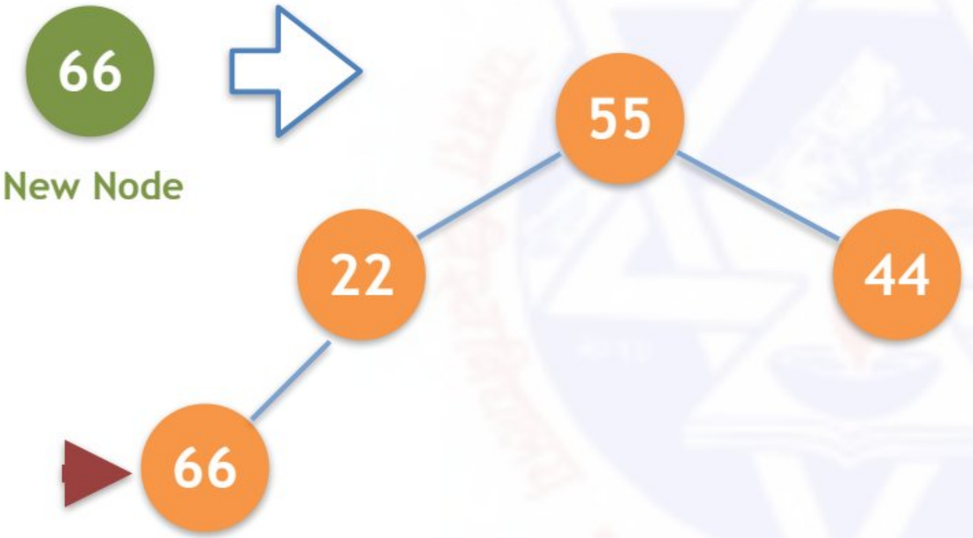
Build a max heap from an array a[] = {22, 55, 44, 66, 58, 77}.

55	22	44	-	-	-
0	1	2	3	4	5

CamScanner

Building a Heap

Build a max heap from an array a[] = {22, 55, 44, 66, 58, 77}.

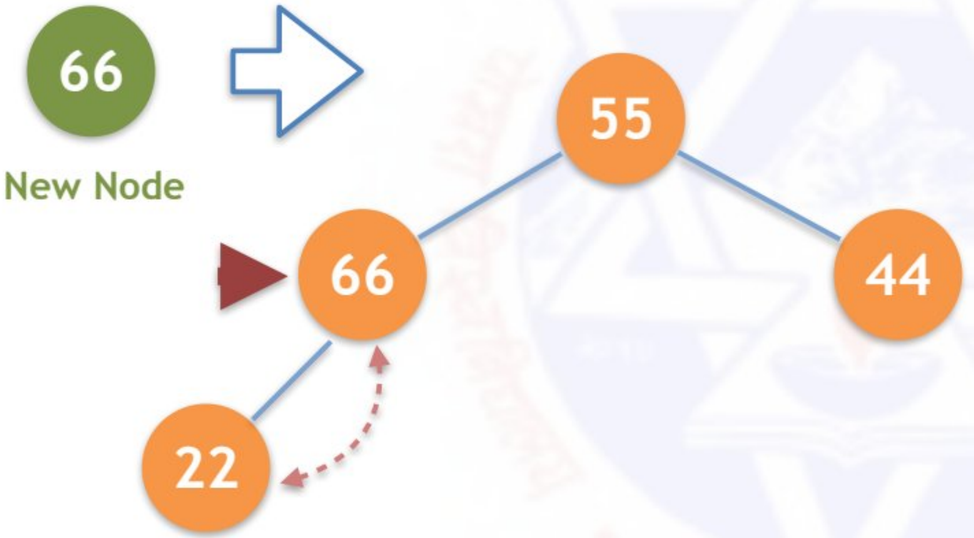


55	22	44	66	-	-
0	1	2	3	4	5

CamScanner

Building a Heap

Build a max heap from an array a[] = {22, 55, 44, 66, 58, 77}.

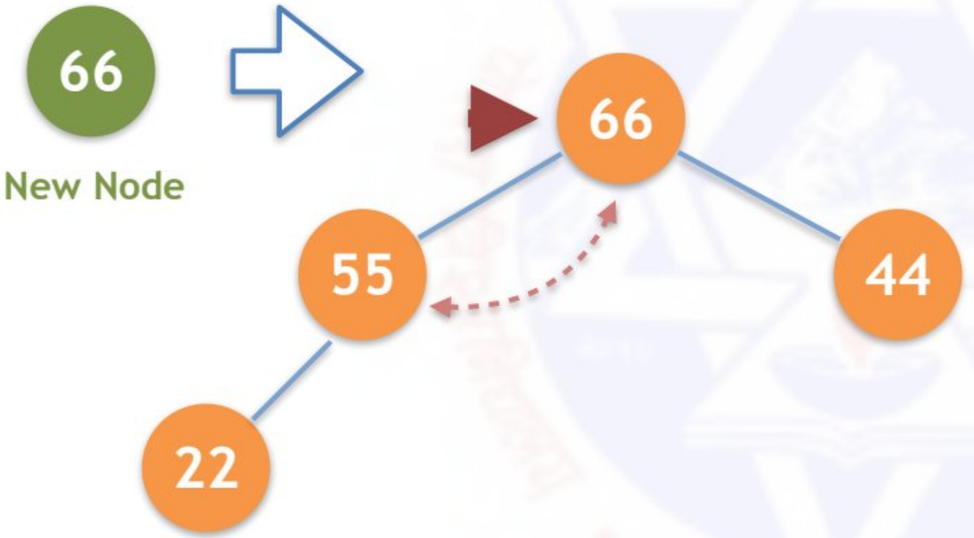


55	66	44	22	-	-
0	1	2	3	4	5

CamScanner

Building a Heap

Build a max heap from an array a[] = {22, 55, 44, 66, 58, 77}.

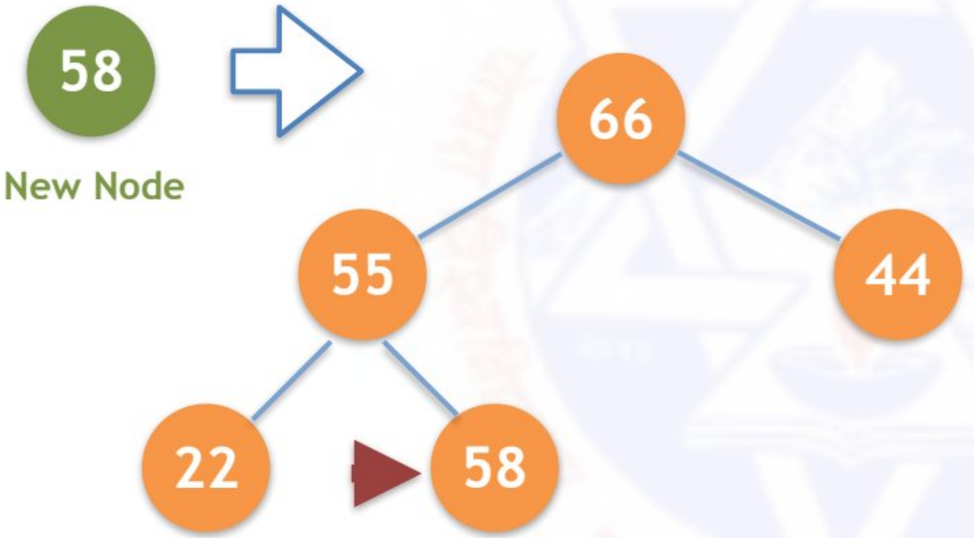


66	55	44	22	-	-
0	1	2	3	4	5

CamScanner

Building a Heap

Build a max heap from an array a[] = {22, 55, 44, 66, 58, 77}.

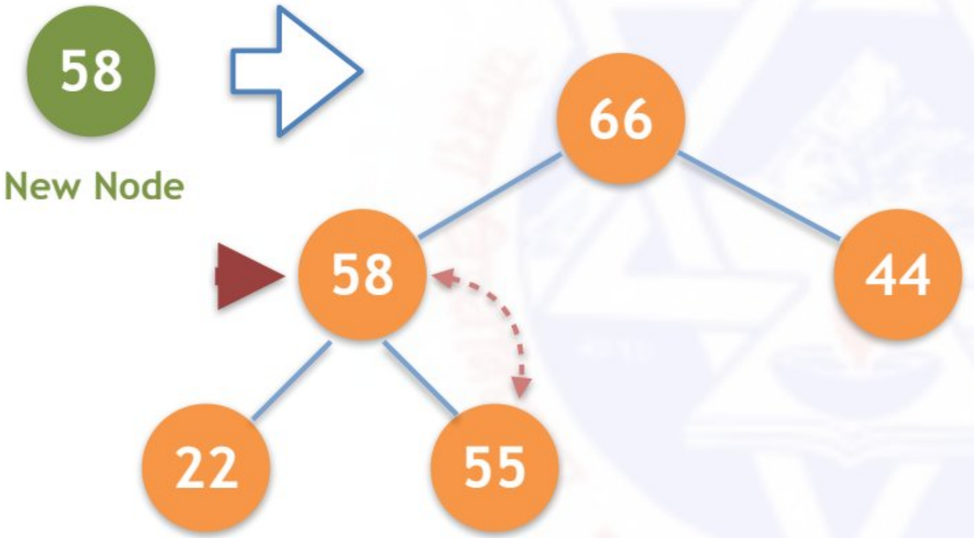


66	55	44	22	58	-
0	1	2	3	4	5

CamScanner

Building a Heap

Build a max heap from an array a[] = {22, 55, 44, 66, 58, 77}.

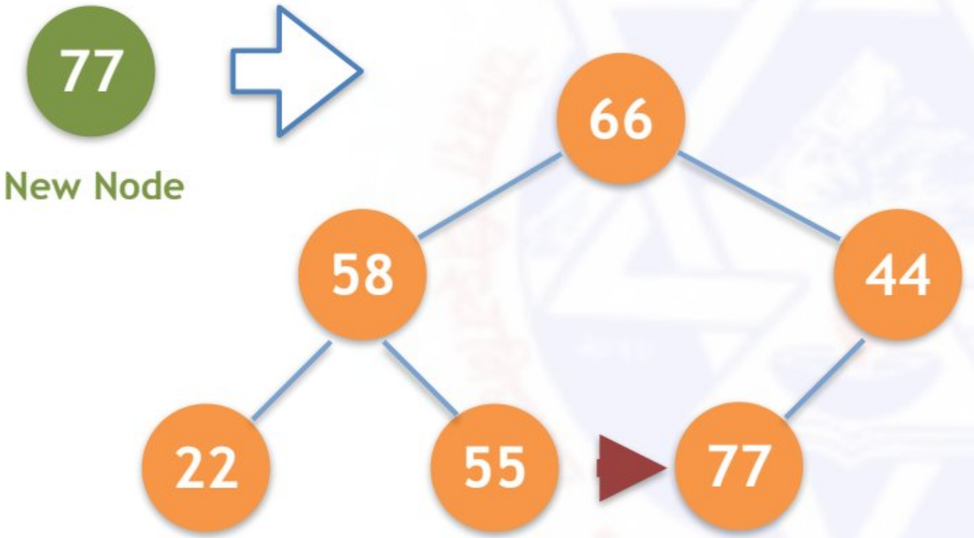


66	58	44	22	55	-
0	1	2	3	4	5

CamScanner

Building a Heap

Build a max heap from an array a[] = {22, 55, 44, 66, 58, 77}.

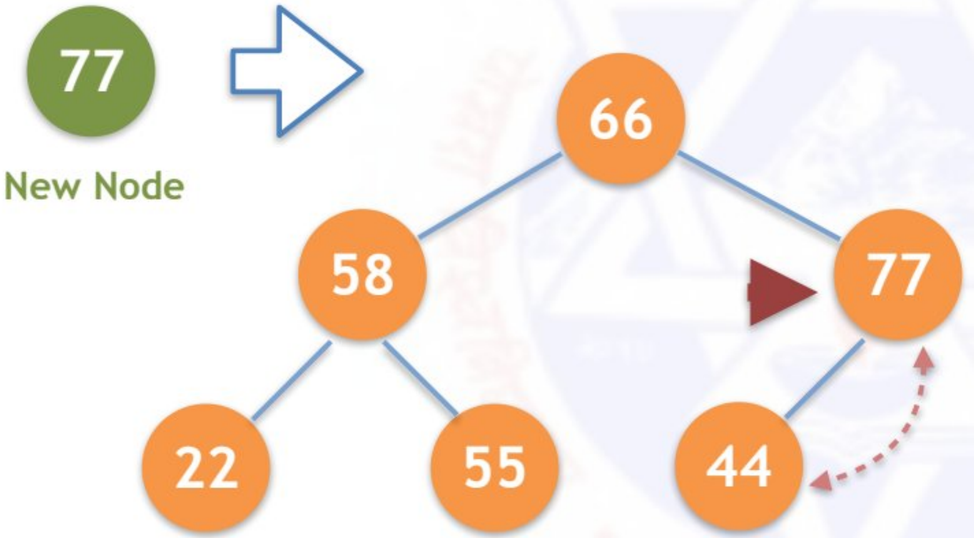


66	58	44	22	55	77
0	1	2	3	4	5

CamScanner

Building a Heap

Build a max heap from an array a[] = {22, 55, 44, 66, 58, 77}.

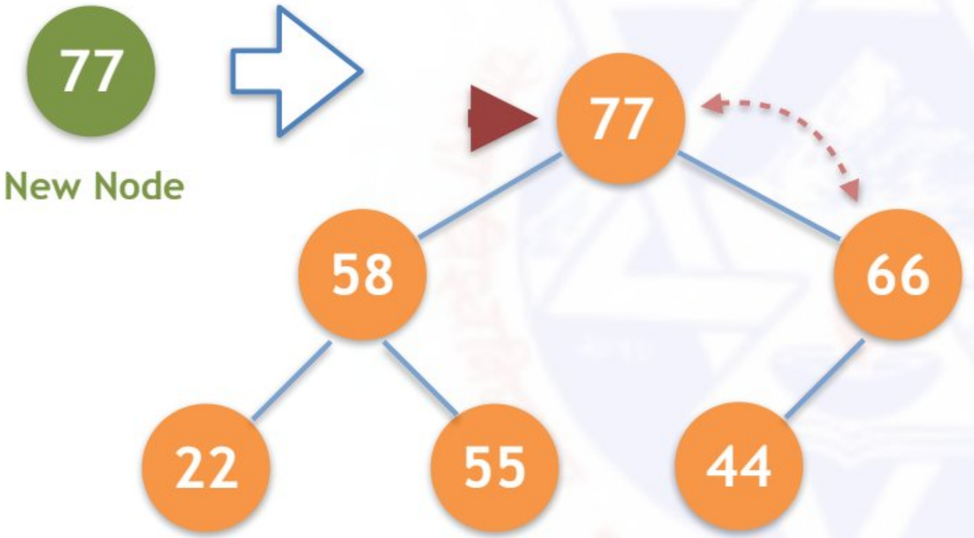


66	58	77	22	55	44
0	1	2	3	4	5

CamScanner

Building a Heap

Build a max heap from an array a[] = {22, 55, 44, 66, 58, 77}.

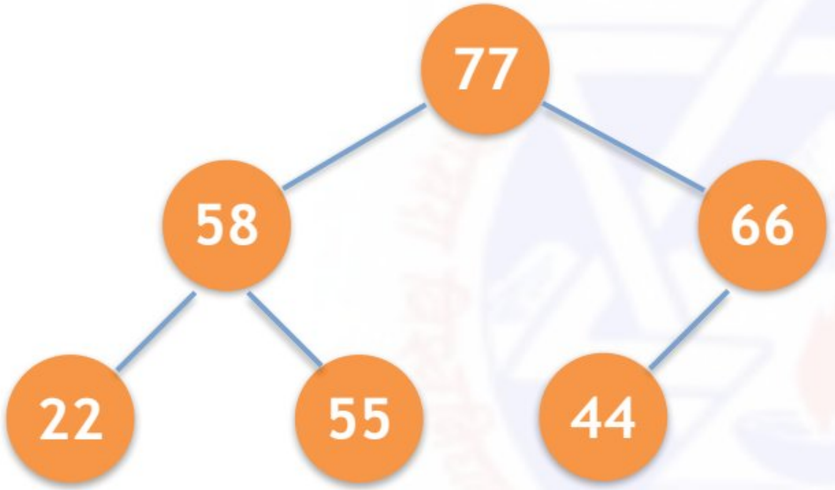


77	58	66	22	55	44
0	1	2	3	4	5

CamScanner

Heap Sort (Deleting elements from heap)

Using max heap, sort an array a[] = {22, 55, 44, 66, 58, 77}.



77	58	66	22	55	44
0	1	2	3	4	5

Algorithm:

1. Build a max heap from the array
2. Repeat the following steps until the heap contains a single node:
   - Swap the root node with the last node.
   - Decrease the size of heap by 1
   - Check the heap property. If it is violated, restore the heap property (heapify).

CamScanner

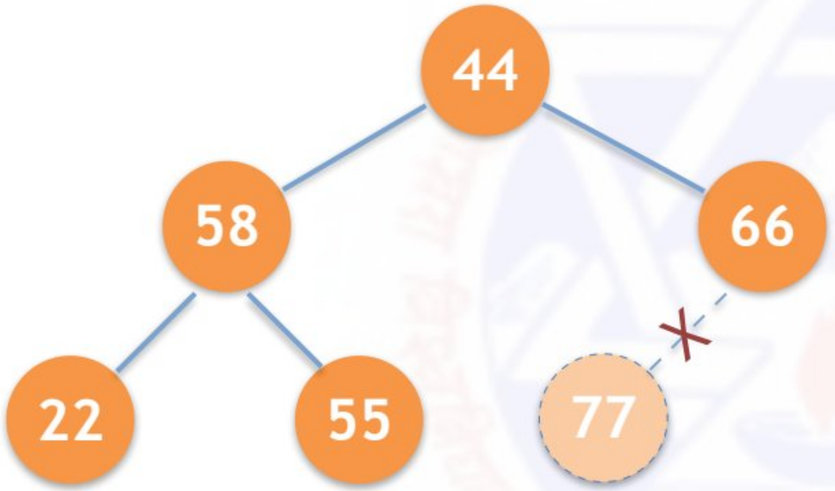
Heap Sort (Deleting elements from heap)

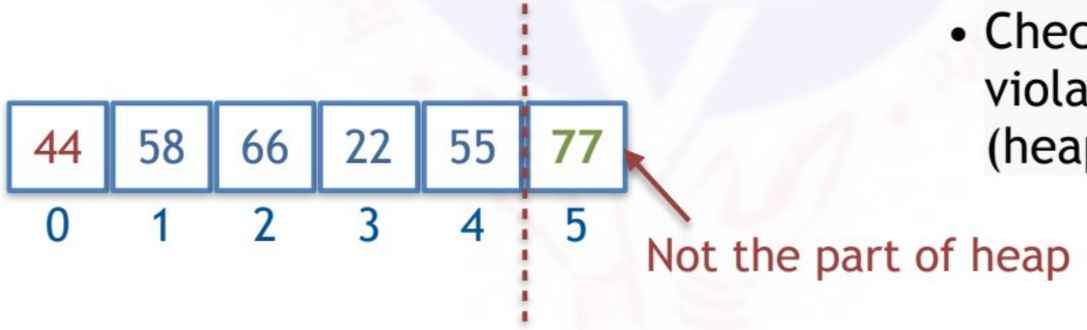
Algorithm:

1. Build a max heap from the array
2. Repeat the following steps until the heap contains a single node:
   - Swap the root node with the last node.
   - Decrease the size of heap by 1
   - Check the heap property. If it is violated, restore the heap property (heapify).

CamScanner

Heap Sort (Deleting elements from heap)





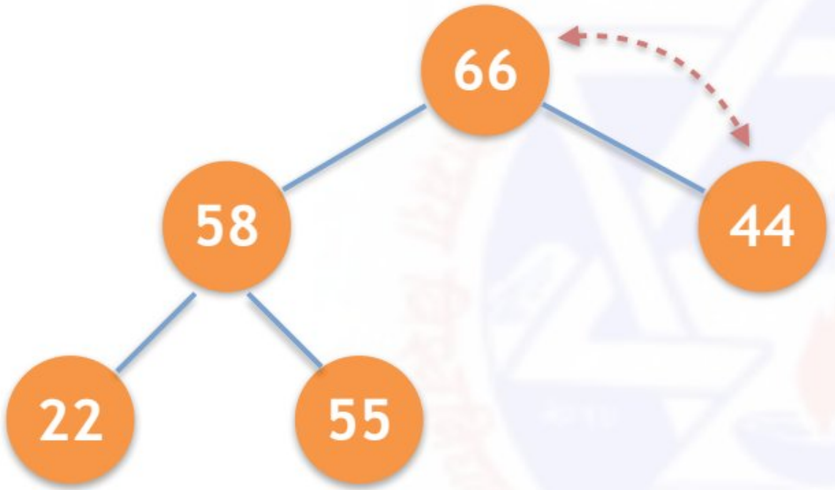
Algorithm:

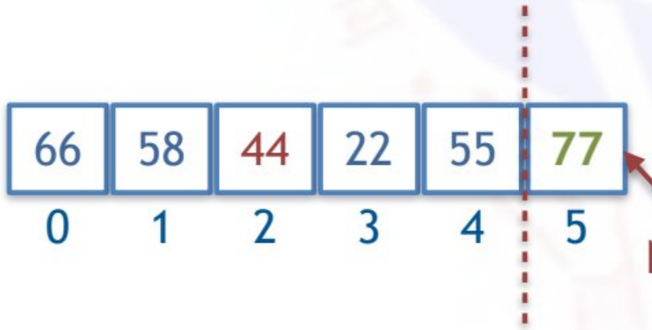
1. Build a max heap from the array
2. Repeat the following steps until the heap contains a single node:
   - Swap the root node with the last node.
   - Decrease the size of heap by 1
   - Check the heap property. If it is violated, restore the heap property (heapify).

CamScanner

Heap Sort (Deleting elements from heap)

Compare two children of root, pick the greater one and compare it with the root node.
If root isn't greater, swap the greater child with root.



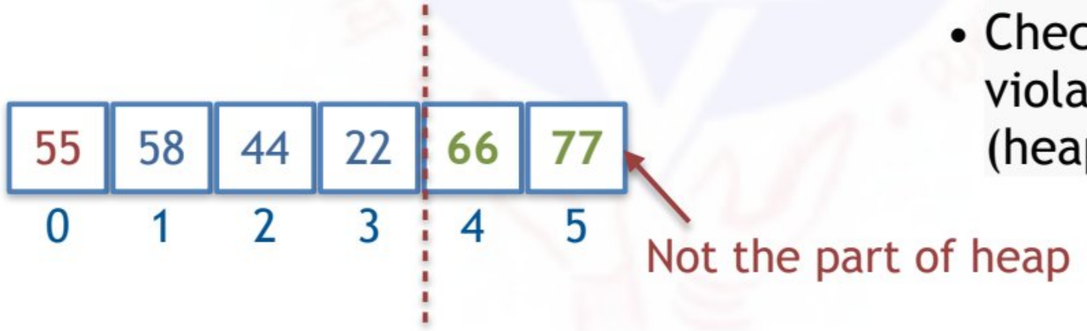


Algorithm:

1. Build a max heap from the array
2. Repeat the following steps until the heap contains a single node:
   - Swap the root node with the last node.
   - Decrease the size of heap by 1
   - Check the heap property. If it is violated, restore the heap property (heapify).

CamScanner

Heap Sort (Deleting elements from heap)

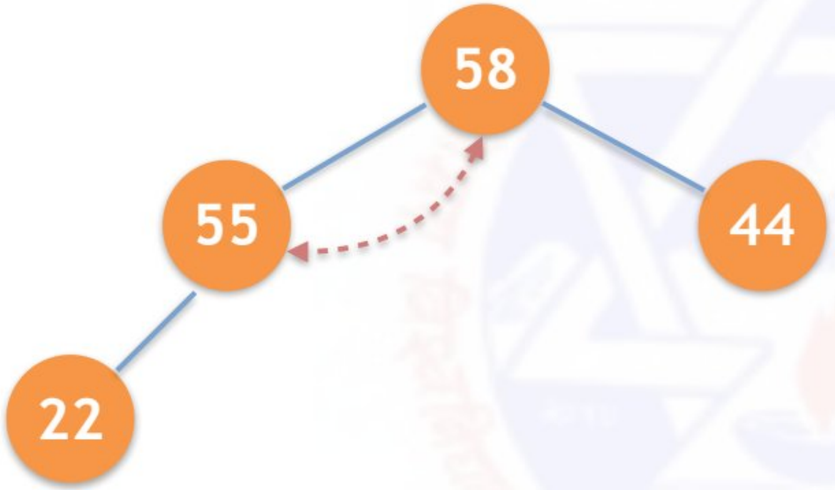


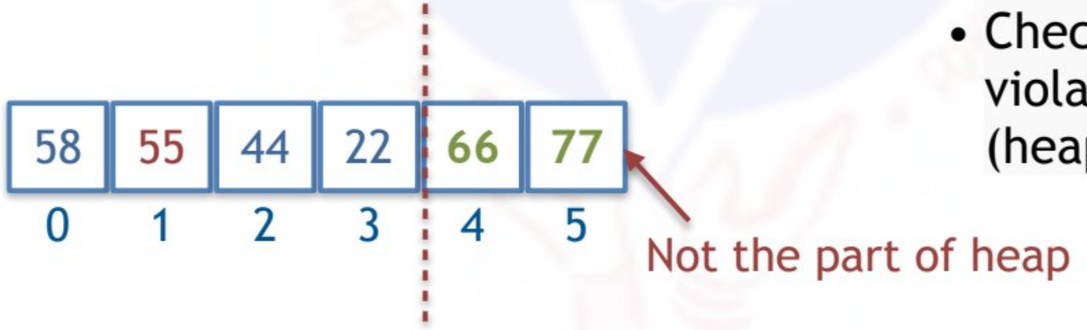
Algorithm:

1. Build a max heap from the array
2. Repeat the following steps until the heap contains a single node:
   - Swap the root node with the last node.
   - Decrease the size of heap by 1
   - Check the heap property. If it is violated, restore the heap property (heapify).

CamScanner

Heap Sort (Deleting elements from heap)



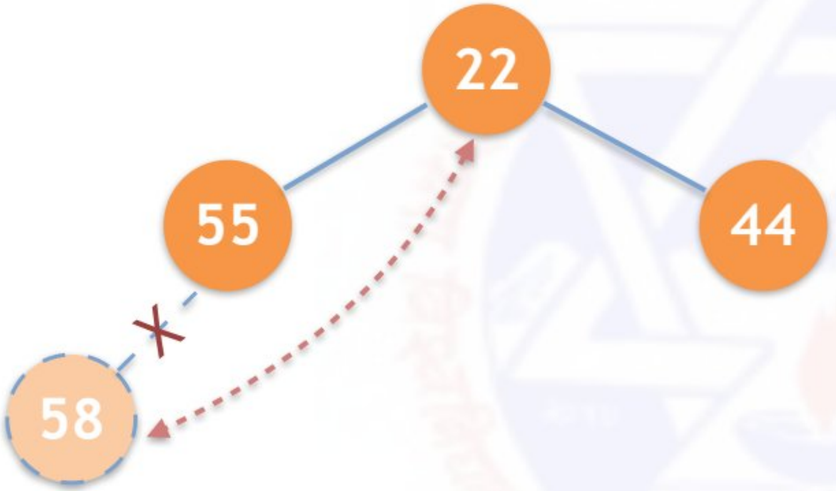


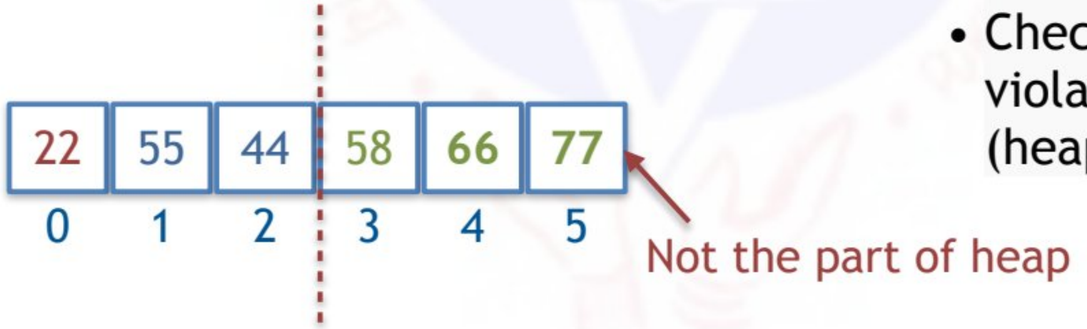
Algorithm:

1. Build a max heap from the array
2. Repeat the following steps until the heap contains a single node:
   - Swap the root node with the last node.
   - Decrease the size of heap by 1
   - Check the heap property. If it is violated, restore the heap property (heapify).

CamScanner

Heap Sort (Deleting elements from heap)



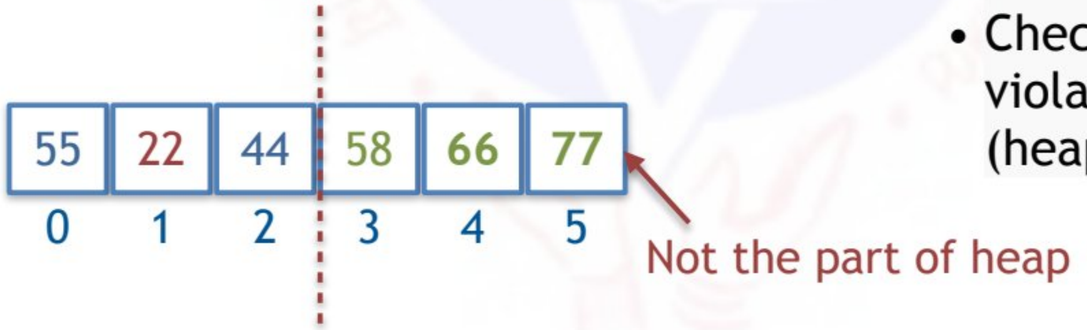


Algorithm:

1. Build a max heap from the array
2. Repeat the following steps until the heap contains a single node:
   - Swap the root node with the last node.
   - Decrease the size of heap by 1
   - Check the heap property. If it is violated, restore the heap property (heapify).

CamScanner

Heap Sort (Deleting elements from heap)

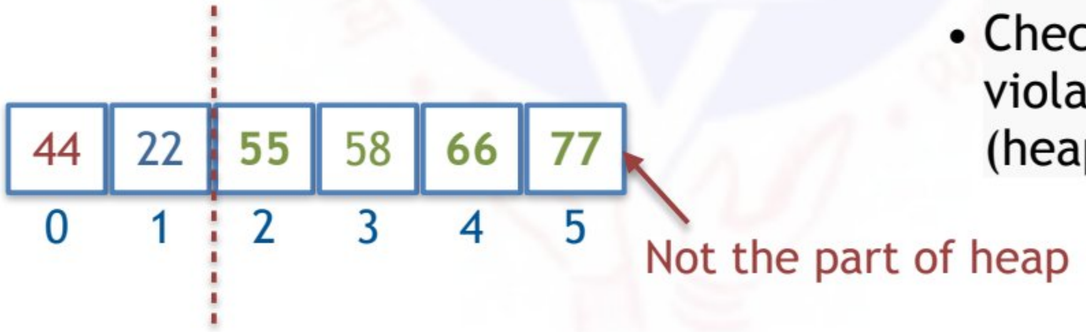


Algorithm:

1. Build a max heap from the array
2. Repeat the following steps until the heap contains a single node:
   - Swap the root node with the last node.
   - Decrease the size of heap by 1
   - Check the heap property. If it is violated, restore the heap property (heapify).

CamScanner

Heap Sort (Deleting elements from heap)

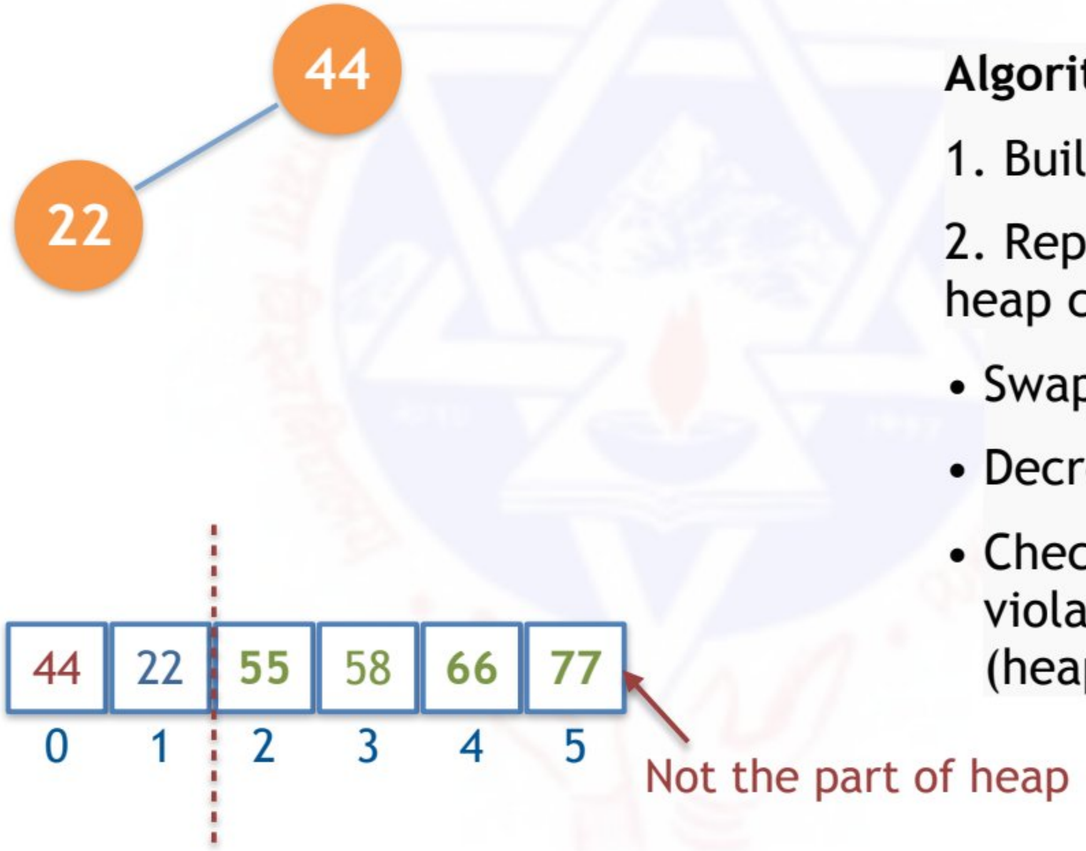


Algorithm:

1. Build a max heap from the array
2. Repeat the following steps until the heap contains a single node:
   - Swap the root node with the last node.
   - Decrease the size of heap by 1
   - Check the heap property. If it is violated, restore the heap property (heapify).

CamScanner

Heap Sort (Deleting elements from heap)

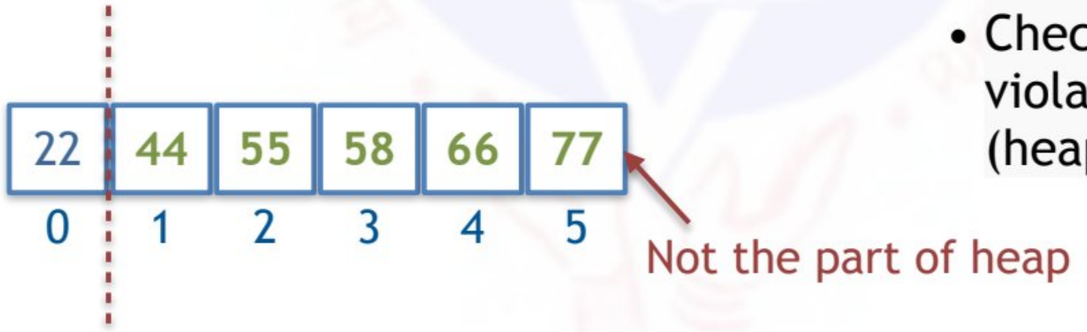


Algorithm:

1. Build a max heap from the array
2. Repeat the following steps until the heap contains a single node:
   - Swap the root node with the last node.
   - Decrease the size of heap by 1
   - Check the heap property. If it is violated, restore the heap property (heapify).

CamScanner

Heap Sort (Deleting elements from heap)

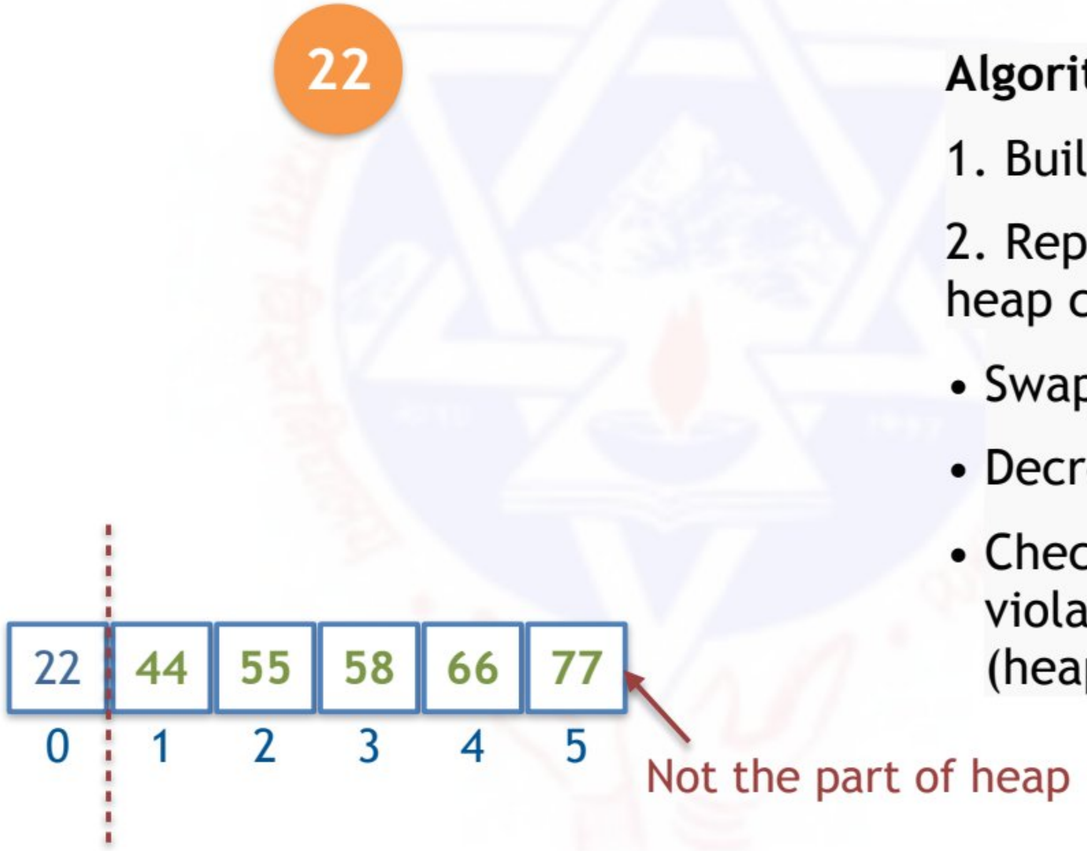


Algorithm:

1. Build a max heap from the array
2. Repeat the following steps until the heap contains a single node:
   - Swap the root node with the last node.
   - Decrease the size of heap by 1
   - Check the heap property. If it is violated, restore the heap property (heapify).

CamScanner

Heap Sort (Deleting elements from heap)



Algorithm:

1. Build a max heap from the array
2. Repeat the following steps until the heap contains a single node:
   - Swap the root node with the last node.
   - Decrease the size of heap by 1
   - Check the heap property. If it is violated, restore the heap property (heapify).

CamScanner

Heap Sort (Deleting elements from heap)

Algorithm:

1. Build a max heap from the array
2. Repeat the following steps until the heap contains a single node:
   - Swap the root node with the last node.
   - Decrease the size of heap by 1
   - Check the heap property. If it is violated, restore the heap property (heapify).

CamScanner

Analysis of Heap Sort

```
HEAPSORT(A, n)
1  BUILD-MAX-HEAP(A, n)
2  for i = n downto 2
3      exchange A[1] with A[i]
4      A.heap-size = A.heap-size − 1
5      MAX-HEAPIFY(A, 1)
```

What is the time complexity of HEAPSORT(A,n)?

** Class Discussion **

CamScanner

Analysis of Heap Sort

```
HEAPSORT(A, n)
1  BUILD-MAX-HEAP(A, n)
2  for i = n downto 2
3      exchange A[1] with A[i]
4      A.heap-size = A.heap-size − 1
5      MAX-HEAPIFY(A, 1)
```

Since the call to BUILD-MAX-HEAP procedure takes O(n) time and each of the n-1 calls to MAX-HEAPIFY takes O(log n) time.

Therefore, complexity = O(n log(n))

CamScanner

Analysis of Heap Sort

- The heap sort algorithm HEAPSORT(A, n), starts by calling the BUILD-MAX-HEAP(A, n) procedure to build a max-heap on the input array A[1:n].
- Since the maximum element of the array is stored at the root A[1], HEAPSORT can place it into its correct final position by exchanging it with A[n]. If the procedure then discards node n from the heap and it can do so by simply decrementing heap-size by 1. The children of the root remain max-heaps, but the new root element might violate the max-heap property.
- To restore the max-heap property, the procedure just calls MAX-HEAPIFY(A, 1) , which leaves a max-heap in A[1:n-1].
- The HEAPSORT procedure then repeats this process for the max-heap of size n-1 down to a heap of size 2.
- The HEAPSORT procedure takes O(n log n) time since the call to BUILD-MAX-HEAP procedure takes O(n) time and each of the n-1 calls to MAX-HEAPIFY takes O(log n) time.

CamScanner

Comparison Based Sorting

- ***Comparison based Sorting Algorithms:*** The sorted order they determine is based on comparisons between the input elements.

Sorting Algorithm	Best Case	Average Case	Worst Case	Space	Stable
Insertion Sort	$O(n)$ - nearly sorted	$O(n^2)$	$O(n^2)$	$O(1)$ in–place	Yes
Selection sort	$O(n^2)$	$O(n^2)$	$O(n^2)$	$O(1)$ in–place	No
Bubble Sort	$O(n)$ - already sorted	$O(n^2)$	$O(n^2)$	$O(1)$ in–place	Yes
Merge Sort	$O(n \log n)$	$O(n \log n)$	$O(n \log n)$	$O(n)$ extra memory for merging	Yes
Heap Sort	$O(n \log n)$	$O(n \log n)$	$O(n \log n)$	$O(1)$ in–place	No
Quick Sort	$O(n \log n)$	$O(n \log n)$	$O(n^2)$	$O(\log n)$ for recursive calls	No

CamScanner

Cost Analysis of Sorting Problems

- Here we consider a more difficult task — an **analysis for the cost of a *problem*** as opposed to an *algorithm*.
- The **upper bound**
  - the asymptotic cost of the ***fastest known algorithm***.
  - It is $O(n \log n)$
- The **lower bound**
  - best possible efficiency for *any* algorithm that solves the problem, including algorithms not yet invented
  - It might be $O(n)$
- Cost of sorting problem:- $O(n)$ to $O(n \log n)$

CamScanner

Lower Bounds of Sorting Problems

- Estimation of lower bound of a problem can be obtained by measuring
  - the size of the input that must be read and
  - The size of the output that must be written
  - Certainly no algorithm can be more efficient than the problem's I/O time.
  - Therefore, **the *sorting problem* cannot be solved by *any* algorithm in less than $\Omega(n)$ time** because it takes at least n steps to read and write the n values to be sorted.
- Based on our current knowledge of sorting algorithms and the size of the input (n), we know that **the *problem* of sorting is bounded by $\Omega(n)$ and $O(n \log n)$.**

CamScanner

Lower Bounds of Sorting Problems

- Computer scientists have spent much time devising efficient general-purpose sorting algorithms, but no one has ever found one that is faster than $O(n\ log\ n)$ in the worst or average cases.
  - Should we keep searching for a faster sorting algorithm?
  - Or can we prove that there is no faster sorting algorithm by finding a tighter lower bound?
- The answer is: No sorting algorithm based on key comparisons can possibly be faster than Ω(n log n) in the worst case.
  - you need not bang your head against the wall searching for an $O(n)$ sorting algorithm based on key comparisons
  - *(See Proof: at book "A Practical Introduction to Data Structures and Algorithm Analysis by Clifford A. Shaffer (Page no: 275))*

CamScanner

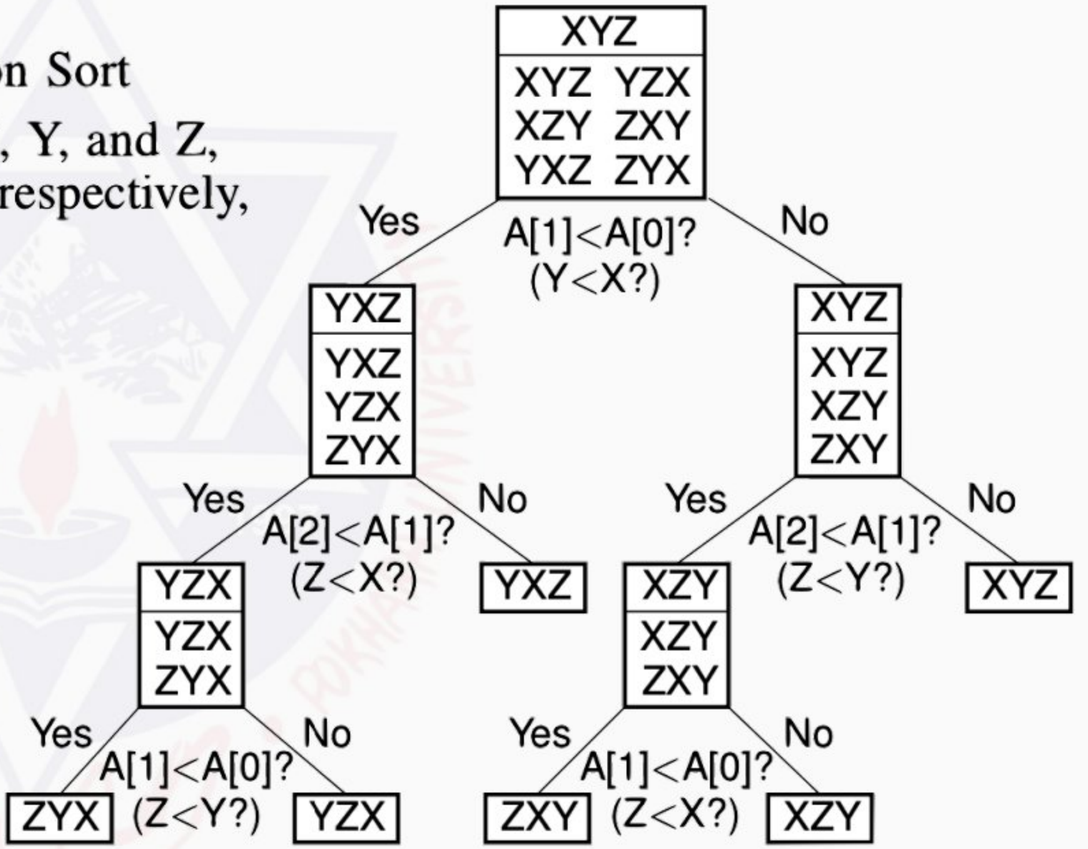
Lower Bounds of Sorting Problems

- **Decision Tree:**
  - A decision tree is a ***full binary tree*** that can model the processing for any algorithm that makes decisions. It represents the comparisons between elements that are performed by a particular sorting algorithm operating on an inout of a given size.
  - If two keys are compared and the first is less than the second, then this is modeled as a left branch in the decision tree.
  - In the case where the first value is greater than the second, the algorithm takes the right branch.
- We use ***decision tree*** to prove the lower bound of comparison based sorting algorithms is $\Omega(n \log n)$

CamScanner

Lower Bounds of Sorting Problems

Figure 7.14 Decision tree for Insertion Sort when processing three values labeled X, Y, and Z, initially stored at positions 0, 1, and 2, respectively, in input array A.



What are the possible outputs?

- The correct output could be any permutation of the input values.
- For three values, there are n! = 6 permutations.
- Thus, the root node of the decision tree lists all six permutations that might be the eventual result of the algorithm

CamScanner

Lower Bounds of Sorting Problems

- **Decision Tree:**
  - Any sorting algorithm based on comparisons can be modeled by a decision tree and all sorting algorithms can be viewed as algorithms to “find” the correct permutation of the input that yields a sorted list.
  - Each algorithm based on comparisons can be viewed as proceeding by making branches in the tree based on the results of key comparisons, and each algorithm can terminate once a node with a single permutation has been reached.
  - The decision tree shows the decisions made by an algorithm for all possible inputs of a given size. Each path through the tree from the root to a leaf is one possible series of decisions taken by the algorithm.
    - **A lower bound for the Worst-case:** The length of the longest path from the root of a decision tree to any of the reachable leaves represents the worst case number of comparison that the corresponding sorting algorithm performs.
    - Consequently, the worst-case number of comparisons for a given comparison sort algorithm equals the height of its decision tree.

CamScanner

Lower Bounds of Sorting Problems

- **Decision Tree:**
  - There are many comparison-based sorting algorithms, and each will be modeled by a different decision tree.
  - Some decision trees might be well-balanced ($O(logn)$ levels), others might be unbalanced ($O(n)$ levels).
  - Some trees will have more nodes than others (those with more nodes might be making “unnecessary” comparisons). In fact, a poor sorting algorithm might have an arbitrarily large number of nodes in its decision tree, with leaves of arbitrary depth.
  - However, we are interested here in knowing what the *best* sorting algorithm could have as its minimum cost in the worst case.

CamScanner

Lower Bounds of Sorting Problems

- **Decision Tree:**
  - Because there are at least n! nodes in the tree, we know that the tree must have $\Omega(log\ n!)$ levels.
  - The decision tree for any comparison-based sorting algorithm must have nodes $\Omega(n\ log\ n)$ levels deep.
  - Thus, in the worst case, any such sorting algorithm must require $\Omega(n\ log\ n)$ comparisons.
- Any sorting algorithm requiring $\Omega(n\ log\ n)$ comparisons in the worst case requires $\Omega(n\ log\ n)$ running time in the worst case.
- Because any sorting algorithm requires $\Omega(n\ log\ n)$ running time, ***the problem of sorting also requires*** $\Omega(n\ log\ n)$ ***time. (Hence proved using decision tree)***
- ***Therefore,*** No comparison-based sorting algorithm can improve on existing $\Theta(n\ log\ n)$ time sorting algorithms by more than a constant factor.

CamScanner

Sorting in Linear Time

- We have seen some algorithms that
  - can sort n numbers in $O(n\ log\ n)$ time.
- The merge sort and heap sort
  - achieve this upper bound $O(n\ log\ n)$ in the worst case,
- The quick sort
  - achieves the upper bound $O(n\ log\ n)$ on average case.
- In previous slides, we saw
  - Any comparison sort must make $\Omega(n\ log\ n)$ comparisons in the worst case to sort n elements.
  - The ***merge sort*** and ***heap sort*** are asymptotically optimal and no comparison sort exists that is faster by more than a constant factor.

CamScanner

Sorting in Linear Time

- As we have mentioned, it can be proved that
  - a comparison sorting algorithm can never have a worst-case time better than $\Omega(n \log n)$.
- Under some special conditions having to do with the values to be sorted, it is possible to design other kinds of sorting algorithms that have better worst-case times.
- We next examine three such sorting algorithms
  - counting sort, radix sort, and bucket sort (***non-comparison based sorting***)
    - These run in ***linear time*** on certain types of inputs.
  - Of course, these algorithms use operations other than comparisons to determine the sorted order.

CamScanner

Counting Sort

- Counting sort assumes that each of the n input elements is an integer in the range 0 to k, for some integer k.
- It runs in $\Theta(n + k)$ time, so that when $k = O(n)$, counting sort runs in $\Theta(n)$ time.
- How it works?
  - Counting sort first determines, for each input element x, the number of elements less than or equal to x.
  - It then uses this information to place element x directly into its position in the output array. For example, if 4 elements are less than or equal to x, then x belongs in output position 4.
  - We must modify this scheme slightly to handle the situation in which several elements have the same value, since we do not want them all to end up in the same position.

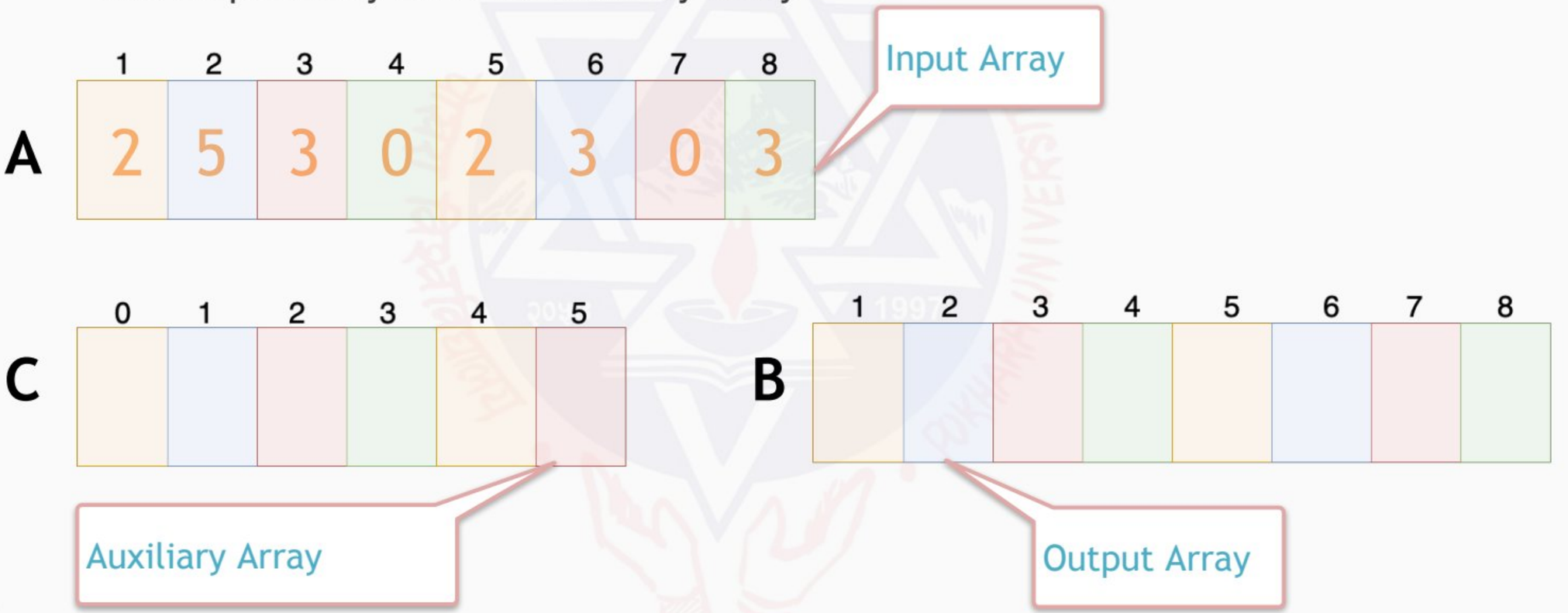
CamScanner

Counting Sort

- Example: Apply counting sort algorithm to sort the array A.

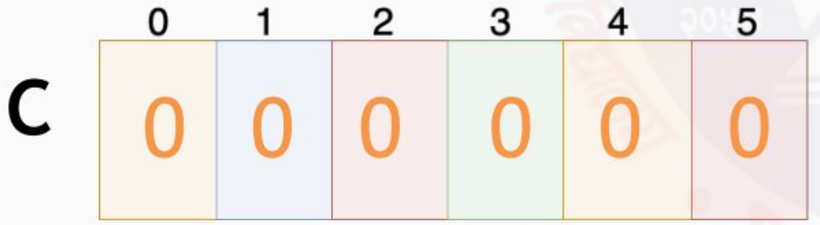
Counting Sort

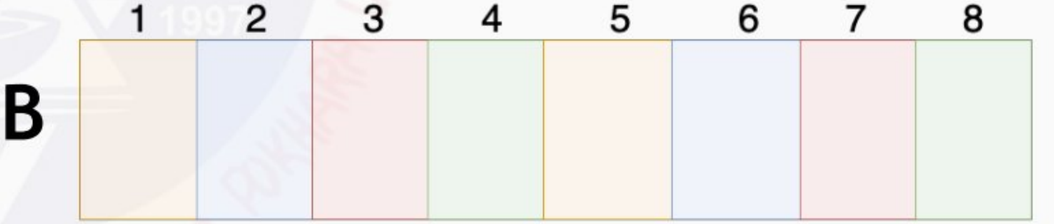
- To apply counting sort algorithm to sort the given array A, we need two extra arrays- one output array and one auxiliary array.



Counting Sort

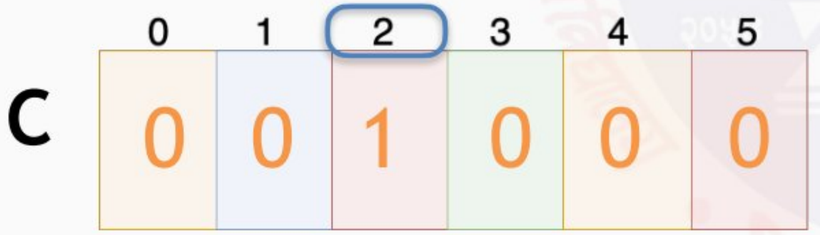
- Initialize $C[i] = 0$ for $0 \leq i \leq k$

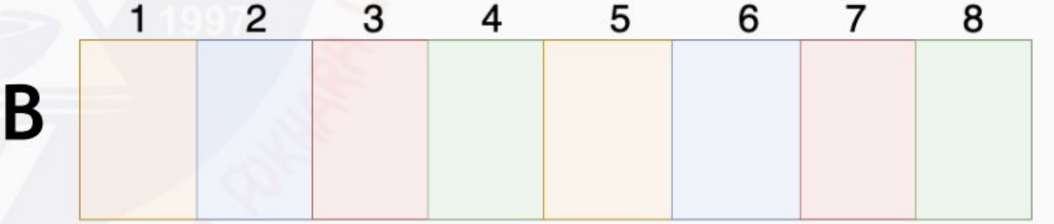




Counting Sort

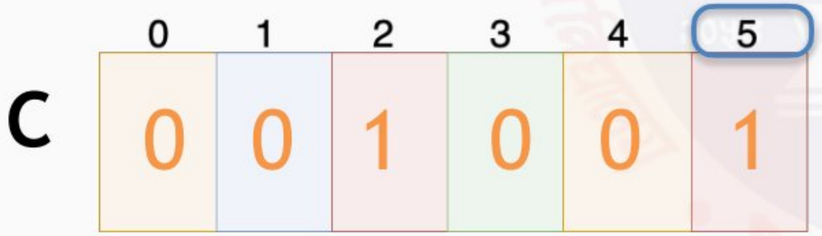
- Start updating the values of array C.
- For each element x (it is 2) of A, increment the value of array C at index x (i.e. 2) by 1

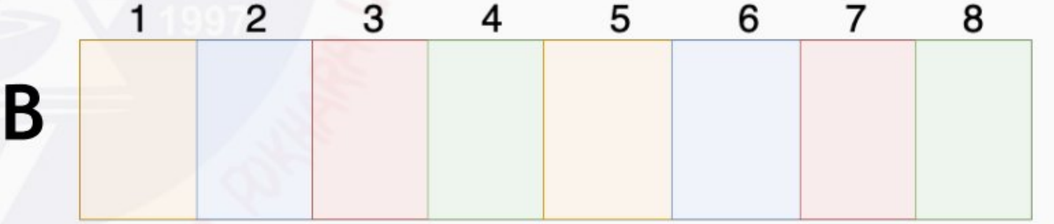




Counting Sort

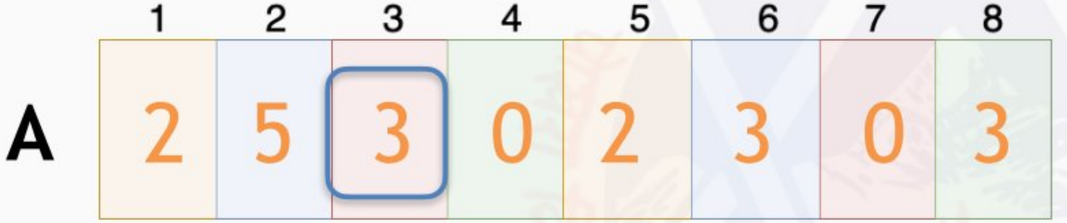
- Keep updating the values of array C.
- For each element x (it is now 5) of A, increment the value of array C at index x (i.e. 5) by 1.

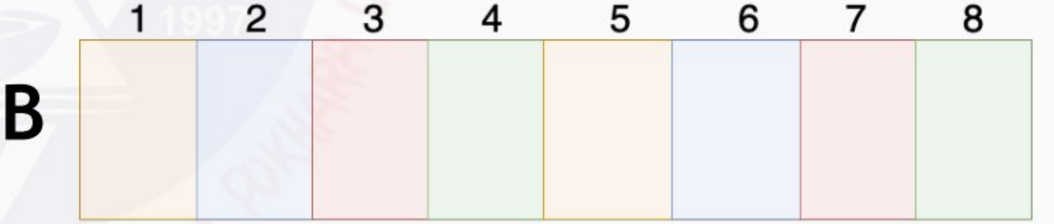




Counting Sort

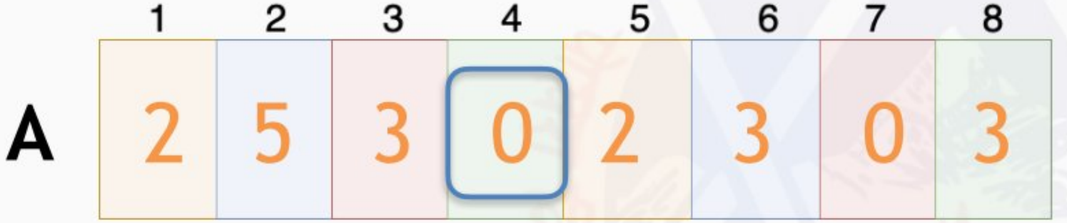
- Keep updating the values of array C.
- For each element x (it is now 3) of A, increment the value of array C at index x (i.e. 3) by 1.

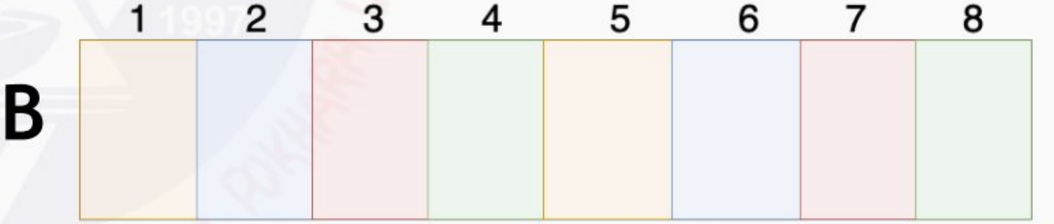




Counting Sort

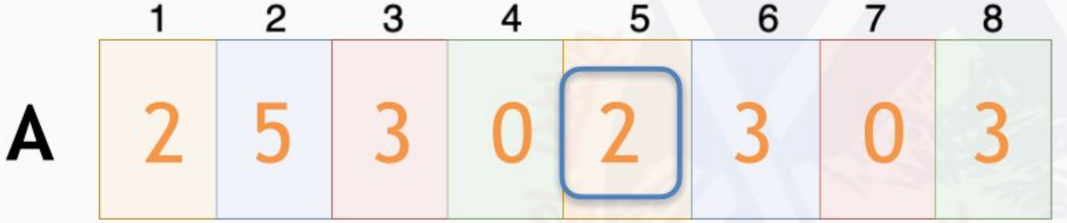
- Keep updating the values of array C.
- For each element x (it is now 0) of A, increment the value of array C at index x (i.e. 0) by 1.

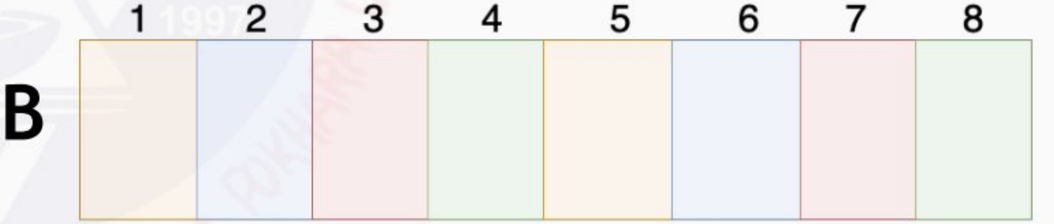




Counting Sort

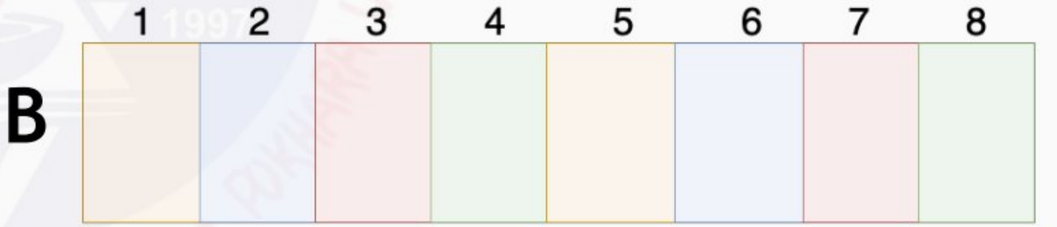
- Keep updating the values of array C.
- For each element x (it is now 2) of A, increment the value of array C at index x (i.e. 2) by 1.





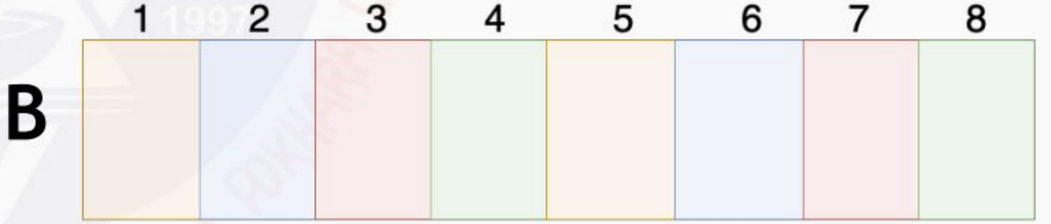
Counting Sort

- Keep updating the values of array C.
- For each element x (it is now 3) of A, increment the value of array C at index x (i.e. 3) by 1.



Counting Sort

- Keep updating the values of array C.
- For each element x (it is now 0) of A, increment the value of array C at index x (i.e. 0) by 1.

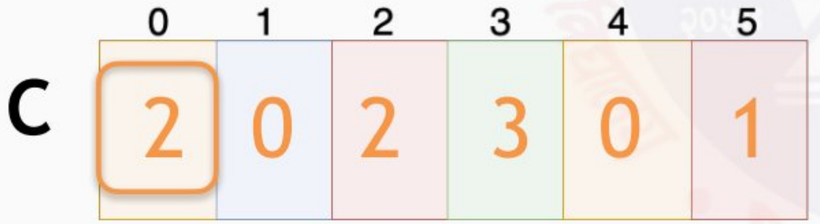


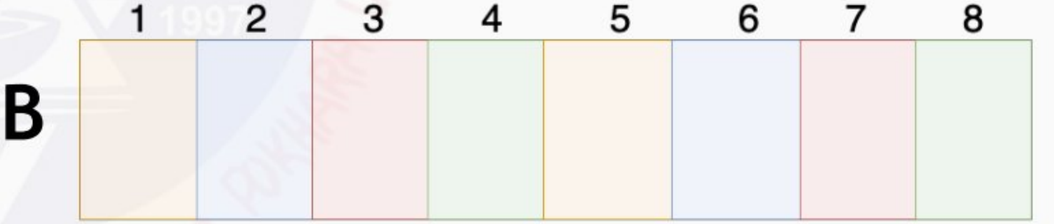
Counting Sort

- Keep updating the values of array C.
- For each element x (it is now 3) of A, increment the value of array C at index x (i.e. 3) by 1.

Counting Sort

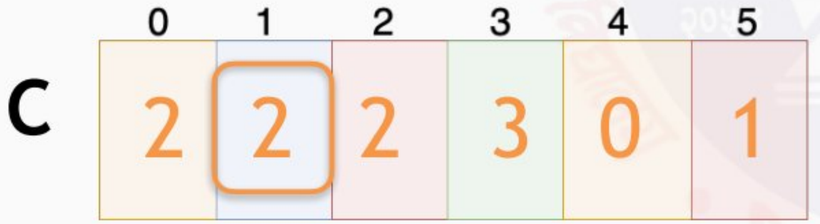
- Determine for each $i = 0,1,2,...,k$, how many input elements are less than or equal to i by keeping a running sum of the array C.

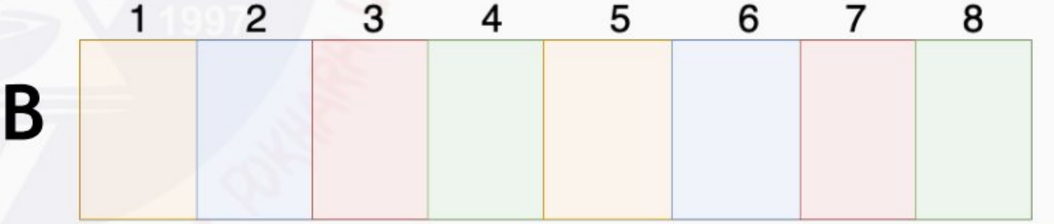




Counting Sort

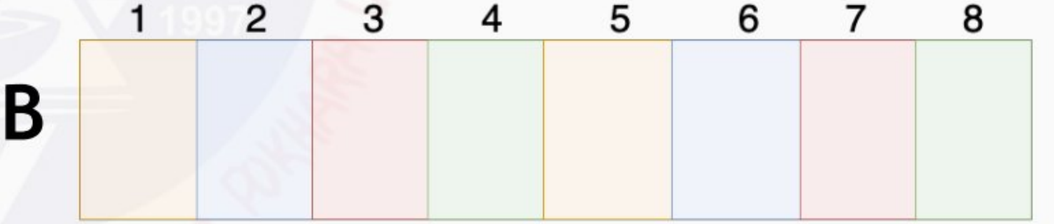
- Determine for each $i = 0,1,2,...,k$, how many input elements are less than or equal to i by keeping a running sum of the array C.





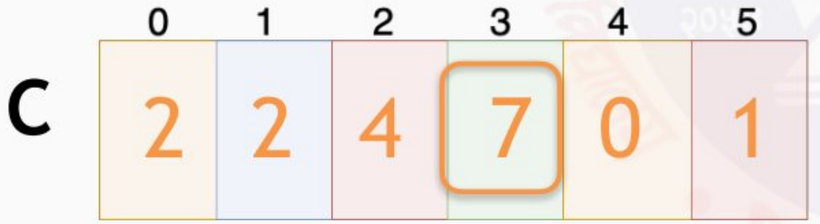
Counting Sort

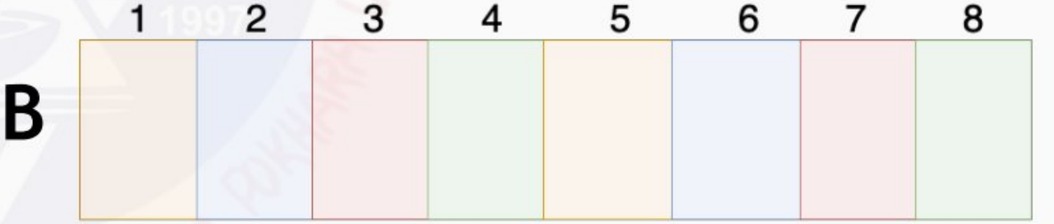
- Determine for each $i = 0,1,2,...,k$, how many input elements are less than or equal to i by keeping a running sum of the array C.



Counting Sort

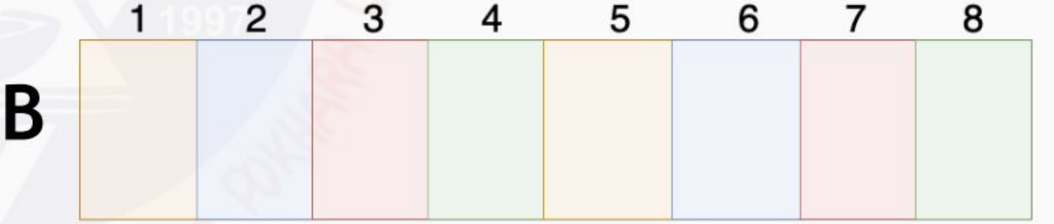
- Determine for each $i = 0,1,2,...,k$, how many input elements are less than or equal to i by keeping a running sum of the array C.





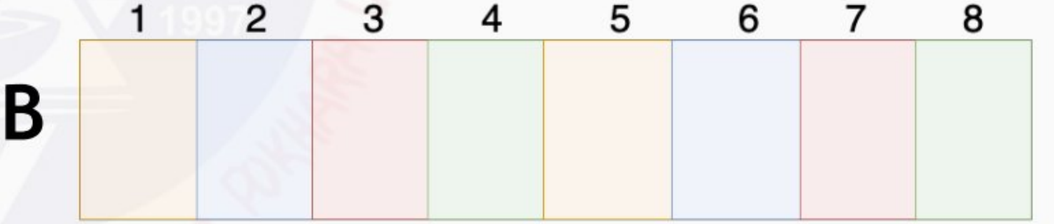
Counting Sort

- Determine for each $i = 0,1,2,...,k$, how many input elements are less than or equal to i by keeping a running sum of the array C.



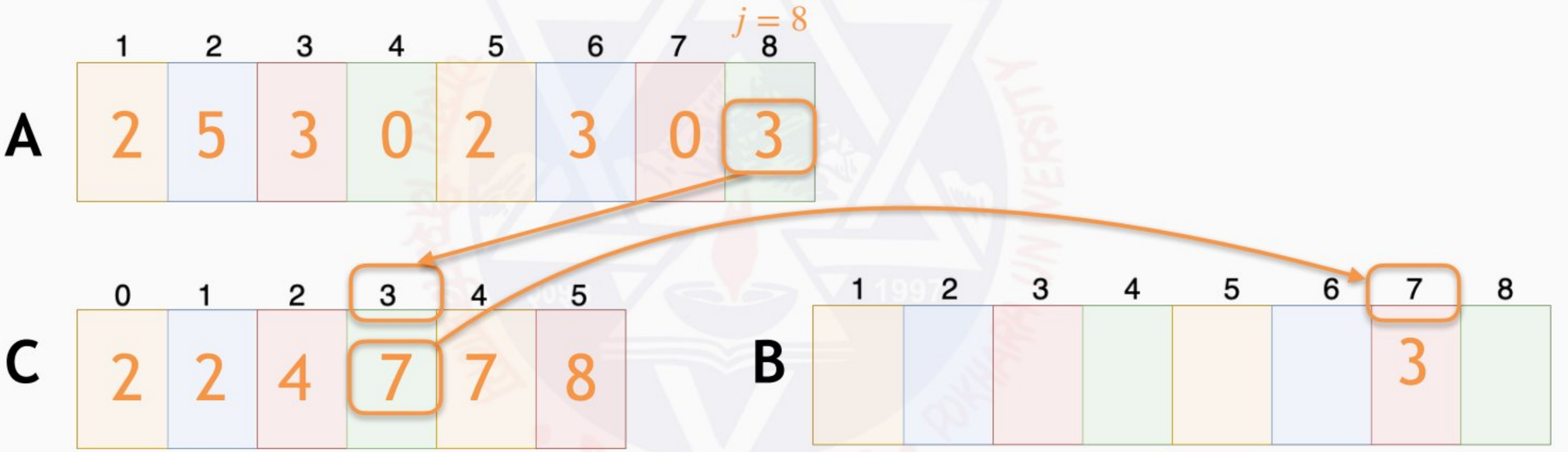
Counting Sort

- Determine for each $i = 0,1,2,...,k$, how many input elements are less than or equal to i by keeping a running sum of the array C.



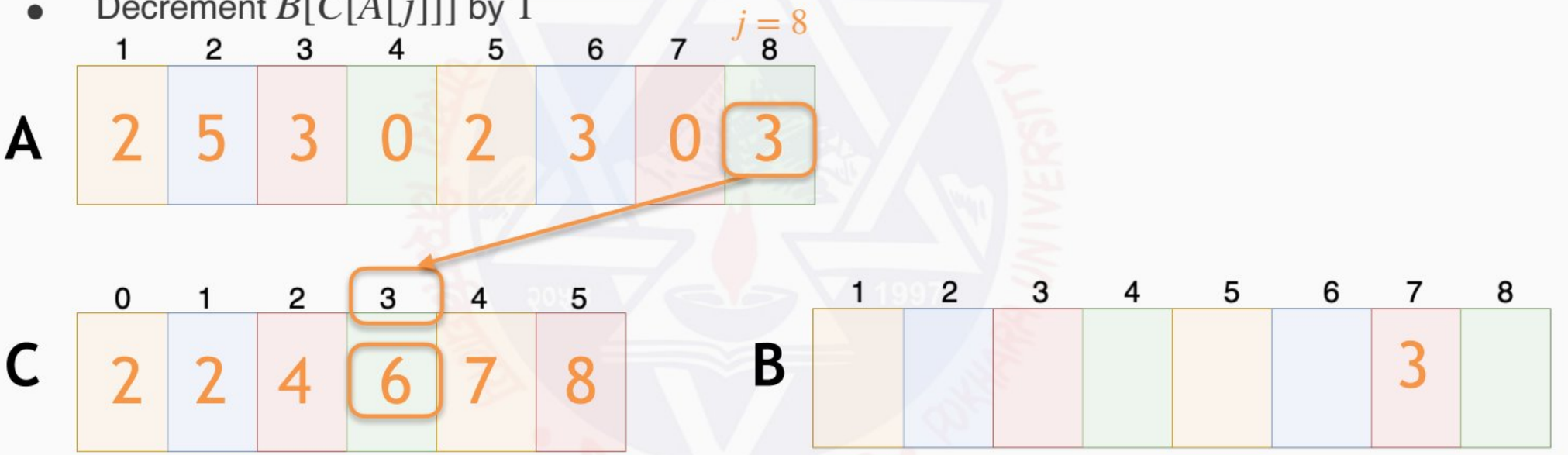
Counting Sort

- Now use the information of C, to place the elements from A at the right position at B.
- For $j = n$ to 1 in A, find $C[A[j]]$ and assign $B[C[A[j]]] = A[j]$



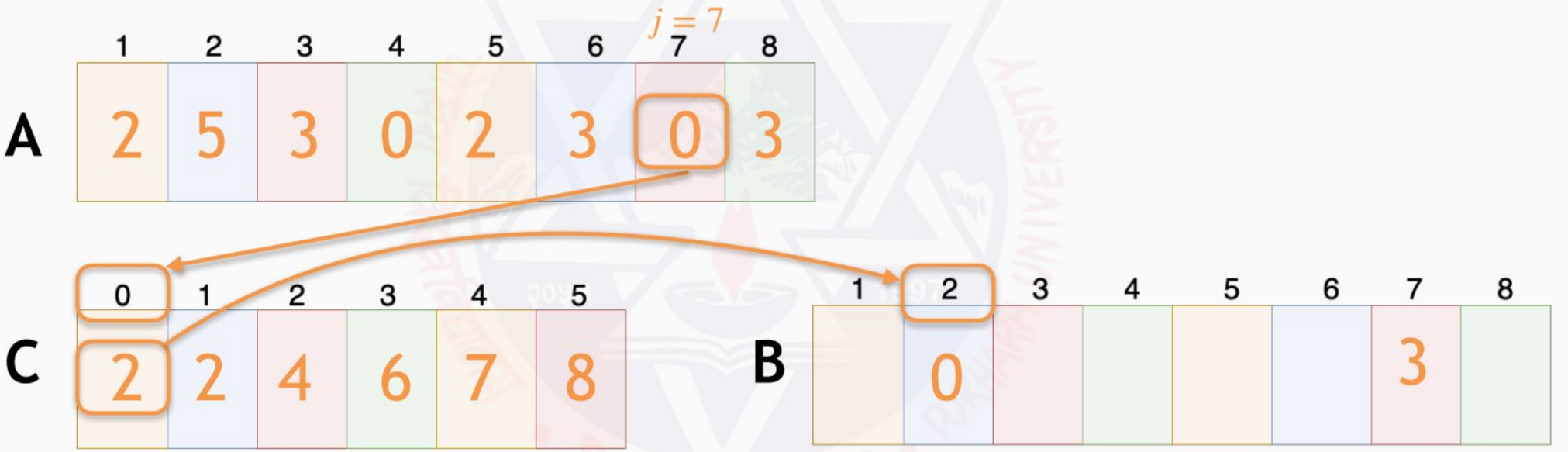
Counting Sort

- Now use the information of C, to place the elements from A at the right position at B.
- For $j = n$ to 1 in A, find $C[A[j]]$ and assign $B[C[A[j]]] = A[j]$
- Decrement $B[C[A[j]]]$ by 1



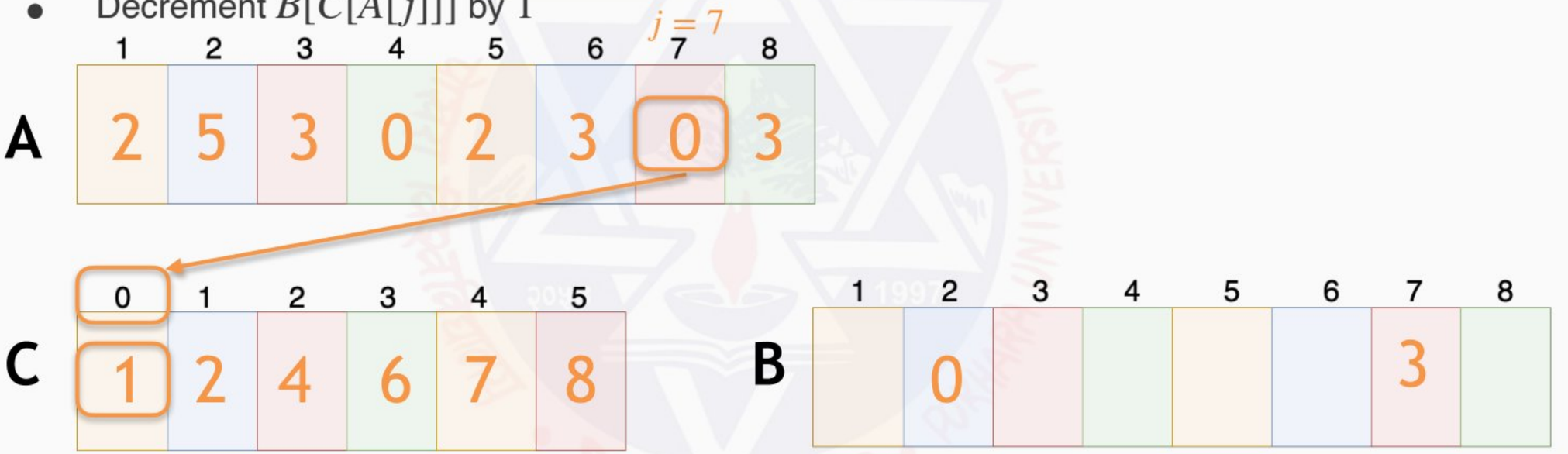
Counting Sort

- Now use the information of C, to place the elements from A at the right position at B.
- For $j = n$ to 1 in A, find $C[A[j]]$ and assign $B[C[A[j]]] = A[j]$



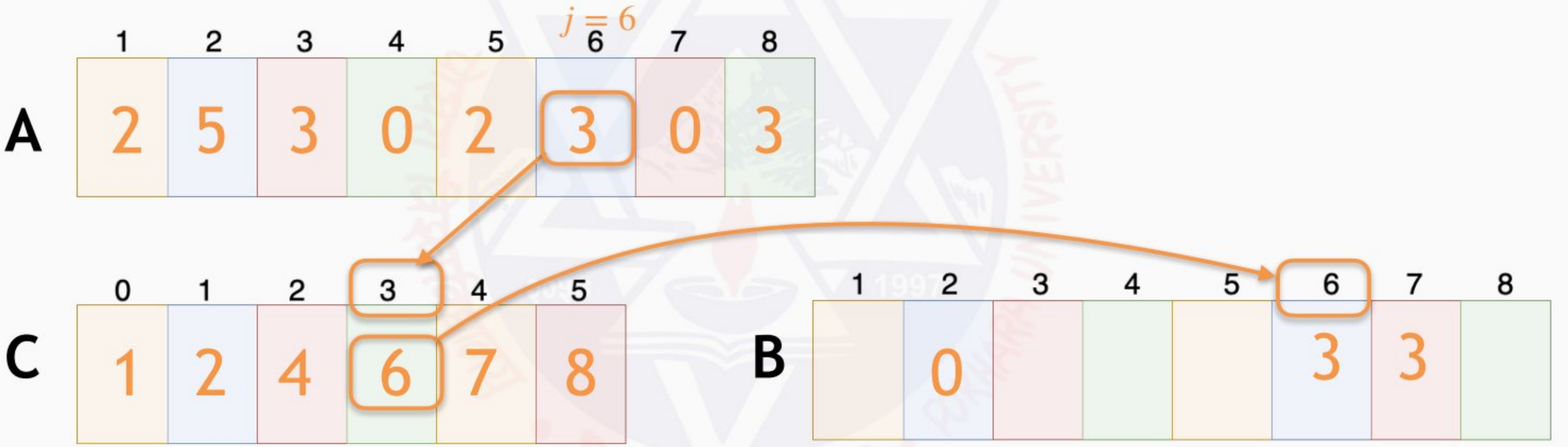
Counting Sort

- Now use the information of C, to place the elements from A at the right position at B.
- For $j = n$ to 1 in A, find $C[A[j]]$ and assign $B[C[A[j]]] = A[j]$
- Decrement $B[C[A[j]]]$ by 1



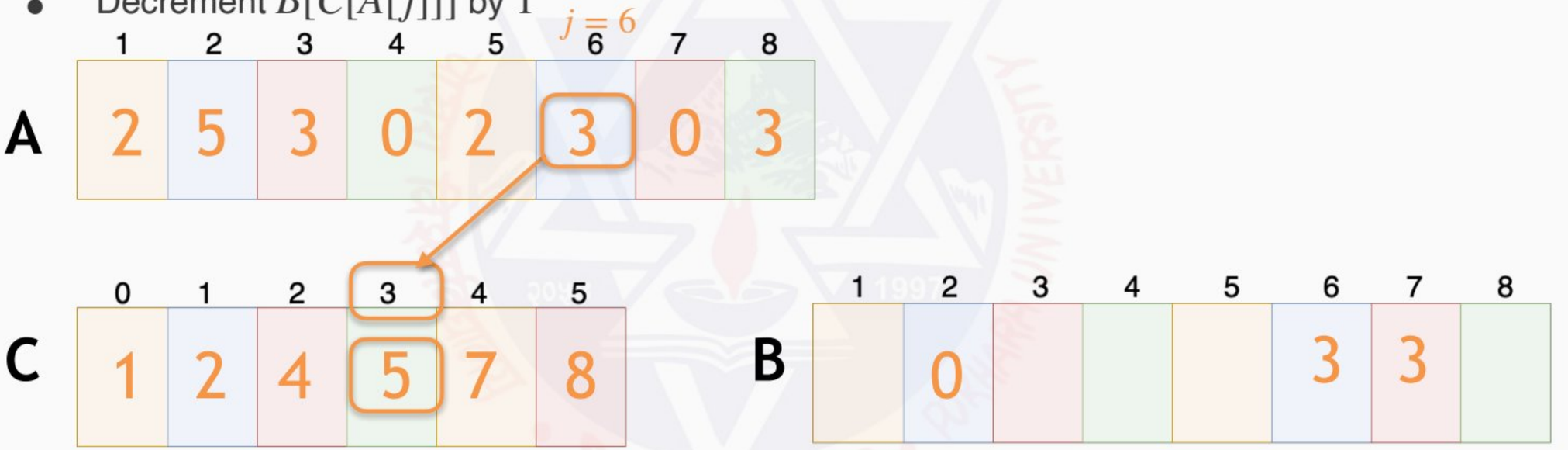
Counting Sort

- Now use the information of C, to place the elements from A at the right position at B.
- For $j = n$ to 1 in A, find $C[A[j]]$ and assign $B[C[A[j]]] = A[j]$



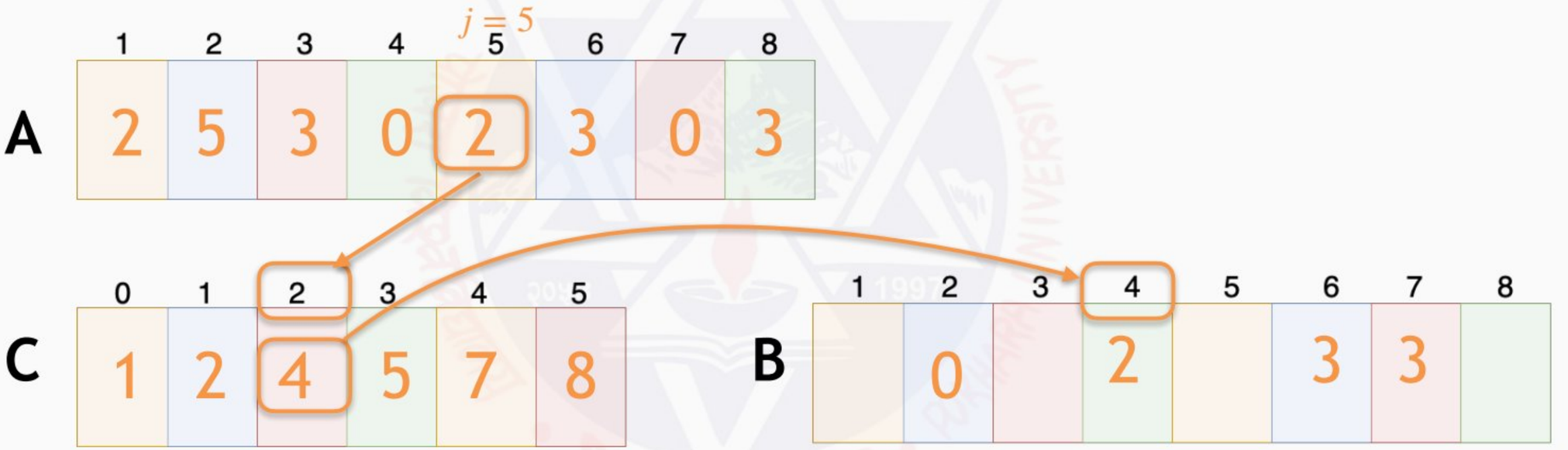
Counting Sort

- Now use the information of C, to place the elements from A at the right position at B.
- For $j = n$ to 1 in A, find $C[A[j]]$ and assign $B[C[A[j]]] = A[j]$
- Decrement $B[C[A[j]]]$ by 1



Counting Sort

- Now use the information of C, to place the elements from A at the right position at B.
- For $j = n$ to 1 in A, find $C[A[j]]$ and assign $B[C[A[j]]] = A[j]$.

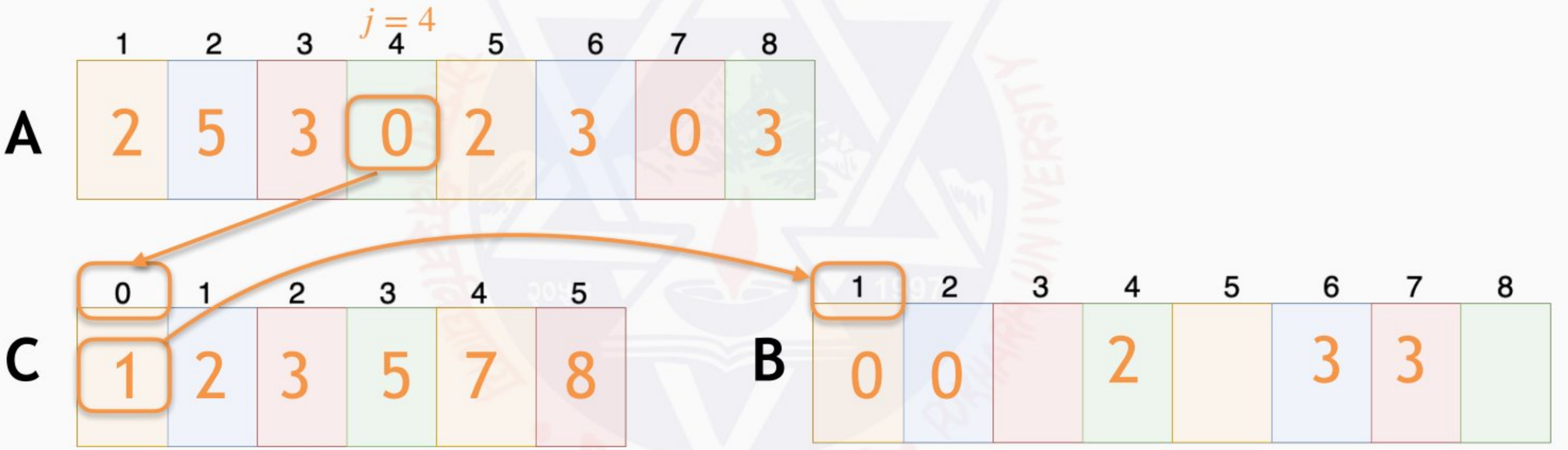


Counting Sort

- Now use the information of C, to place the elements from A at the right position at B.
- For $j = n$ to 1 in A, find $C[A[j]]$ and assign $B[C[A[j]]] = A[j]$. Decrement $B[C[A[j]]]$ by 1

Counting Sort

- Now use the information of C, to place the elements from A at the right position at B.
- For $j = n$ to 1 in A, find $C[A[j]]$ and assign $B[C[A[j]]] = A[j]$.

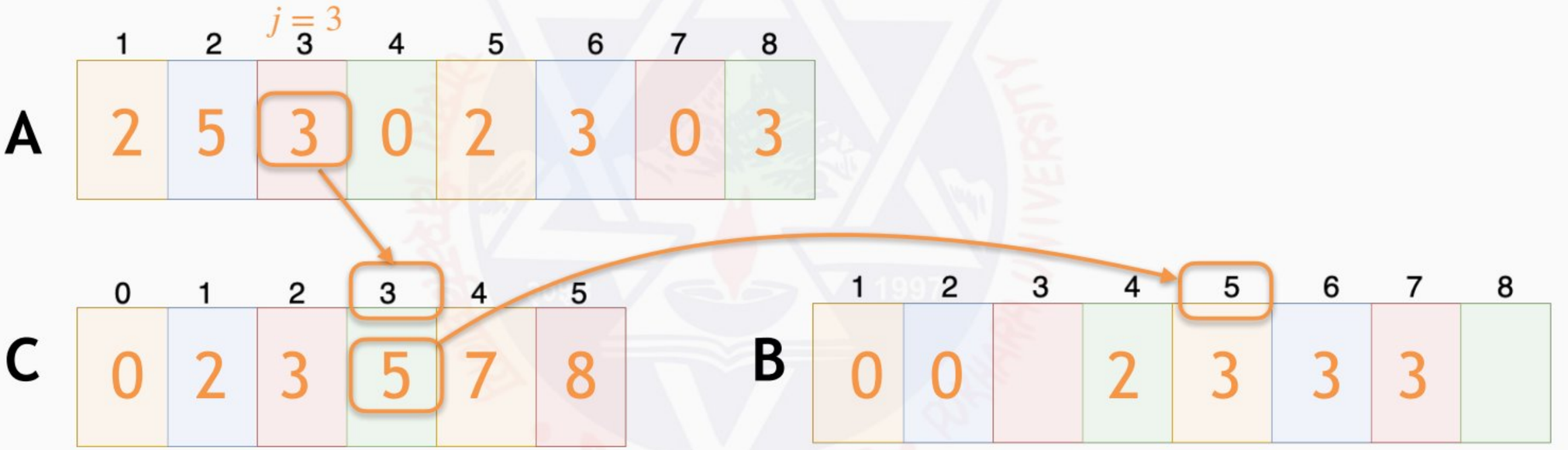


Counting Sort

- Now use the information of C, to place the elements from A at the right position at B.
- For $j = n$ to 1 in A, find $C[A[j]]$ and assign $B[C[A[j]]] = A[j]$. Decrement $B[C[A[j]]]$ by 1

Counting Sort

- Now use the information of C, to place the elements from A at the right position at B.
- For $j = n$ to 1 in A, find $C[A[j]]$ and assign $B[C[A[j]]] = A[j]$.

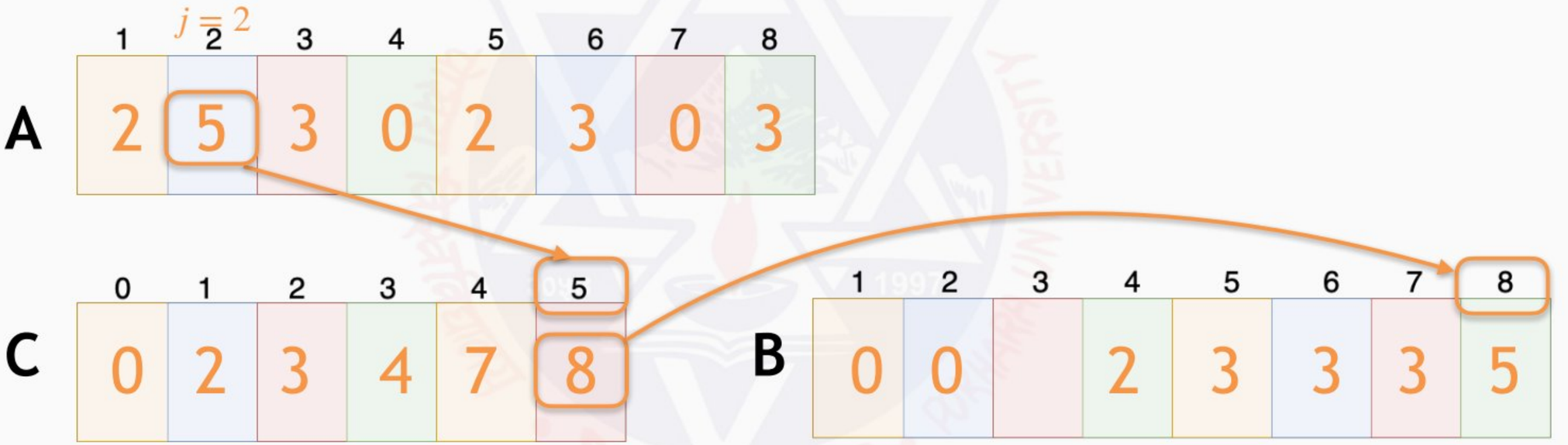


Counting Sort

- Now use the information of C, to place the elements from A at the right position at B.
- For $j = n$ to 1 in A, find $C[A[j]]$ and assign $B[C[A[j]]] = A[j]$. Decrement $B[C[A[j]]]$ by 1

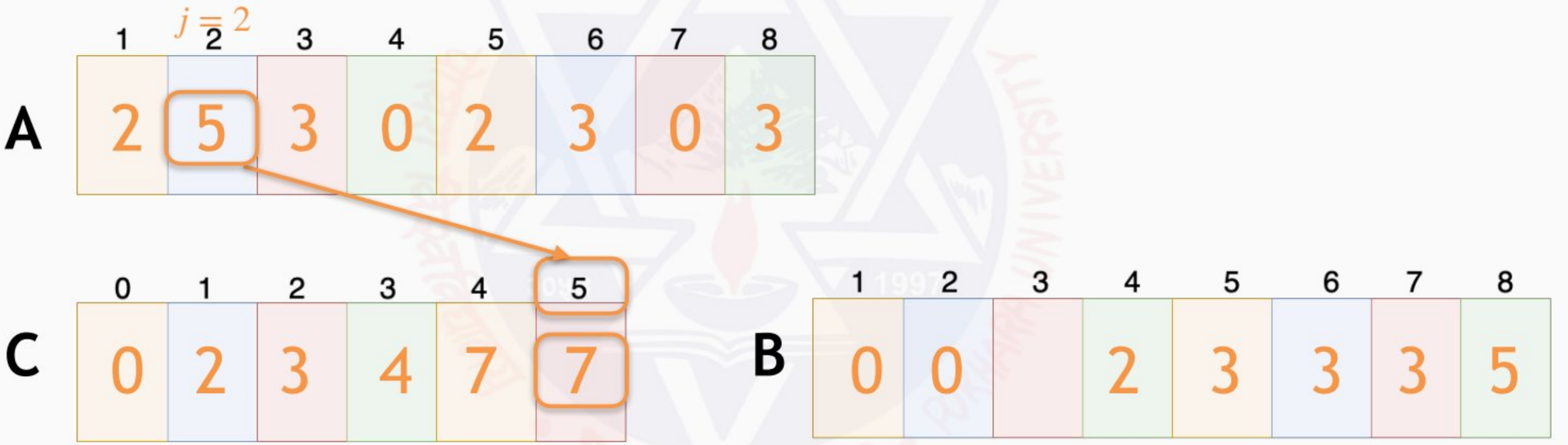
Counting Sort

- Now use the information of C, to place the elements from A at the right position at B.
- For $j = n$ to 1 in A, find $C[A[j]]$ and assign $B[C[A[j]]] = A[j]$.



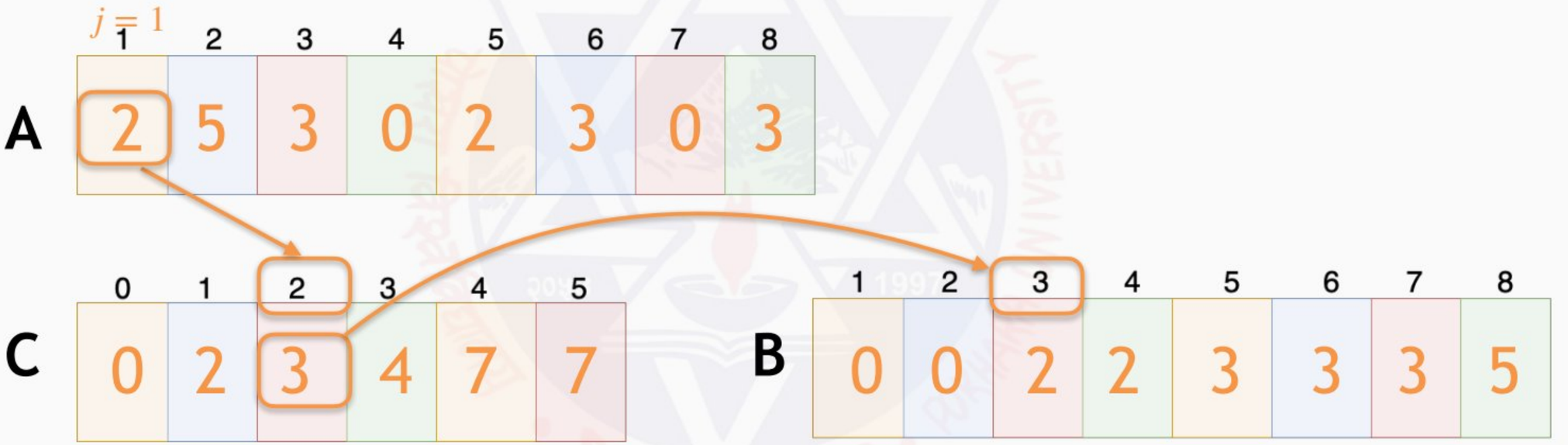
Counting Sort

- Now use the information of C, to place the elements from A at the right position at B.
- For $j = n$ to 1 in A, find $C[A[j]]$ and assign $B[C[A[j]]] = A[j]$. Decrement $B[C[A[j]]]$ by 1



Counting Sort

- Now use the information of C, to place the elements from A at the right position at B.
- For $j = n$ to 1 in A, find $C[A[j]]$ and assign $B[C[A[j]]] = A[j]$.



Counting Sort

- Now use the information of C, to place the elements from A at the right position at B.
- For $j = n$ to 1 in A, find $C[A[j]]$ and assign $B[C[A[j]]] = A[j]$. Decrement $B[C[A[j]]]$ by 1

Counting Sort

- Now use the information of C, to place the elements from A at the right position at B.
- For $j = n$ to 1 in A, find $C[A[j]]$ and assign $B[C[A[j]]] = A[j]$. Decrement $B[C[A[j]]]$ by 1

	1	2	3	4	5	6	7	8
A	2	5	3	0	2	3	0	3

	0	1	2	3	4	5
C	0	2	3	4	7	7

Sorted elements in Array B

	1	2	3	4	5	6	7	8
B	0	0	2	2	3	3	3	5

CamScanner

Counting Sort

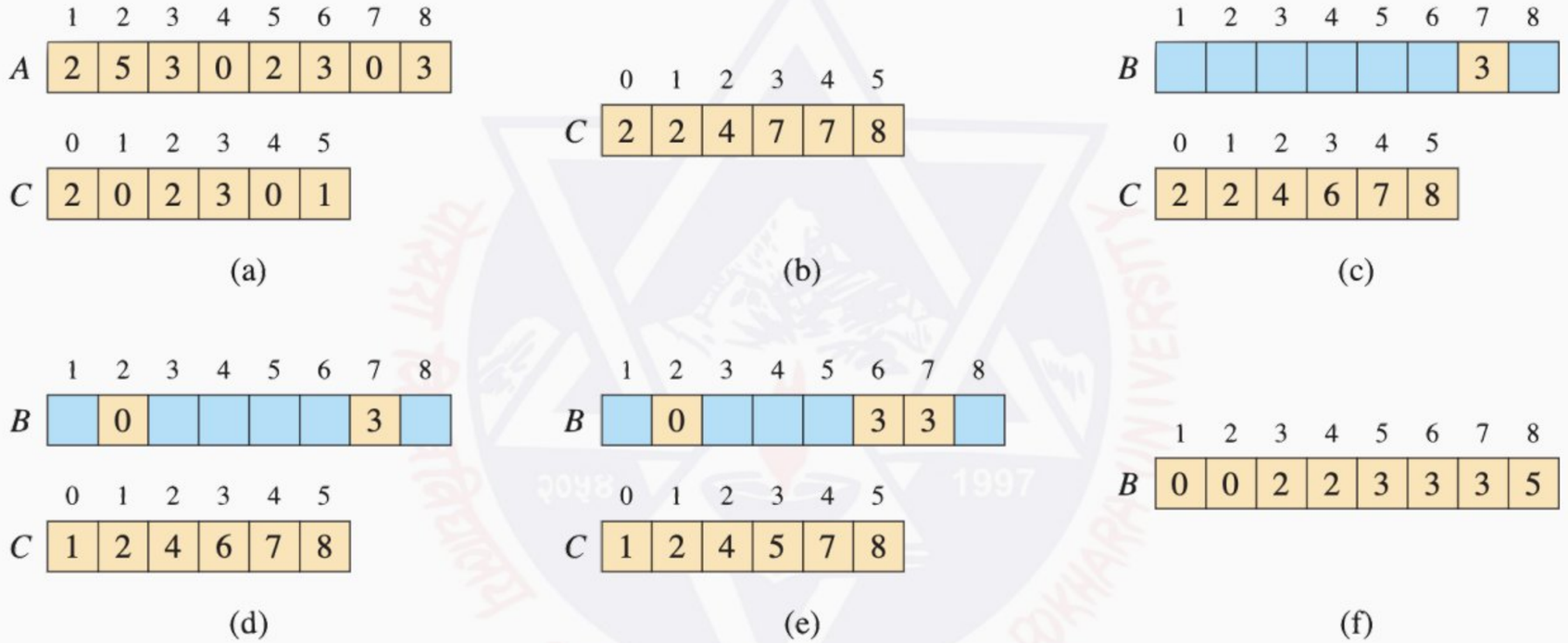


Figure 8.2 The operation of COUNTING-SORT on an input array $A[1:8]$, where each element of A is a nonnegative integer no larger than $k = 5$. **(a)** The array A and the auxiliary array C after line 5. **(b)** The array C after line 8. **(c)–(e)** The output array B and the auxiliary array C after one, two, and three iterations of the loop in lines 11–13, respectively. Only the tan elements of array B have been filled in. **(f)** The final sorted output array B.

CamScanner

Counting Sort

```
COUNTING-SORT(A, n, k)
1   let B[1 : n] and C[0 : k] be new arrays
2   for i = 0 to k
3       C[i] = 0
4   for j = 1 to n
5       C[A[j]] = C[A[j]] + 1
6   // C[i] now contains the number of elements equal to i.
7   for i = 1 to k
8       C[i] = C[i] + C[i − 1]
9   // C[i] now contains the number of elements less than or equal to i.
10  // Copy A to B, starting from the end of A.
11  for j = n downto 1
12      B[C[A[j]]] = A[j]
13      C[A[j]] = C[A[j]] − 1    // to handle duplicate values
14  return B
```


CamScanner

Counting Sort

```
COUNTING-SORT(A, n, k)
1  let B[1 : n] and C[0 : k] be new arrays
2  for i = 0 to k
3      C[i] = 0
4  for j = 1 to n
5      C[A[j]] = C[A[j]] + 1
6  // C[i] now contains the number of elements equal to i.
7  for i = 1 to k
8      C[i] = C[i] + C[i − 1]
9  // C[i] now contains the number of elements less than or equal to i.
10 // Copy A to B, starting from the end of A.
11 for j = n downto 1
12     B[C[A[j]]] = A[j]
13     C[A[j]] = C[A[j]] − 1    // to handle duplicate values
14 return B
```

Lines 2 & 3 takes $\Theta(k)$ times.

CamScanner

Counting Sort

```
COUNTING-SORT(A, n, k)
1  let B[1 : n] and C[0 : k] be new arrays
2  for i = 0 to k
3      C[i] = 0
4  for j = 1 to n
5      C[A[j]] = C[A[j]] + 1
6  // C[i] now contains the number of elements equal to i.
7  for i = 1 to k
8      C[i] = C[i] + C[i − 1]
9  // C[i] now contains the number of elements less than or equal to i.
10 // Copy A to B, starting from the end of A.
11 for j = n downto 1
12     B[C[A[j]]] = A[j]
13     C[A[j]] = C[A[j]] − 1    // to handle duplicate values
14 return B
```

Lines 2 & 3 takes $\Theta(k)$ times.

Lines 4 & 5 takes $\Theta(n)$ times.

CamScanner

Counting Sort

```
COUNTING-SORT(A, n, k)
1  let B[1 : n] and C[0 : k] be new arrays
2  for i = 0 to k
3      C[i] = 0
4  for j = 1 to n
5      C[A[j]] = C[A[j]] + 1
6  // C[i] now contains the number of elements equal to i.
7  for i = 1 to k
8      C[i] = C[i] + C[i − 1]
9  // C[i] now contains the number of elements less than or equal to i.
10 // Copy A to B, starting from the end of A.
11 for j = n downto 1
12     B[C[A[j]]] = A[j]
13     C[A[j]] = C[A[j]] − 1    // to handle duplicate values
14 return B
```

Lines 2 & 3 takes $\Theta(k)$ times.

Lines 4 & 5 takes $\Theta(n)$ times.

Lines 7 & 8 takes $\Theta(k)$ times.

CamScanner

Counting Sort

```
COUNTING-SORT(A, n, k)
1  let B[1 : n] and C[0 : k] be new arrays
2  for i = 0 to k
3      C[i] = 0
4  for j = 1 to n
5      C[A[j]] = C[A[j]] + 1
6  // C[i] now contains the number of elements equal to i.
7  for i = 1 to k
8      C[i] = C[i] + C[i − 1]
9  // C[i] now contains the number of elements less than or equal to i.
10 // Copy A to B, starting from the end of A.
11 for j = n downto 1
12     B[C[A[j]]] = A[j]
13     C[A[j]] = C[A[j]] − 1    // to handle duplicate values
14 return B
```

Lines 2 & 3 takes $\Theta(k)$ times.

Lines 4 & 5 takes $\Theta(n)$ times.

Lines 7 & 8 takes $\Theta(k)$ times.

Lines 11-13 takes $\Theta(n)$ times.

CamScanner

Counting Sort

Overall time $T(n)$ is given by
$T(n) = \Theta(k + n)$

If $k = O(n)$, then
$T(n) = \Theta(n)$

The counting sort is **stable**.

Lines 2 & 3 takes $\Theta(k)$ times.

Lines 4 & 5 takes $\Theta(n)$ times.

Lines 7 & 8 takes $\Theta(k)$ times.

Lines 11-13 takes $\Theta(n)$ times.

Assumption:

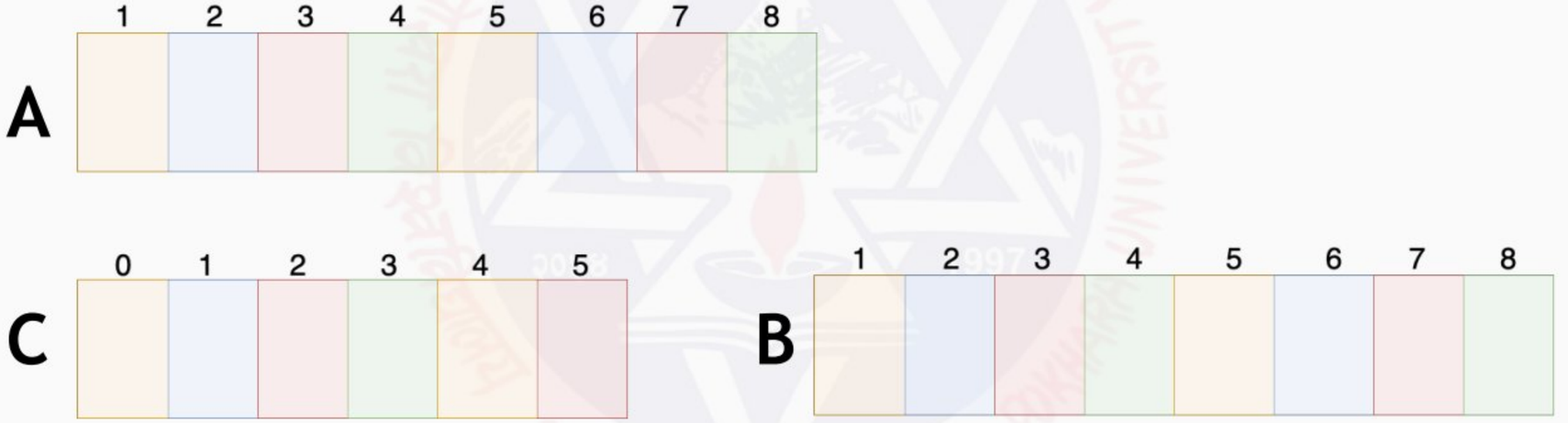
The counting sort assumes that the input consists of integers in a small range

CamScanner

Counting Sort

Assumption:

The counting sort assumes that the input consists of integers in a small range



Why?

Bucket Sort

- Imagine that for the past year, as you paid your various bills, you then simply piled all the paperwork onto the top of a table somewhere.
- Now the year has ended and its time to sort all of these papers by what the bill was for (electricity, phone, rent, etc.) and date.
- A pretty natural approach are
  - ***Approach1***: is to make some space on the floor, and as you go through the pile of papers, put the phone bills into one pile, the electric bills into another pile, and so on. Once this initial assignment of bills to piles is done (in one pass), you can sort each pile by date relatively quickly because they are each fairly small. (**Basic idea of Binsort**)
  - ***Approach 2***: is to make 12 bins (or buckets)- Baishak, Jestha, Tsar, … , and Chaitra and as you go through the pile of papers, put the bills payed on Jestha into Jesth bin, and so on. Once this initial assignment of bills to 12 bins is done (in one pass), you can sort each bin by date relatively quickly because they are each fairly small. Then finally. (Basic idea behind the **bucket sort and radix sort)**.

CamScanner

Bucket Sort

- Assumption:
  - The bucket sort assumes something about the input.
  - The input are uniformly distributed into each ***bucket***.
- Bucket sort works as follows:
  - It divides the interval $[0,1)$ into n equal-sized subintervals, or ***buckets.***
  - It distributes the input numbers into the buckets. (Since the inputs are uniformly and independently distributed over $[0,1)$, we do not expect many numbers to fall into each bucket.)
  - Sort the numbers in each bucket using any appropriate sorting algorithms
  - List the numbers from the ***buckets*** going through in order.

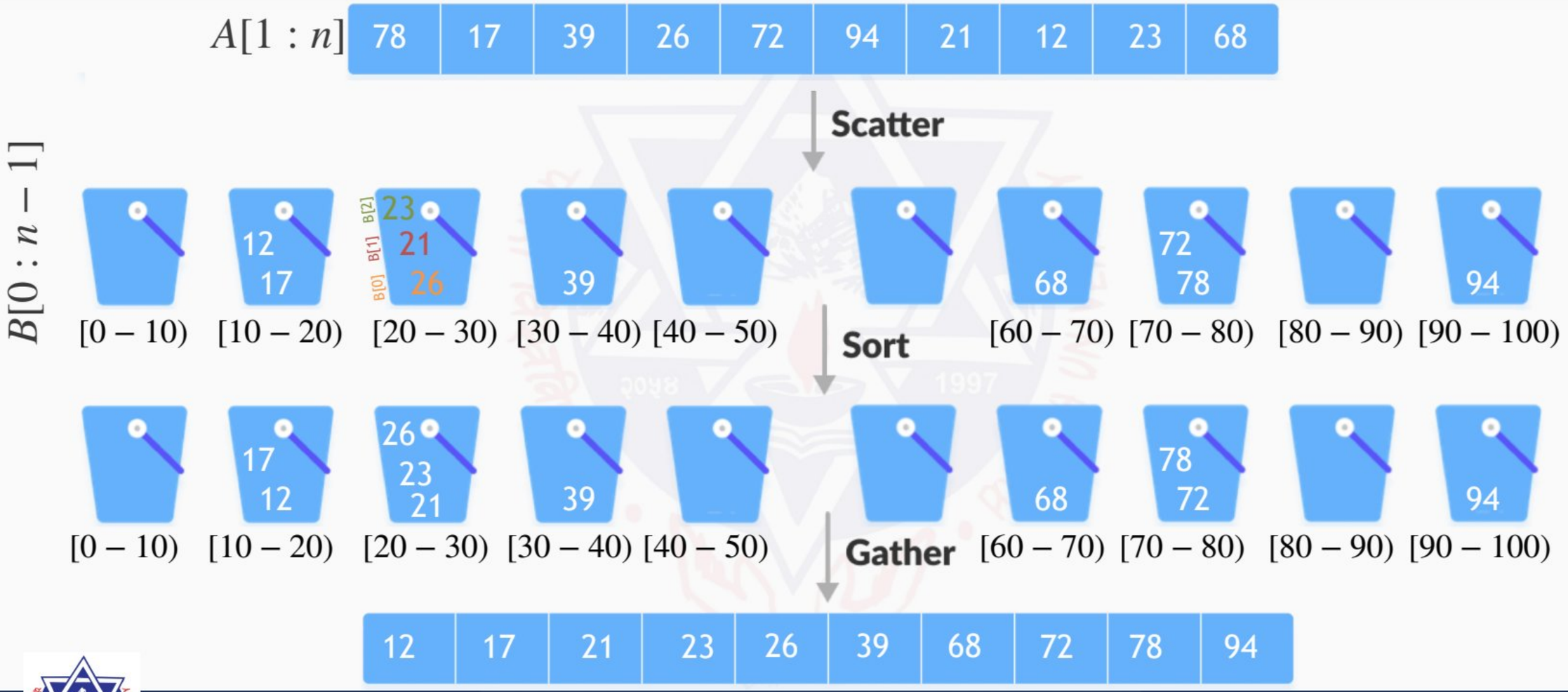
CamScanner

Bucket Sort

- Bucket sort works as follows:
  - The $BUCKET - SORT$ procedure assumes that the input is an array $A[1 : n]$ and that each element $A[i]$ in the array satisfies $0 \leq A[i] < 1$.
  - The code requires an auxiliary array $B[0 : n - 1]$ of linked lists (buckets) and assumes that there is a mechanism for maintaining such lists.

CamScanner

Bucket Sort


A[1 : n]
78 17 39 26 72 94 21 12 23 68
Scatter
B[0 : n − 1]
B[2]
B[1]
B[0]
23
21
26
12
17
39
72
78
68
94
[0 – 10)
[10 – 20)
[20 – 30)
[30 – 40)
[40 – 50)
[60 – 70)
[70 – 80)
[80 – 90)
[90 – 100)
Sort
17
12
26
23
21
Gather
12 17 21 23 26 39 68 72 78 94

Bucket Sort

```
BUCKET-SORT(A, n)
1  let B[0 : n − 1] be a new array
2  for i = 0 to n − 1
3      make B[i] an empty list
4  for i = 1 to n
5      insert A[i] into list B[⌊n · A[i]⌋]
6  for i = 0 to n − 1
7      sort list B[i] with insertion sort
8  concatenate the lists B[0], B[1], . . . , B[n − 1] together in order
9  return the concatenated lists
```

CamScanner

Bucket Sort

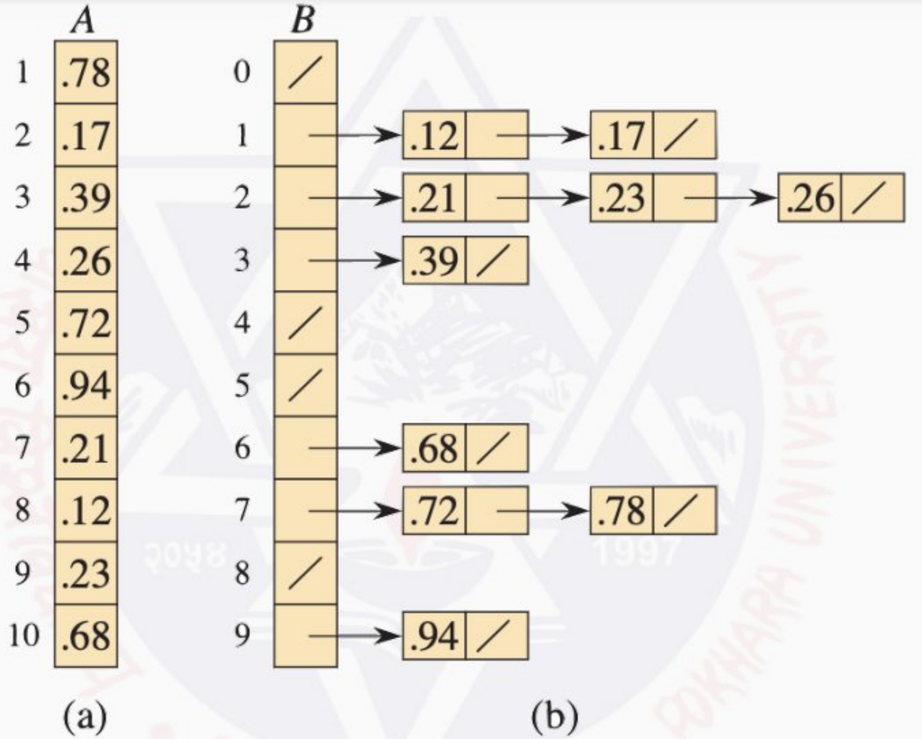


Figure 8.4 The operation of BUCKET-SORT for $n = 10$. **(a)** The input array $A[1:10]$. **(b)** The array $B[0:9]$ of sorted lists (buckets) after line 7 of the algorithm, with slashes indicating the end of each bucket. Bucket i holds values in the half-open interval $[i/10, (i+1)/10)$. The sorted output consists of a concatenation of the lists $B[0], B[1], \ldots, B[9]$ in order.

CamScanner

Analysis of Bucket Sort

```
BUCKET-SORT(A, n)
1  let B[0 : n − 1] be a new array
2  for i = 0 to n − 1
3      make B[i] an empty list
4  for i = 1 to n
5      insert A[i] into list B[⌊n · A[i]⌋]
6  for i = 0 to n − 1
7      sort list B[i] with insertion sort
8  concatenate the lists B[0], B[1], . . . , B[n − 1] together in order
9  return the concatenated lists
```

Lines 2 & 3 takes $O(n)$ times.

Lines 4 & 5 takes $O(n)$ times.

Need to analysis cost for sorting elements in each small buckets.

CamScanner

Analysis of Bucket Sort

- **Worst Case:**
  - In our previous example we made n buckets. So line 7 is repeated for n time to sort elements of all n buckets.
  - Worst case occurs when all the numbers to be sorted falls into a single bucket.
  - In such case, the line 6 is executed for n times and the line 7 calls the insertion sort for n buckets and one of the bucket contains all the n numbers of the input array. The cost of line 6 and 7 is therefore $n \,.\, O(n^2)$.

$$T(n) = \Theta(n) + \sum_{i=0}^{n-1} O(n^2)$$

$$= \Theta(n) + n \,.\, O(n^2)$$

$$= O(n^2)$$

CamScanner

Analysis of Bucket Sort

- **Average Case:**
  - Average case occurs when all the buckets contains almost equal elements of the input array. That is all the elements are uniformly distributed in all buckets.
  - Suppose we have k buckets, therefore each bucket has approximately $\frac{n}{k}$ elements.

    So insertion sort takes $O((\frac{n}{k})^2)$ time to sort each bucket.

    Therefore, cost of sorting all k buckets using insertion sort = $O(k \,.\, (\frac{n}{k})^2)$

    When k approaches to n, $O(k \,.\, (\frac{n}{k})^2) = O(n \,.\, (\frac{n}{n})^2) = O(n)$) (In our example we made $k = n$ buckets to sort n elements of the given array. In average each bucket contains one element)

    Now, $T(n) = \Theta(n) + O(k \,.\, (\frac{n}{k})^2) = \Theta(n) + O(n) = \Theta(n)$.

    $\therefore T(n) = \Theta(n)$

CamScanner

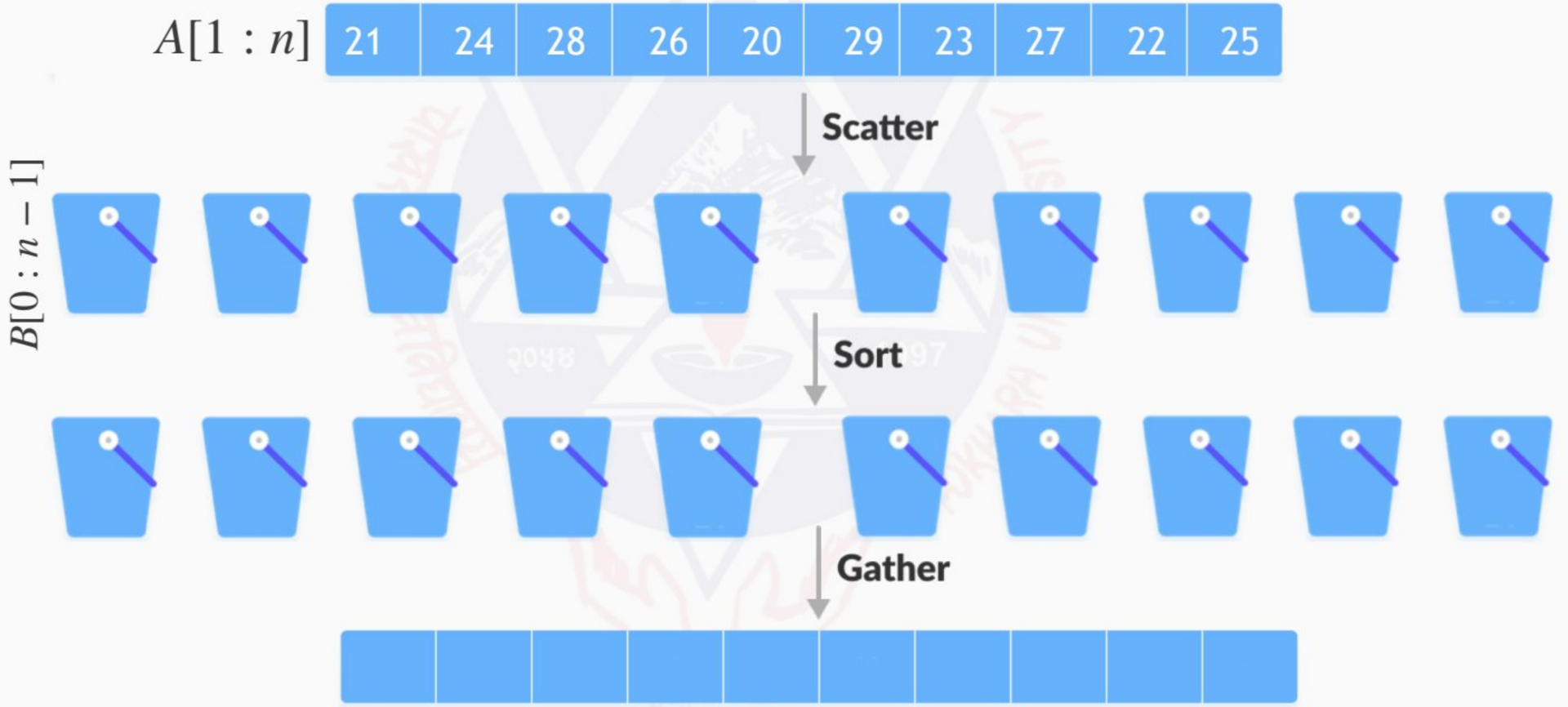
Analysis of Bucket Sort

- Like counting sort, bucket sort is fast.
- Bucket sort has an average-case running time of $O(n)$.
- As long as the input has the property that the sum of the squares of the bucket sizes is linear in the total number of elements, the bucket sort runs in linear time.
- Bucket Sort (Summary)
  - Best Case: $\Omega(n)$
  - Average Case: $\Theta(n)$
  - Worst Case: $O(n)$

CamScanner

Analysis of Bucket Sort

- Sort the elements using bucket sort and find the cost.



Radix Sort

- Radix Sort is a non-comparison based sorting algorithm.
- It is a further variation of binsort to sort integer numbers based on their radix.
- It is called Radix sort because the bin computations are based on the radix or the base of numbers to be sorted.
- This sorting algorithm can be applied any number of elements in an array.
- We simply assign records to bins based on the numbers' digit values working from the rightmost digit (least significant digit) to the leftmost (most significant digit).
- For decimal integers, it requires k bins.
- If d is the number of digits of the largest number in the array, it requires d iteration. In each iteration, generally counting sort is applied to sort each k bins.

CamScanner

Radix Sort

The code for radix sort is straightforward. The RADIX-SORT procedure assumes that each element in array $A[1:n]$ has d digits, where digit 1 is the lowest-order digit and digit d is the highest-order digit.

```
RADIX-SORT(A, n, d)
1  for i = 1 to d
2      use a stable sort to sort array A[1:n] on digit i
```

Although the pseudocode for RADIX-SORT does not specify which stable sort to use, COUNTING-SORT is commonly used.

CamScanner

Radix Sort

Figure 8.3 The operation of radix sort on seven 3-digit numbers. The leftmost column is the input. The remaining columns show the numbers after successive sorts on increasingly significant digit positions. Tan shading indicates the digit position sorted on to produce each list from the previous one.

CamScanner

Radix Sort

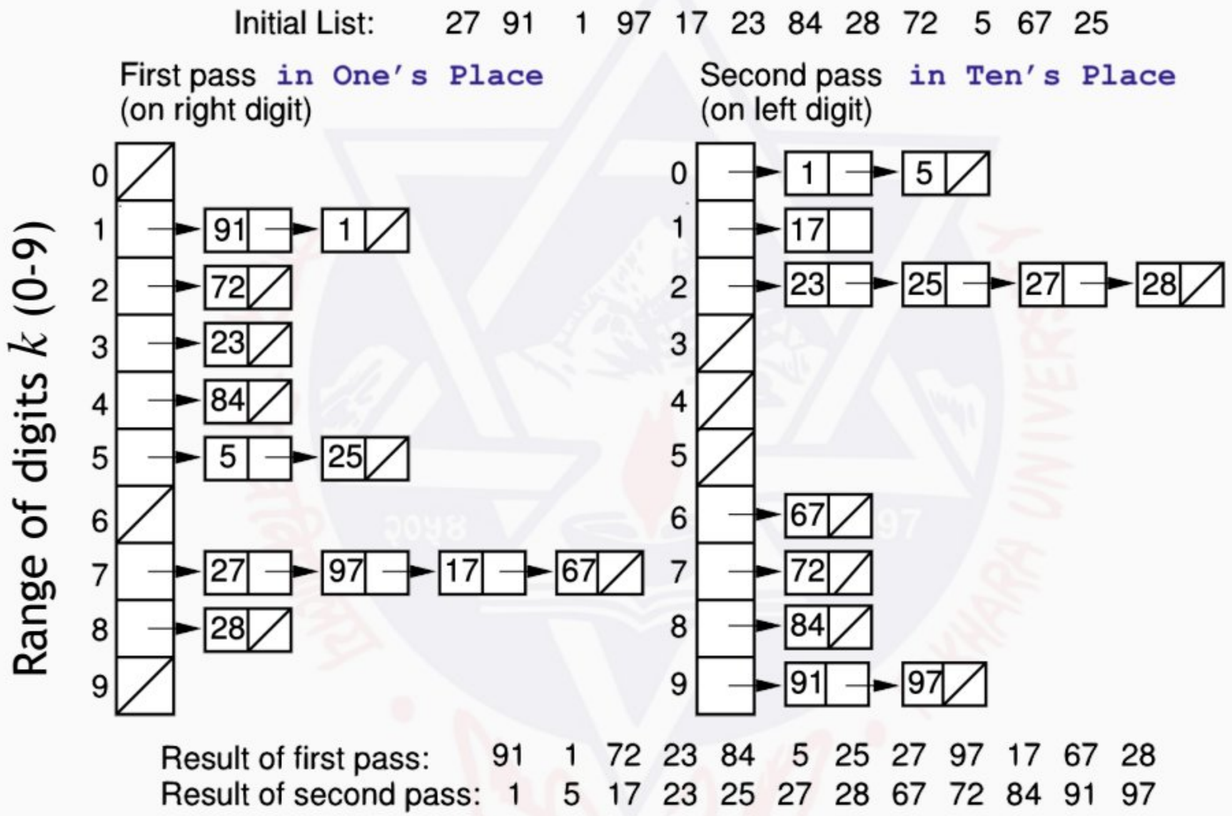


Figure 7.11 An example of Radix Sort for twelve two-digit numbers in base ten. Two passes are required to sort the list.

CamScanner

Analysis of Radix Sort

- Suppose you have array $A[1 : n]$ containing n numbers in which the largest number has d -digits.
- Each digit can take on up to k possible values (Where k is range of digits 0-9 in each pass, so k = 10)

```
RADIX-SORT(A, n, d)
1   for i = 1 to d
2       use a stable sort to sort array A[1 : n] on digit i
```

It is repeated for d times.

Lines 2 takes $\Theta(n + k)$ times in each i^{th} iteration.

- ∴ The total running time $T(n) = \Theta(d \,.\, (n + k))$
- Space Complexity: $O(n + k)$ where $k = 10$

CamScanner

Radix Sort

- **Advantages:**
  - Stable
  - Efficient
- **Disadvantages:**
  - Works only for integers and strings
  - Requires additional space to apply counting sort
- **Applications:**
  - Very efficient to sort large integers
  - Very efficient to sort strings

CamScanner

1. Multicore Systems
2. Distributed Systems

PARALLEL SORTING ALGORITHMS

CamScanner

Parallel Algorithms

- **Serial algorithms**
  - You studied many serial algorithms- merge sort, quick sort etc
  - These are suitable for running on a uniprocessor computer (executes only one instruction at a time)
- **Parallel algorithms**
  - multiple instructions can execute simultaneously in multiple cores
  - **Objective**: to solve computational problems more efficiently by dividing the workload across multiple processing units (e.g., CPU cores or distributed systems)
- **Multicore Programming**
  - depending on how computations are distributed across multiple cores or processors.
    - Thread Parallelism
    - Task Parallelism

CamScanner

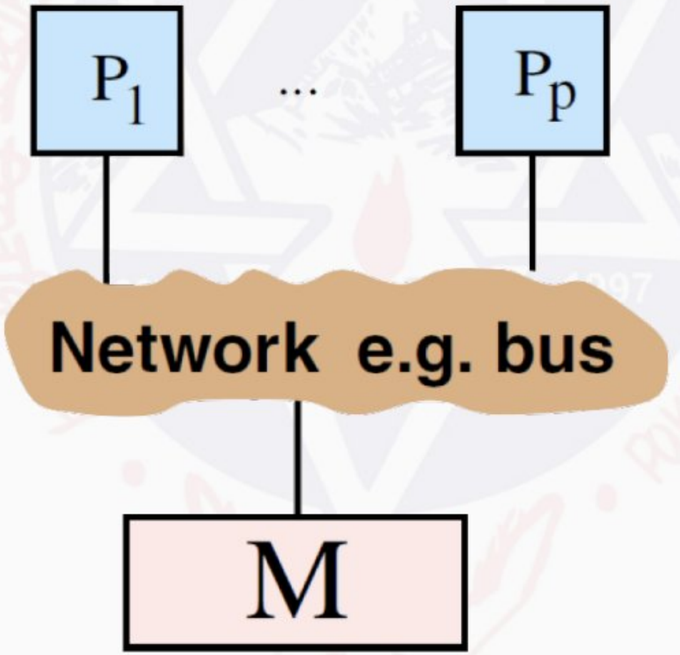
Parallel Computer

- **A Parallel Computer**
  - **Two or more processors** that can cooperate and communicate to solve a large problem faster
  - **One or more memory modules**
  - An **interconnection network** that connects processors with each other and/or with the memory modules
- **Multi-processor**
  - Tightly connected processors (***Shared Memory***)
- **Multi-computer**
  - Loosely connected processors (***Distributed Memory***)

CamScanner

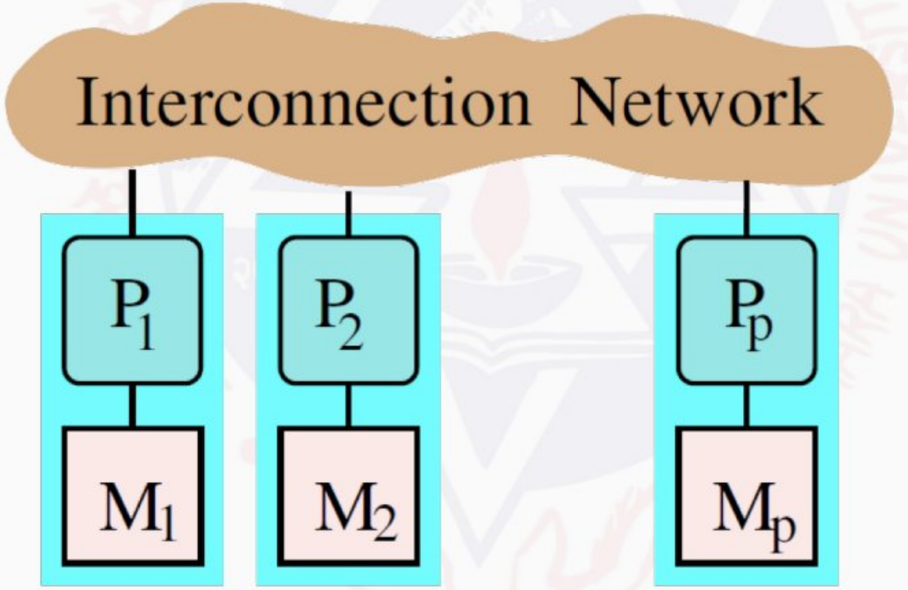
Shared Memory System

- Multiprocessor or computer with a standard multicore CPU
- They uses a shared memory



Distributed Memory System

- Multicores can be aggregated into larger systems, such as clusters, by using a network to interconnect them (**Multicore clusters**).
- **Multicore clusters** have a ***distributed memory***



Thread Parallelism

- **Thread Parallelism**
  - multiple threads execute in parallel
  - explicitly controlled by the programmer
  - Each thread may perform the same or different operations.
- **Multiple Threads**
  - share the resources such as CPU, memory etc.
  - so synchronization mechanisms (mutex, locks) are often required.
- **Each thread maintains**
  - its own program counter
  - can execute code independently of the other threads.

CamScanner

Thread Parallelism

- **Role of *Operating System***
  - loads a thread onto a processing core for execution and
  - switches it out when another thread needs to run
- **Difficult and Error-prone**
  - due to shared-memory, need to take care of race conditions and deadlocks
  - Difficult to divide or provide the work (with same load) among the threads
  - Direct control over threads
- **Useful for *CPU-intensive* tasks**
  - If you need direct control over thread and CPU cores

CamScanner

Print Symbol without Thread Parallelism

```
1   #include<iostream>
2
3
4   /* A single function serially run by 2 function calls: one function
    call prints '+'. After it finishes its task, another prints '-' */
5   void print_symbol(char symbol) {
6       for(int i = 0; i < 100; i++) {
7           std::cout << symbol << " ";
8       }
9   }
10
11  int main() {
12      // call two functions
13      print_symbol('+');
14      print_symbol('-');
15
16      return 0;
17  }
```

Output:

```
[Running] cd "/Users/udayadhungana/Parallel Algorithms/Thread+Programming/
+ + + + + + + + + + + + + + + + + + + + + + + + + + + + + + + + + + + + + +
+ + + + + + + + + + + + + + + + + + + + + + + + + + + + + + + + + + + + + +
+ + + + + + + + + + + + + + + + + + + + + + + + + - - - - - - - - - - - -
- - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - -
- - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - -
- - - - - - - - - - - - - - - - - -
[Done] exited with code=0 in 4.614 seconds
```


CamScanner

Print Symbol using Thread Parallelism

```cpp
#include<iostream>
#include<thread>

/* A single function parallelly run by 2 threads:
   One thread print +, another prints - parallely */
void print_symbol(char symbol) {
    for(int i = 0; i < 100; i++) {
        std::cout << symbol << " ";
    }
}

int main() {
    // create two threads
    std::thread t1(print_symbol, '+');
    std::thread t2(print_symbol, '-');

    // wait for threads to finish
    t1.join();
    t2.join();

    return 0;
}
```

Output 1:

```
[Running] cd "/Users/udayadhungana/Parallel Algorithms/Thread+Programming/
+ + + + + + + + + + + + + + - - - - - - - - + - - - - + + + + + + + + + + +
+ + + + + + + + + + + + + + + + + + + + + + + + + + + + + + + + + + + + + +
+ + + + + + + + + + + + + + + + + + + + + + + + + + + + + + + + + + + + + +
+ + + + + + + + + + + + + + + + + + + + + + + + + + + + + + + + + + + + + +
+ + + + + + + + + + + + + + + + + + + + + + + + + + + + + + + + + + + + + +
+ + + + + + + + + + + + + + + + + + + + + + + + + + + + + + + + + + + + + +
[Done] exited with code=0 in 2.905 seconds
```

Output 2:

```
[Running] cd "/Users/udayadhungana/Parallel Algorithms/Thread+Programming/
+ + + + + + + + + + + + + + + + + + + + + + + + + + + + + + + + + + + + + +
+ + + + + + + + + + + + + + + + + + + + + + + + + + + + + + + + + + + + + +
+ + + + + + + + + + + + + + + + + + + + + + + + + + + + + + + + + + + + + +
+ + + + + + + + + + + + + + + + + + + + + + + + + + + + + + + + + + + + + +
+ + + + + + + + + + + + + + + + + + + + + + + + + + + + + + + + + + + + + +
+ + + + + + + + + + + + + + + + + + + + + + + + + + + + + + + + + + + + + +
[Done] exited with code=0 in 1.341 seconds
```


CamScanner

Printing Plus/Minus without Parallelism

```cpp
#include <iostream>
#include <thread>

void display_plus() {
  for(int i=0; i<=100; i++){
    std::cout<<" +";
  }
}

void display_minus() {
  for(int i=0; i<=100; i++){
    std::cout<<" -";
  }
}

int main() {

  display_plus();
  display_minus();
}
```

```
[Running] cd "/Users/udayadhungana/Parallel Algorithms/Thread
+Programming/odd_even_multiThread/" && g++ -std=c++17 test.cpp -o
test && "/Users/udayadhungana/Parallel Algorithms/"test
+ + + +
```

Printing Plus/Minus with Thread Parallelism

```cpp
#include <iostream>
#include <thread>

void display_plus() {
  for(int i=0; i<=100; i++){
    std::cout<<" +";
  }
}

void display_minus() {
  for(int i=0; i<=100; i++){
    std::cout<<" -";
  }
}

int main() {

  std::thread t1(display_plus);
  std::thread t2(display_minus);

  t1.join();
  t2.join();
}
```

```
[Running] cd "/Users/udayadhungana/Parallel Algorithms/Thread
+Programming/odd_even_multiThread/" && g++ -std=c++17 test.cpp -o
test && "/Users/udayadhungana/Parallel Algorithms/"test
+ + + + + + + + + + + + + + + + + + + + + - + + + + - - - - - - - - - - -
- - - - - - - - - - - + + + + + + + + + + + + + + + + + + + + + + + + + +
+ + + + + + + + - - - - - - - - - - - - - - - - - - - - - - - - - - - - - -
- - - - - - - - - - - - - - - - - - - - - - - - - - - + + + + + + + + + +
+ + + + + + + + + + + + + + + - - - - - - - - - - - - - - - - - - - - - - -
- - - - - + - - - + + + + + + + + + + + + + + + + + + + + + + + + + + + + +
[Done] exited with code=0 in 1.893 seconds
```


CamScanner

Merge Sort without Parallelism

```
39  // Merge Sort function
40  void mergeSort(std::vector<int>& arr, int left, int right) {
41      if (left < right) {
42          int mid = left + (right - left) / 2;
43          //divide the array into two halves
44          mergeSort(arr, left, mid);
45          mergeSort(arr, mid + 1, right);
46
47          //merge two halves
48          merge(arr, left, mid, right);
49      }
50  }
```

CamScanner

Merge Sort using Thread Parallelism

```
40  // Parralel Merge Sort function
41  void P_mergeSort(std::vector<int>& arr, int left, int right) {
42      if (left < right) {
43          int mid = left + (right - left) / 2;
44          //divide the array into two halves
45          std::thread t1(P_mergeSort, std::ref(arr), left, mid);
46          std::thread t2(P_mergeSort, std::ref(arr), mid + 1, right);
47
48          t1.join();
49          t2.join();
50
51          //merge two halves
52          merge(arr, left, mid, right);
53      }
54  }
```

CamScanner

Merge Sort using Thread Parallelism

```
40  // Parralel Merge Sort function
41  void P_mergeSort(std::vector<int>& arr, int left, int right) {
42      if (left < right) {
43          int mid = left + (right - left) / 2;
44          //divide the array into two halves
45          std::thread t1(P_mergeSort, std::ref(arr), left, mid);
46          std::thread t2(P_mergeSort, std::ref(arr), mid + 1, right);
47
48          t1.join();
49          t2.join();
50
51          //merge two halves
52          merge(arr, left, mid, right);
53      }
54  }
```

What is the problem in this implementation?

CamScanner

Merge Sort using Thread Parallelism

What is the problem in this implementation?

->> Depth of thread creation is not defined.

->> Too many threads degrades the system performance

- no sufficient tasks for created threads
- only overhead to thread management

CamScanner

Optimized Parallel Merge Sort using Thread

```cpp
// Optimized Parralel Merge Sort function
void P_mergeSort(std::vector<int>& arr, int left, int right, int depth = 0) {
    if (left < right) {
        int mid = left + (right - left) / 2;

        //limit the thread creation upto maxDepth
        if(depth <= (maxDepth)) {
            //divide the array into two halves
            std::thread t1(P_mergeSort, std::ref(arr), left, mid, depth + 1);
            std::thread t2(P_mergeSort, std::ref(arr), mid + 1, right, depth + 1);

            //Wait until the two threads completes their tasks
            t1.join();
            t2.join();
        }
        else {
            //After maxDepth is reached, don't create the threads
            //just use it as normal merge sort
            P_mergeSort(arr, left, mid);
            P_mergeSort(arr, left, mid);
        }
        //merge two halves
        merge(arr, left, mid, right);
    }
}
```

```cpp
    //Determining max numbers of logical processors. So
    //we only creates threads that just fit to number of
    //logical processing cores availbale at the machine
    int maxThreads = std::thread::hardware_concurrency();
    //Finding max depth until we create the threads. After
    //this depth we don't create threads further.
    maxDepth = std::log2(maxThreads);

    P_mergeSort(arr, 0, n - 1);

```


CamScanner

Why to limit the parallel depth in Merge Sort

- **To avoid too many threads**
  - Each recursive call of P_mergeSort() ***spawns*** two new threads.
  - Without a depth limit, the number of threads grows exponentially (i.e., 2^depth threads).
  - If depth is too large, it can create thousands of threads, overwhelming the CPU and causing performance degradation due to:
    - Context switching overhead (CPU constantly switching between threads).
    - Memory exhaustion (each thread requires stack memory).
- **Threads beyond CPU cores are inefficient**
  - CPUs have a limited number of cores (e.g., 4-core, 8-core, etc.).
  - If we create more threads than available cores, they compete for CPU time.
  - This causes slower execution due to excessive scheduling.
- **Recursive overhead**
  - Each recursive call requires additional memory (stack space).
  - Too many threads and deep recursion together can lead to stack overflow or memory exhaustion.

CamScanner

Why to limit the parallel depth in Merge Sort

- **Why depth<=maxDepth?**
  - maxDepth = maximum number of CPU cores in your computer. Suppose, you computer has 4 CPU cores, then maxDepth = $log_2(4)$= 2
  - depth = 2 means the recursion tree goes 2 levels deep in parallel before switching to sequential execution.
  - This results in at most 2^2 = 4 threads at the deepest level.
- **What Happens When depth > maxDepth?**
  - Instead of creating more threads, the function switches to sequential execution.
- **A good heuristic is**
  - number of threads = $log_2(\text{no_of_cores})$.

CamScanner

Quick Sort using Thread Parallelism

```cpp
#include <iostream>
#include <vector>
#include <thread>

int maxDepth = 0;

// Partition function for Quick Sort
int partition(std::vector<int>& arr, int low, int high) {
    int pivot = arr[high];
    int i = low - 1;

    for (int j = low; j < high; ++j) {
        if (arr[j] <= pivot) {
            ++i;
            std::swap(arr[i], arr[j]);
        }
    }
    std::swap(arr[i + 1], arr[high]);
    return i + 1;
}
```

CamScanner

Quick Sort using Thread Parallelism

```
20  // Serial Quick Sort
21  void quickSort(std::vector<int>& arr, int low, int high) {
22      if (low < high) {
23          int pivotIndex = partition(arr, low, high);
24          quickSort(arr, low, pivotIndex - 1);
25          quickSort(arr, pivotIndex + 1, high);
26      }
27  }
```

CamScanner

Quick Sort using Thread Parallelism

```
31  // Parallel Quick Sort using threads
32  void P_QuickSort(std::vector<int>& arr, int low, int high) {
33      if (low < high) {
34          int p = partition(arr, low, high);
35
36          /* Create threads for parallel execution  */
37          std::thread t1(P_QuickSort, std::ref(arr), low, p - 1);
38          std::thread t2(P_QuickSort, std::ref(arr), p + 1, high);
39
40          // Wait for both threads to finish
41          t1.join();
42          t2.join();
43      }
44  }
```

Parallel but NOT optimized

CamScanner

Quick Sort using Thread Parallelism

```
31  // Parallel Quick Sort using threads
32  void P_QuickSort(std::vector<int>& arr, int low, int high, int depth = 0) {
33      if (low < high) {
34          int p = partition(arr, low, high);
35
36          if (depth <= maxDepth) {
37              /* Create threads for parallel execution until the no of threads
38                 equal to number of logical cores in the machine  */
39              std::thread t1(P_QuickSort, std::ref(arr), low, p - 1, depth + 1);
40              std::thread t2(P_QuickSort, std::ref(arr), p + 1, high, depth + 1);
41
42              // Wait for both threads to finish
43              t1.join();
44              t2.join();
45          } else {
46              /* After maxDepth is reached, don't create the threads,
47                 fallback to sequential Quick Sort */
48              quickSort(arr, low, p - 1);
49              quickSort(arr, p + 1, high);
50          }
51      }
52  }
```

CamScanner

Task-Parallelism

- Why **Task Parallelism**?
  - Programmer need to aware of
  - To overcome difficulties of thread programming
- **Task Parallelism**
  - provide a layer of software on top of threads to coordinate, schedule, and manage the processors of a multicore
  - allows parallelism to be specified in a ***processor oblivious*** fashion,
  - The programmer only needs to identify what computational tasks may run in parallel but does not need to indicate which thread or processor performs the task
  - the programmer is freed from worrying about communication protocols, load balancing etc.
- **Scheduler**
  - **Task parallelism model** contains a scheduler,
  - scheduler automatically load-balances the tasks across the processors
- **Task parallelism model**:
  - ***fork-join parallelism***

CamScanner

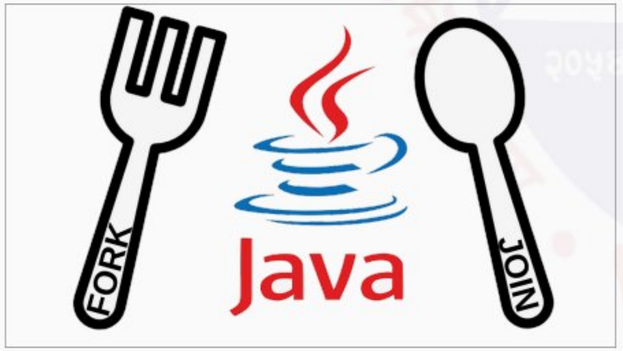
Task-Parallelism

- In Task Parallelism, different tasks (units of work) run in parallel.
- Tasks do not need to be mapped to specific threads manually.
  - Programmer is free from such difficulties as in thread programming
- A task scheduler dynamically assigns tasks to available threads.
  - Programmer is free from such difficulties as in thread programming
- **Thread Parallelism** is ***best***
  - when you need fine-grained control over threads.
- **Task Parallelism** is ***best***
  - when working with independent units of work and want better scalability.

CamScanner

Task-Parallelism

- It is often implemented using ***Cilk***, ***OpenMP***, ***TBB***, or ***thread pools***.



Intel® Threading Building Blocks

Task-Parallelism using OpenMP

```
1   #include <iostream>
2   #include <omp.h>
3
4   /* function prints '+' */
5   void print_plus() {
6       for(int i = 0; i < 100; i++) {
7           std::cout << "+ ";
8       }
9   }
10
11  /* function prints '-' */
12  void print_minus() {
13      for(int i = 0; i < 100; i++) {
14          std::cout << " - ";
15      }
16  }
17
```

```
18  int main() {
19      #pragma omp parallel sections
20      {
21          #pragma omp section
22          {
23              print_plus();
24          }
25          #pragma omp section
26          {
27              print_minus();
28          }
29      }
30      return 0;
31  }
```

CamScanner

Task-Parallelism using OpenMP

```
/*  For My MAC:
    Compile using Clang:
        clang++ -Xpreprocessor -fopenmp -I/usr/local/opt/libomp/
        include -L/usr/local/opt/libomp/lib -lomp -o print_symbol_omp
        print_symbol_omp.cpp

    Run: ./print_symbol_omp
*/
```

```
PROBLEMS 1   OUTPUT   TERMINAL   ···                zsh + ···
udayadhungana@Udayas-MacBook-Pro Thread+Programming % clang++ -Xpreprocessor -
fopenmp -I/usr/local/opt/libomp/include -L/usr/local/opt/libomp/lib -lomp -o p
rint_symbol_omp print_symbol_omp.cpp
udayadhungana@Udayas-MacBook-Pro Thread+Programming % ./print_symbol_omp
+ + + + + + + + + + + + + + + + + + + + + + + + + + + + + + + + + + + + + + + +
  - - - - - - - - - + + + + + + + + + + + + + + + + + + + + + + + + + + + + + -
  - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - -
  - - - - - - - - - + + + + + + + + + + + + + + + + + + + + + + + + + + + + + +
 + + + + + + + + + + + + + + + + + + + - - - - - - - - - - - - - - - - - - - -
  - - + + + + + + + + - - - - - - - - - - - - - - - - - - - - - - - - - - - -
- - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - -
```


CamScanner

Task-Parallelism using OpenMP

If we keep track the work done by different threads, the output will look like:

```
udayadhungana@Udayas-MacBook-Pro Thread+Programming % clang++ -Xpreprocessor -fopenmp -I/usr/local/opt/libo
mp/include -L/usr/local/opt/libomp/lib -lomp -o print_symbol_omp print_symbol_omp.cpp
udayadhungana@Udayas-MacBook-Pro Thread+Programming % ./print_symbol_omp

Thread number: 0
+ + + +
```

Task-Parallelism using OpenMP

If we keep track the work done by different threads, the output will look like:

```
udayadhungana@Udayas-MacBook-Pro Thread+Programming % clang++ -Xpreprocessor -fopenmp -I/usr/local/opt/libo
mp/include -L/usr/local/opt/libomp/lib -lomp -o print_symbol_omp print_symbol_omp.cpp
udayadhungana@Udayas-MacBook-Pro Thread+Programming % ./print_symbol_omp

Task done by Thread- 0
+ + + +
```

Task-Parallelism

- No need to manage threads manually
- Efficient load balancing
- Less control over individual threads

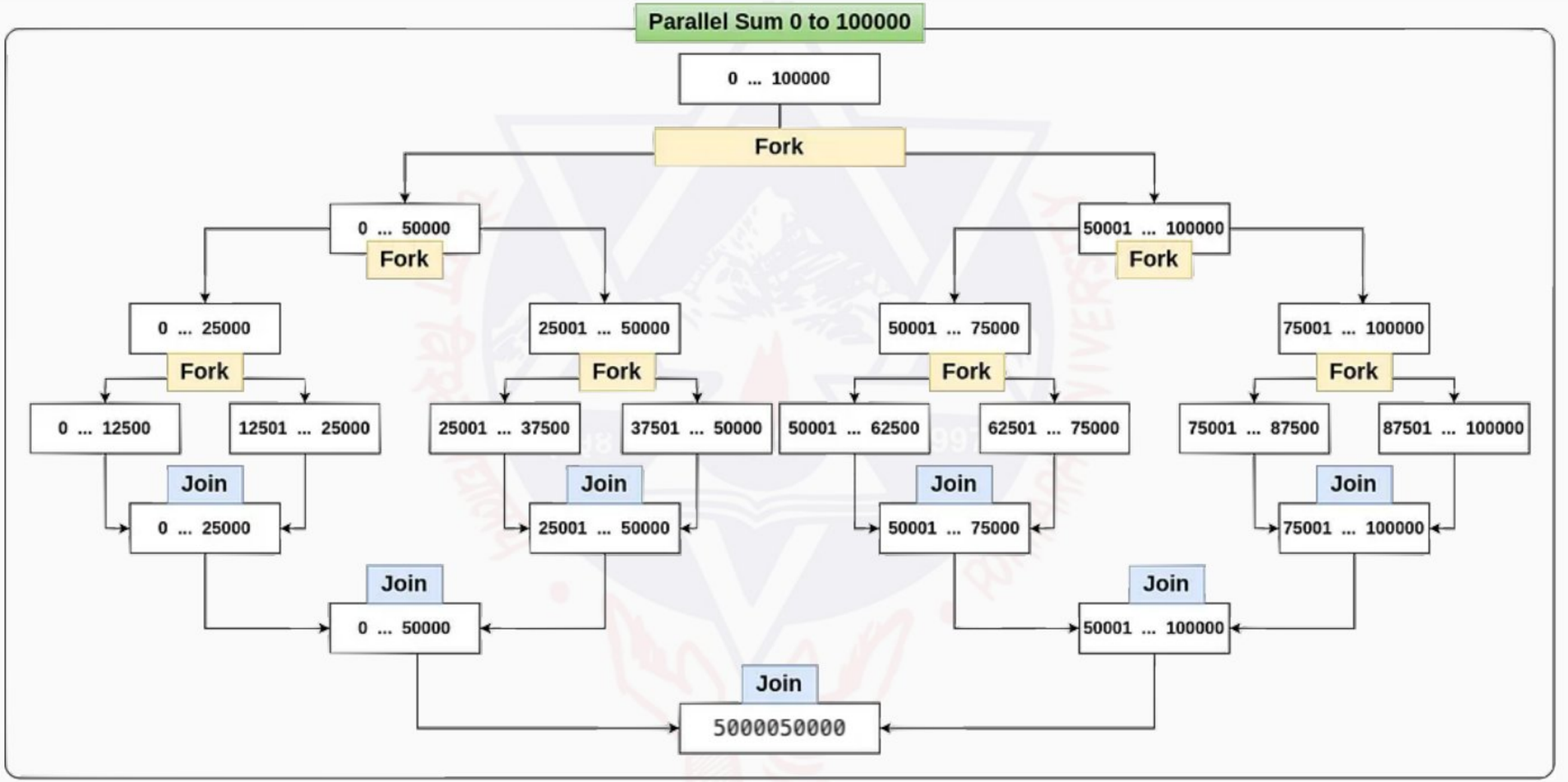
CamScanner

Fork-join Parallelism

- **Fork-Join Parallelism**
  - Is a **divide-and-conquer parallel computing** model
  - a task is forked ("split") into multiple subtasks
  - these multiple subtasks are executed in parallel
  - their results are joined ("combined") after execution.
- **Fork-Join parallel algorithms**
  - **Spawning**
    - Spawning allows a subroutine to be forked and executed independently. At the same time the caller (parent) can continue to execute other task in parallel while the spawned subroutine computes its result.
  - **Parallel loops**
    - A parallel loop is like an ordinary **for** loop, except that multiple iterations of the loop can execute at the same time.

CamScanner

Fork-join Parallelism



Fork-join Parallelism- Fibonacci Numbers

- The Fibonacci numbers

$$F_i = \begin{cases} 0 & \text{if } i = 0 \\ 1 & \text{if } i = 1 \\ F_{i-1} + F_{i-2} & \text{if } i \geq 2 \end{cases}$$

- To calculate the n^{th} Fibonacci number recursively, you could use the ordinary serial algorithm in the procedure FIB as:

```
FIB(n)
1  if n ≤ 1
2      return n
3  else x = FIB(n − 1)
4       y = FIB(n − 2)
5       return x + y
```


CamScanner

Fork-join Parallelism- Fibonacci Numbers

- The serial algorithm FIB does needless repeated work,
  - So parallelizing it can improve its performance/efficiency
- The P-FIB is a parallel algorithm that computes Fibonacci numbers using ***parallel keywords***
  - **spawn**
  - **sync**

.

```
P-FIB(n)
1  if n ≤ 1
2        return n
3  else x = spawn P-FIB(n − 1)   // don't wait for subroutine to return
4       y = P-FIB(n − 2)         // in parallel with spawned subroutine
5       sync                     // wait for spawned subroutine to finish
6       return x + y
```


CamScanner

Fork-join Parallelism- Fibonacci Numbers

- ***Spawning*** occurs when the keyword **spawn** precedes a procedure call, as in line 3 of P-FIB.
- When P-FIB executes the spawn- the ***parent -*** may continue to execute in parallel with the spawned subroutine- its ***child-*** instead of waiting for the child to finish.
- In this case, while the spawned child is computing P-FIB$(n-1)$, the parent may go on to compute P-FIB$(n-2)$ in line 4 in parallel with the spawned child.
- Since the P-FIB procedure is recursive, these two subroutine calls themselves create nested parallelism, as do their children, thereby creating a potentially vast tree of subcomputations, all executing in parallel
- The keyword **spawn** does not say, however, that a procedure *must* execute in parallel with its spawned children, only that it *may*.
- The parallel keywords express the ***logical parallelism*** of the computation, indicating which parts of the computation may proceed in parallel.
- At runtime, it is up to a ***scheduler*** to determine which subcomputations actually run in parallel by assigning them to available processors as the computation unfolds.

CamScanner

Fork-join Parallelism- Fibonacci Numbers

- A procedure cannot safely use the values returned by its spawned children until after it executes a **sync** statement, as in line 5.
- The keyword **sync** indicates that the procedure must wait as necessary for all its spawned children to finish before proceeding to the statement after the **sync** - the "join" of a fork-join parallel computation.
- The P-FIB procedure requires a **sync** before the **return** statement in line 6 to avoid the anomaly that would occur if and were summed before P-FIB $(n-1)$ had finished and its return value had been assigned to x
- Every procedure executes a sync implicitly before it returns, thus ensuring that all children finish before their parent finishes.

CamScanner

Parallel fib() - OpenCilk

- The named child function cilk_spawn fib(n-1) may execute in parallel with its parent executes fib(n-2).
- Cilk is a small set of linguistic extensions to C++ (resp. C) supporting fork-join parallelism

```
int fib (int n) {
  if (n<2) return (n);
  else {
    int x,y;
    x = cilk_spawn fib(n-1);
    y = fib(n-2);
    cilk_sync;
    return (x+y);
  }
}
```

Nested Parallelism in Cilk

CamScanner

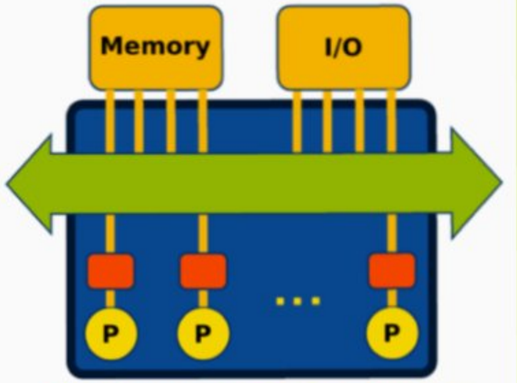
Parallel fib() - OpenCilk

The cilk_scope creates a parallel region

Processor-oblivious Cilk code

```
int p_fib(int n) {
  if (n < 2)
    return n;
  int x, y;
  cilk_scope {
    x = cilk_spawn p_fib(n-1);
    y = p_fib(n-2);
  }
  return x + y;
}
```

Fast multiprocessor computation



https://www.opencilk.org/doc/tutorials/introduction-to-cilk-programming/

https://cesmix.mit.edu/software/opencilk-task-parallelism/

CamScanner

Parallel fib() - OpenCilk

The cilk_scope creates a parallel region spanning lines 6–7. In line 6 the cilk_spawn keyword indicates that p_fib(n-1) may be executed at the same time as the rest of the parallel region (that is, the call to p_fib(n-2) in line 7). The end of the cilk_scope in line 8 guarantees that the spawned function has finished before the return statement in line 9 is executed.

```
1  int p_fib(int n) {
2    if (n < 2)
3      return n;                        // base case
4    int x, y;
5    cilk_scope {                       // begin lexical scope of parallel region
6      x = cilk_spawn p_fib(n-1);       // don't wait for function to return
7      y = p_fib(n-2);                  // may run in parallel with spawned function
8    }                                  // wait for spawned function if needed
9    return x + y;
10 }
```

https://www.opencilk.org/doc/tutorials/introduction-to-cilk-programming/

CamScanner

Parallel Merge Sort using OpenCilk

```
41  // Parralel Merge Sort function using cilk
42  void P_mergeSort(std::vector<int>& arr, int left, int right) {
43      if (left < right) {
44          int mid = left + (right - left) / 2;
45
46          /* Spawn a new parallel task */
47          cilk_spawn mergeSort(arr, left, mid);
48          mergeSort(arr, mid + 1, right);
49
50          //wait for all spawned tasks to complete
51          cilk_sync;
52
53          //merge two halves
54          merge(arr, left, mid, right);
55      }
56  }
```

CamScanner

Parallel Quick Sort using OpenCilk

```
19  // Parallel Quick Sort using Cilk (Not optimized)
20  void parallelQuickSort(vector<int>& arr, int low, int high) {
21      if (low < high) {
22          int pivot = partition(arr, low, high);
23
24          cilk_spawn parallelQuickSort(arr, low, pivot - 1);
25          parallelQuickSort(arr, pivot + 1, high);
26          cilk_sync;
27      }
28  }
```

CamScanner

Fork-join in OpenMP

```
#pragma omp parallel sections
{
    #pragma omp section
    {
        ...
    }
    #pragma omp section
    {
        ...
    }
}
```

Fork- one thread executes code here

Fork- one thread executes code here

Join- wait for all threads to complete their tasks here

CamScanner

Parallel Merge Sort using OpenMP

- To parallelize the merge sort algorithm using OpenMP, you can use OpenMP pragmas to parallelize the recursive calls.
    - Use `#pragma omp parallel sections` to parallelize the recursive calls to mergeSort.
    - Use `#pragma omp section` to define the sections that should run in parallel.

CamScanner

Parallel Merge Sort using OpenMP

```cpp
// Parralel Merge Sort using OpenMP
void P_mergeSort(std::vector<int>& arr, int left, int right) {
    if (left < right) {
        int mid = left + (right - left) / 2;

        /* fork: dived the array into 2 subtasks and
                run each sbtasks in parallel */
        #pragma omp parallel sections
        {
            #pragma omp section
            {
                P_mergeSort(arr, left, mid);
            }
            #pragma omp section
            {
                P_mergeSort(arr, mid + 1, right);
            }
            //syn: wait for all spawned tasks to complete
        }

        //merge two halves
        merge(arr, left, mid, right);
    }
}
```

```
PROBLEMS  OUTPUT  TERMINAL  ···                                  zsh + ···
udayadhungana@Udayas-MacBook-Pro Thread+Programming % clang++ -std=c++11 -Xpre
processor -fopenmp -I/usr/local/opt/libomp/include -L/usr/local/opt/libomp/lib
 -lomp -o merge_sort_MT_omp merge_sort_MT_omp.cpp
udayadhungana@Udayas-MacBook-Pro Thread+Programming % ./merge_sort_MT_omp
Sorted array: 1 2 3 4 5 6 7 8 9 10
udayadhungana@Udayas-MacBook-Pro Thread+Programming %
```


CamScanner

Parallel Quick Sort using OpenMP

Assignment:

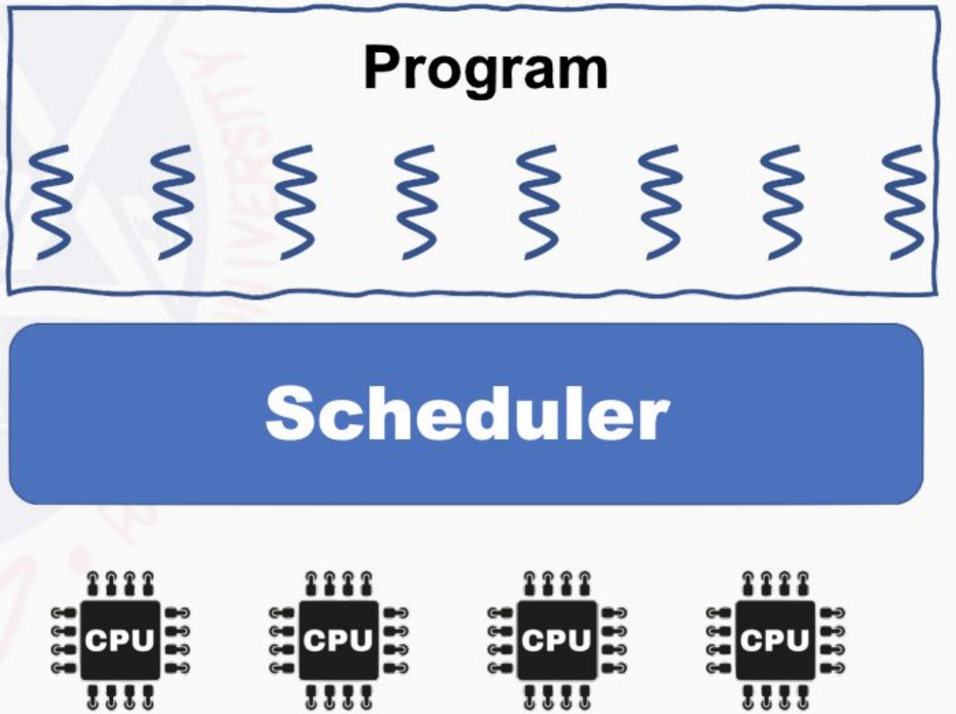
Use OpenMP for implementation of Quick Sort.

CamScanner

Scheduler

- The scheduler maps each task to a processor or a core whenever it is available

- Good performance depends on more than just minimizing the work and span.
- The strands must also be scheduled efficiently onto the processors.
- Our fork-join parallel-programming model provides no way for a programmer to specify which strands to execute on which processors.
- Instead, we rely on the runtime system's scheduler to map the dynamically unfolding computation to individual processors.
- the scheduler maps the strands to static threads, and the operating system schedules the threads on the processors themselves.



CamScanner

Parallel Loops

- Many algorithms contain loops for which all the iterations can operate in parallel.
- Although the **spawn** and **sync** keywords can be used to parallelize such loops, it is more convenient to specify directly that the iterations of such loops can run in parallel.

CamScanner

Parallel for loop in OpenMP

```cpp
#include <iostream>
#include <omp.h>

//parallel loop
void parallel_loop() {
    #pragma omp parallel for
    for(int i = 0; i < 10; i++) {
        std::cout <<i<<" ";
    }
}

int main() {

    parallel_loop();

    std::cout <<std::endl;
    return 0;
}
```

```
PROBLEMS 1  TERMINAL  ···  zsh + ⌵  ···  ^  ×
udayadhungana@Udayas-MacBook-Pro Thread+Programming % clang++ -
std=c++11 -Xpreprocessor -fopenmp -I/usr/local/opt/libomp/inclu
de -L/usr/local/opt/libomp/lib -lomp -o test test.cpp
udayadhungana@Udayas-MacBook-Pro Thread+Programming % ./test
0 18  936 4 5  7 2
udayadhungana@Udayas-MacBook-Pro Thread+Programming %
```


CamScanner

Parallel for loop in OpenMP

Assignment:

Use parallel loop for Matrix Multiplication.

CamScanner

Parallel Distributed Quick Sort

Assignment:

Case study on parallel distributed quick sort using MPI and OpenMP.

Parallel **Distributed Quick Sort: it** is useful for sorting large datasets across multiple nodes in a **distributed memory system**. It leverages **MPI (Message Passing Interface)** along with **OpenMP** or **Cilk Plus** for intra-node parallelism.

CamScanner

EXTERNAL SORTING AND ITS APPLICATIONS

CamScanner

Why Sorting?

- **Efficient data retrieval**
  - Binary research - $O(log\ n)$ - which requires the data in sorted form
  - Without sorting, search requires $O(n)$
- **Data Organization**
  - Ascending or descending - making easier for data access
- **Facilitate other algorithms**
  - Binary search tree, binary search, merge sort, Kruskal's algorithm etc

CamScanner

Why Sorting?

- **Database operations**
  - Data requested from DB in sorted order is **extremely common**
  - e.g., find students in increasing GPA order
- Useful **eliminating *duplicate copies*** in a collection of records
- Sorting is first step in ***bulk loading*** B+ tree index.
- The **sort-merge join** algorithm (later in class) involves **sorting** as a first step

CamScanner

Internal Sorting

- Sorts the list of data that entirely fit into the main memory.
- No I/O operations (read/write on HDD or SSD) are required.
- Fast access (access speeds of RAM)
- Internal Sorting Algorithms
  - Insertion sort - $O(n^2)$
  - Quick sort - $O(n\ log\ n)$
  - Merge sort - $O(n\ log\ n)$
  - Radix sort - $O(n)$

CamScanner

Limitation of Internal Sorting

- Internal Sorting like quick sort merge sort are very efficient but they require the data must be entirely stored in RAM.
- If the data to be sorted is too large to fit in the RAM, they will perform very poor or even fail.
- They fail or poorly perform the data stored in the external storage due to the following reasons
    - A large number of I/O operations are required
    - Accessing data in external drive is massively slow

CamScanner

Limitation of Internal Sorting

- Many real world **data grows massively** as the time goes on
  - They **may not fit** in the **RAM**
- RAMs **are expensive**
  - they are available in limited size
- I**nfeasible** when **data is being streamed** in real-time
  - For example- logs from servers, sensor data, etc.
  - Because all the data must be available **at once** for sorting.
  - **Online sorting** or **external sorting** is used in such cases
    - data can be processed in chunks and sorted incrementally

CamScanner

Limitation of Internal Sorting

- You have a computer
  - **RAM: 16 GB**
  - SDD: 5TB
- You are given a task
  - TASK: ***to sort a list of Students Data*** *(with min # of I/Os)*
  - Size: 1TB

HOW DO YOU SOLVE (SORT THE DATA) THE PROBLEM

CamScanner

Limitation of Internal Sorting

- You have a computer
  - RAM: 16 GB
  - SDD: 5TB
- You are given a task
  - TASK: To Sort a list of Students Data
  - Size: 1TB

External Sorting

HOW DO YOU SOLVE (SORT THE DATA) THE PROBLEM

CamScanner

Sorting: The Big Picture

Simple algorithms: $O(n^2)$	Fancier algorithms: $O(n \log n)$	Comparison lower bound: $\Omega(n \log n)$	Specialized algorithms: $O(n)$	Handling huge data sets
Insertion sort	Heap sort		Bucket sort	External sorting
Selection sort	Merge sort		Radix sort	
Shell sort	Quick sort			
…	…			

CamScanner

External Sorting

- ***External sorting*** is the sorting technique that is applied on the large amount of data stored in the external storage (HDD, SSD etc.)
- **Goal is** to efficiently sort the large amount of data that exceeds the capacity of the system's memory
- **Analysis** of External Sorting Algorithms
  - Access to disk is much slower than RAM
    - So cost of CPU time is not that much important
    - It is because I/O operations takes much more time than that of the CPU
  - Analysis of external sorting therefore focuses on **the number of disk accesses** (I/O operations) not the CPU cost.

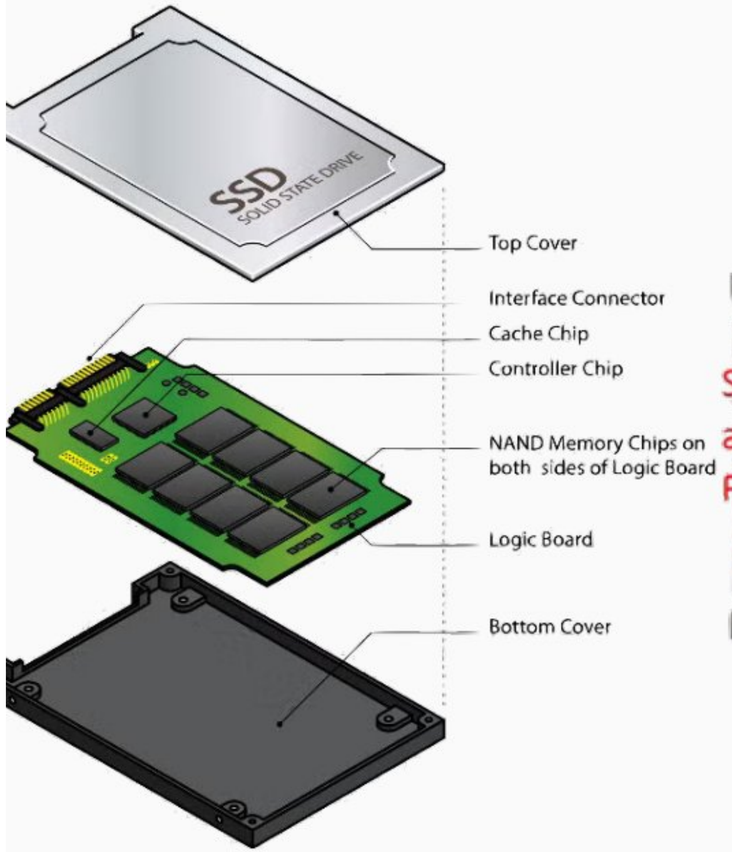
CamScanner

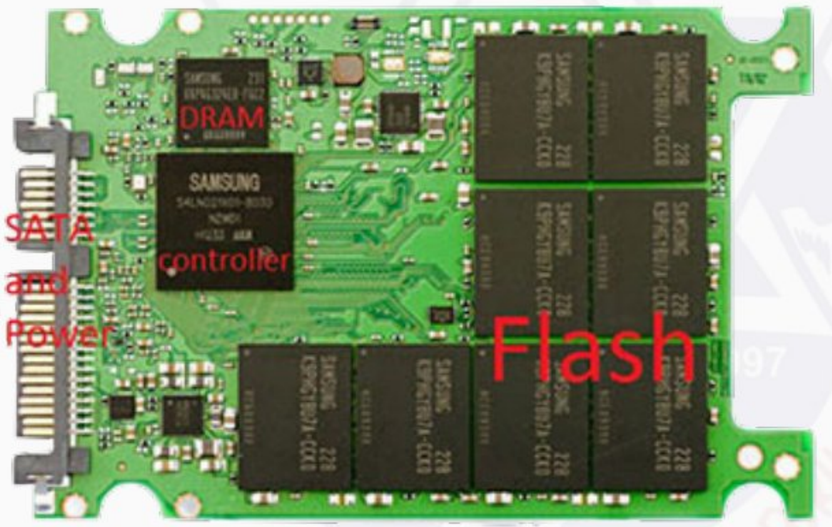
External Sorting Process

1. The large data is divided into **manageable chunks** that fit into memory.
2. **Internal sorting** algorithms such as quick sort or merge sort, or heap sort are used to sort each chunk of data **in memory**.
3. These sorted chunks are then written back to **external storage**
4. **merge** the sorted chunks into a final sorted file.

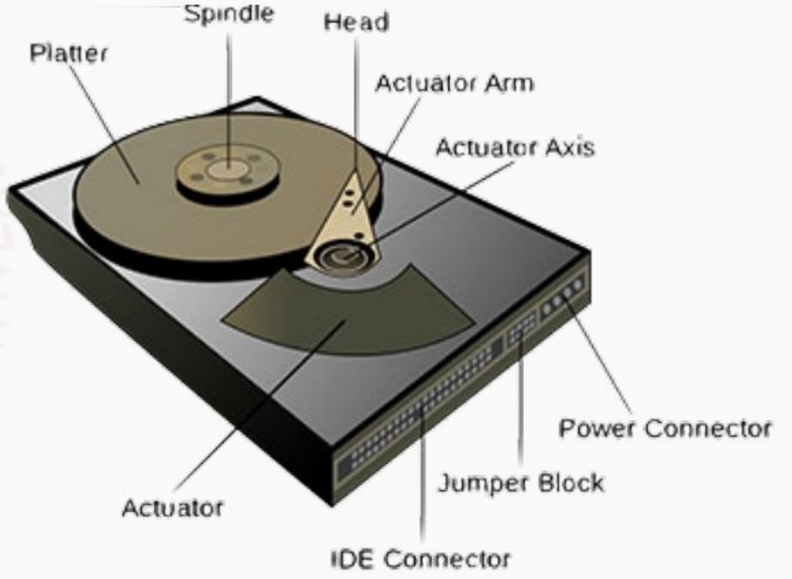
CamScanner

HDD (Hard Disk Drive) vs SSD (Solid State Drive)





SSD



HDD

HDD (Hard Disk Drive) vs SSD (Solid State Drive)

SN	Feature	HDD (Hard Disk Drive)	SSD (Solid State Drive)
1	**Storage Unit**	Sectors & Tracks	Pages & Blocks
2	**Read/Write Unit**	512B – 4KB Sector	4KB – 16KB Page
3	**Erase Unit**	Individual sectors	Entire blocks (128+ pages)
4	**Speed**	Slower (due to moving parts)	Much faster (electronic access)
5	**Wear & Tear**	Mechanical wear (moving parts)	Limited write cycles (flash wear)

CamScanner

External Merge Problem

Input: 2 sorted list (with M and N pages)

Output: ***1 merged sorted list*** (with $M + N$ pages)

Requirement: at least 3 pages.

Can we efficiently (in terms of I/O) merge the two lists using a buffer of size at least *3 pages*?

Yes, Cost: $2(M + N)$ I/Os

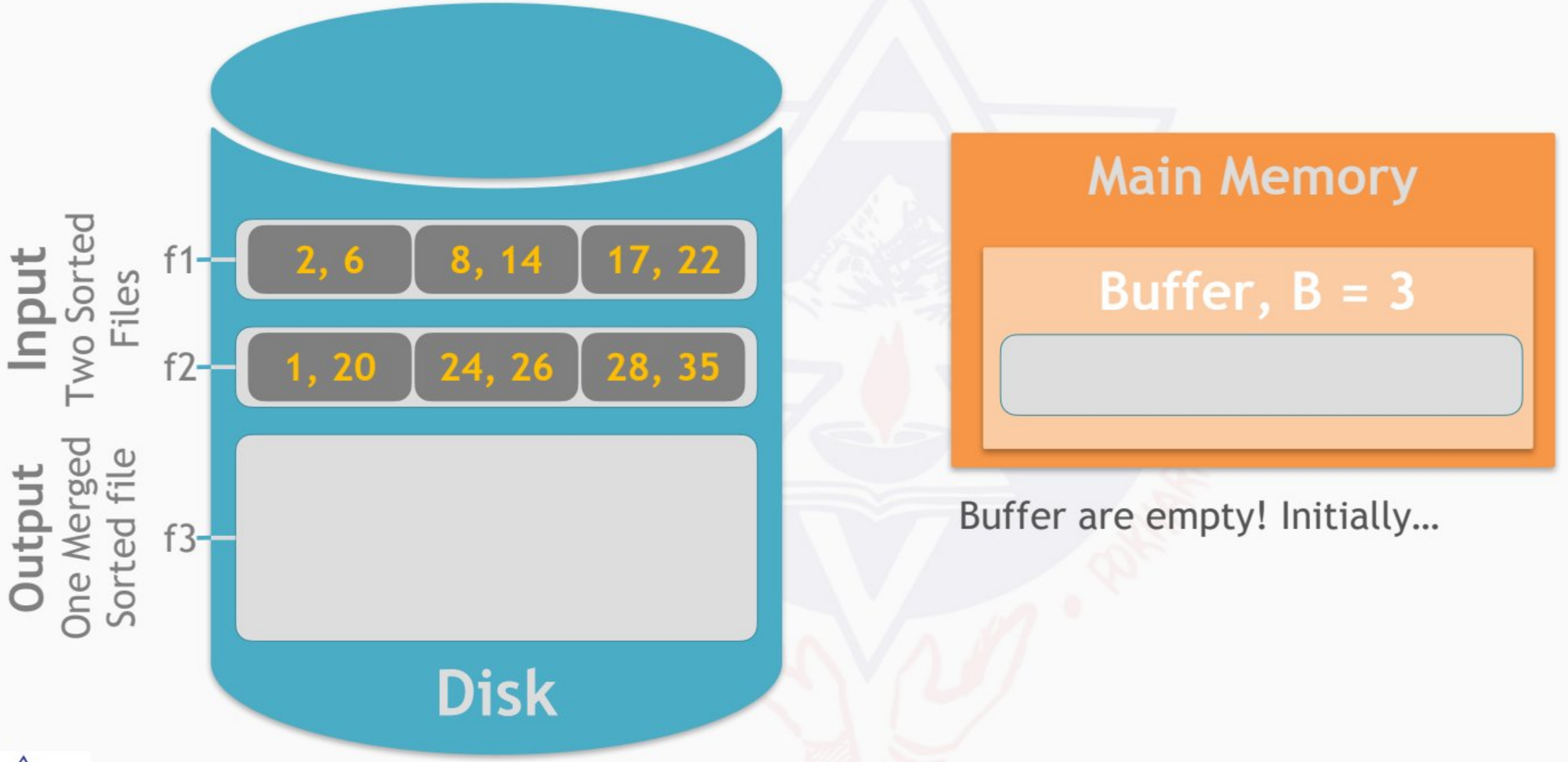
CamScanner

External Merge Algorithm (2 Way Sort)

- Not too practical, but useful to learn basic concepts for external sorting
    - generally, m-way sort is used
- 2-way sort requires only 3 pages (3 Buffers, each of size 1 page) of main memory
    - two buffers to hold 1 page from first sorted list and 1 page from another sorted list
    - one buffer to hold the elements after sorting from the two buffers one by one
- Several sorted sub-files are generated at intermediate steps
- Each sorted sub-file is called a run
    - each run can contain multiple pages (in our example 1 run contains 3 pages)

CamScanner

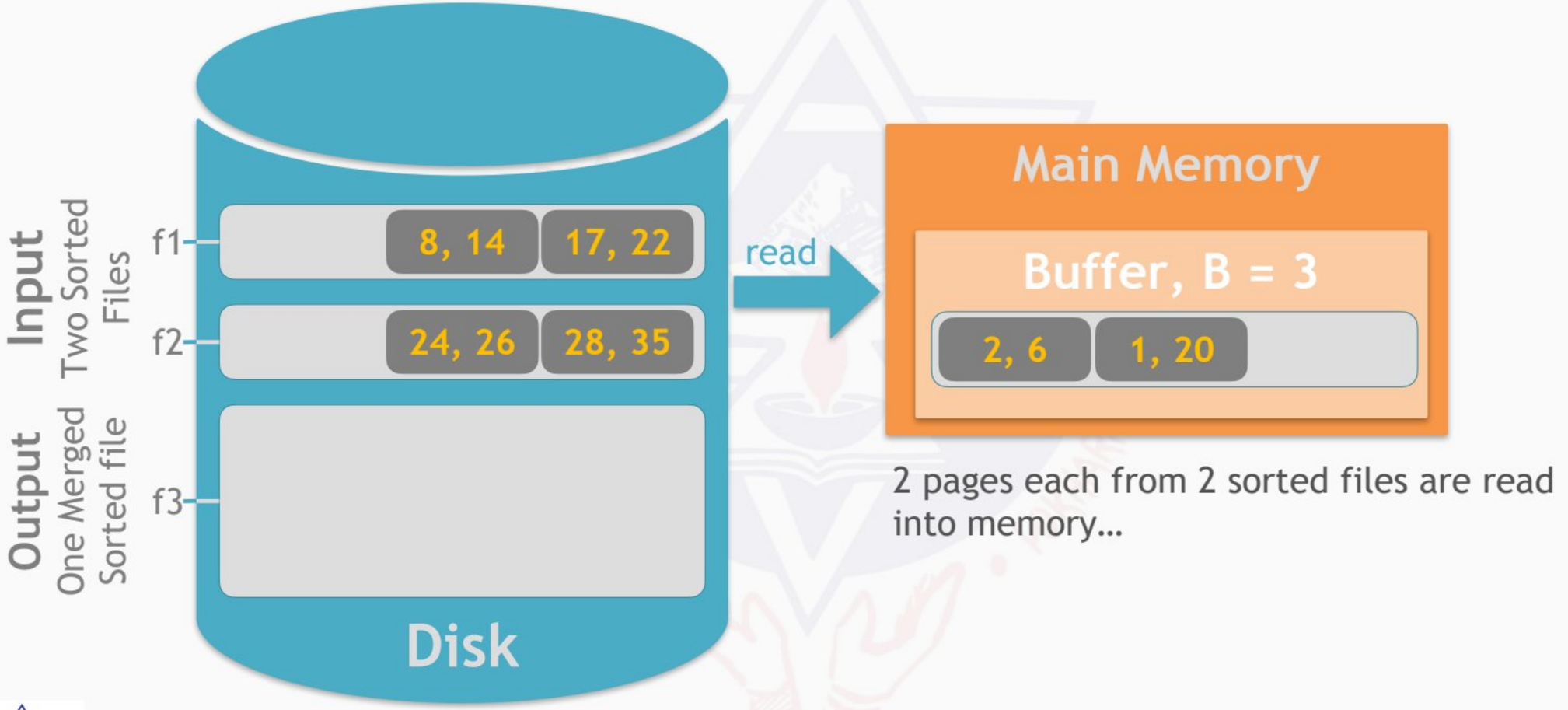
External Merge Algorithm (2 Way Sort)



Buffer are empty! Initially...

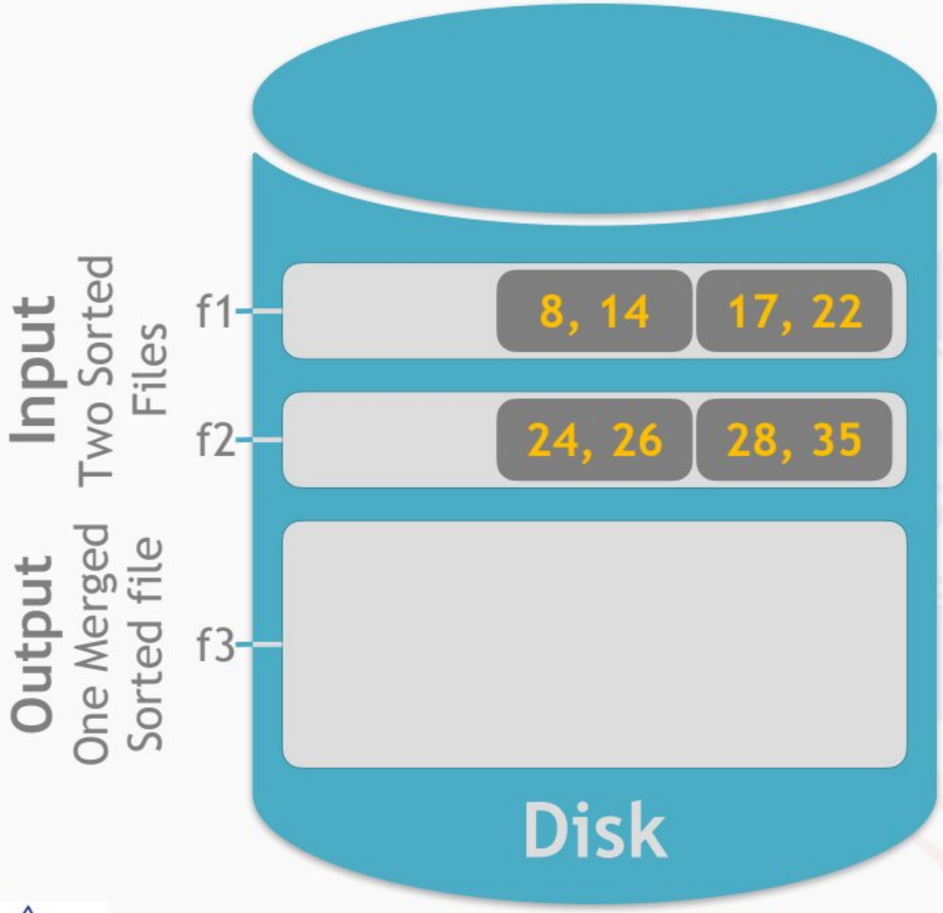
CamScanner

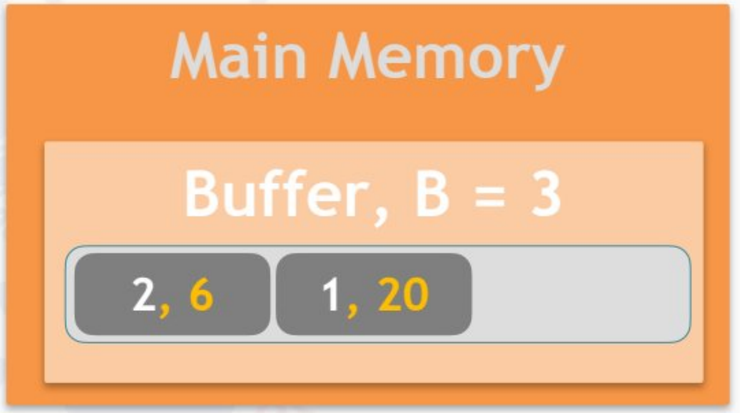
External Merge Algorithm (2 Way Sort)



2 pages each from 2 sorted files are read into memory...

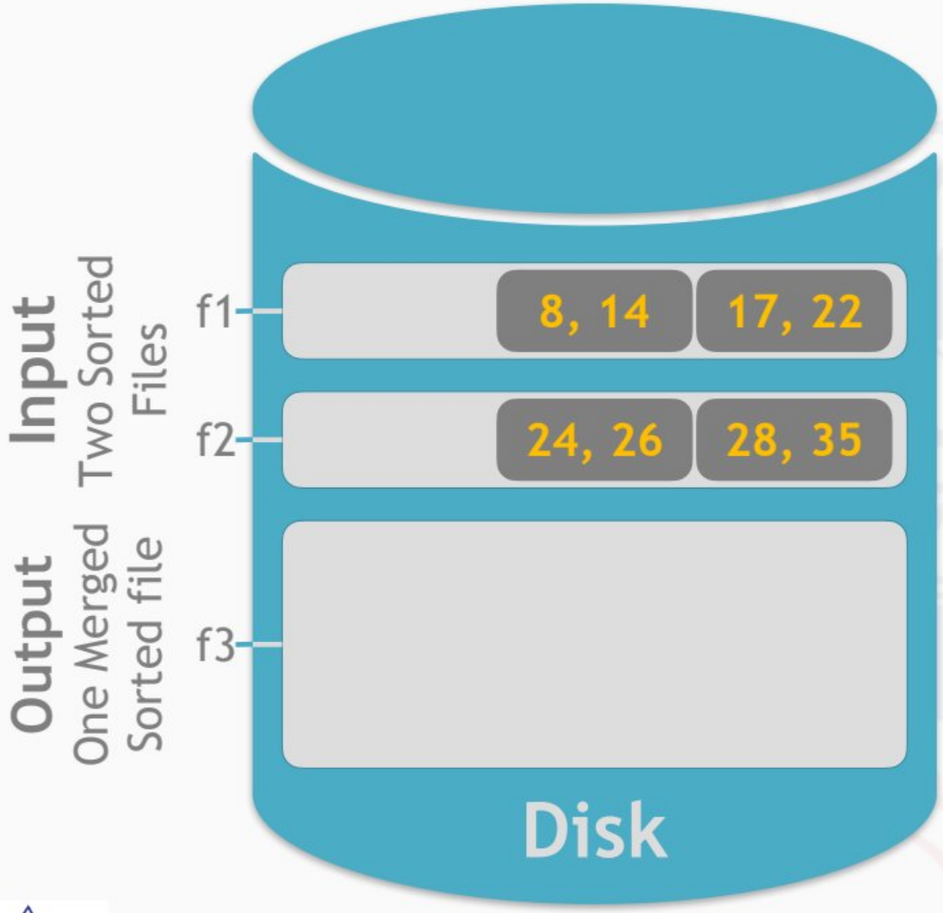
External Merge Algorithm (2 Way Sort)

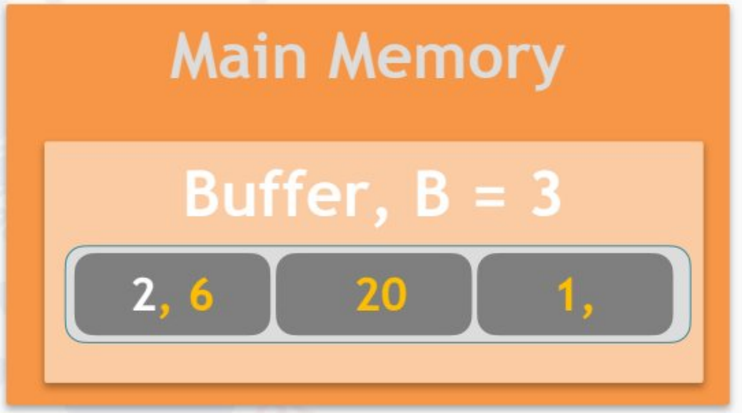




2 pages each from 2 sorted files are read into memory...

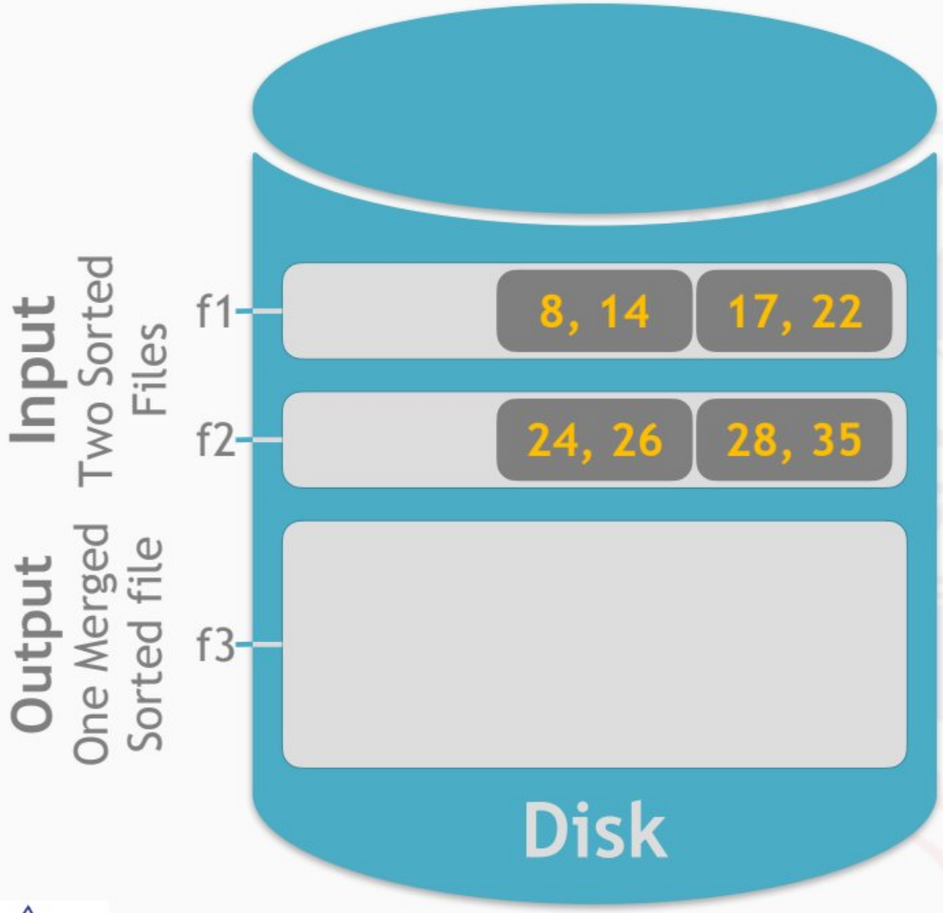
External Merge Algorithm (2 Way Sort)

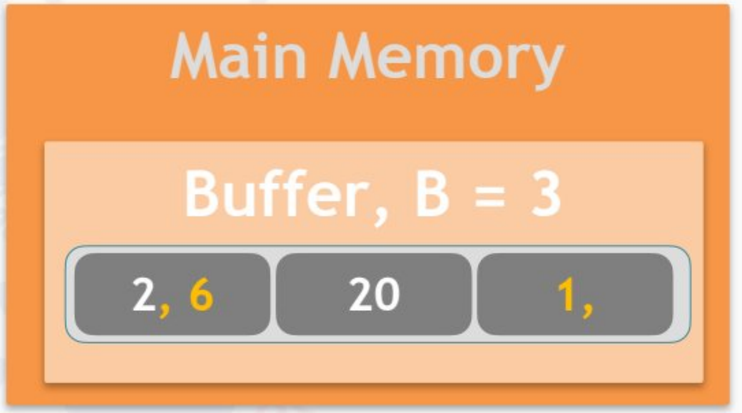




2-way merge takes place.

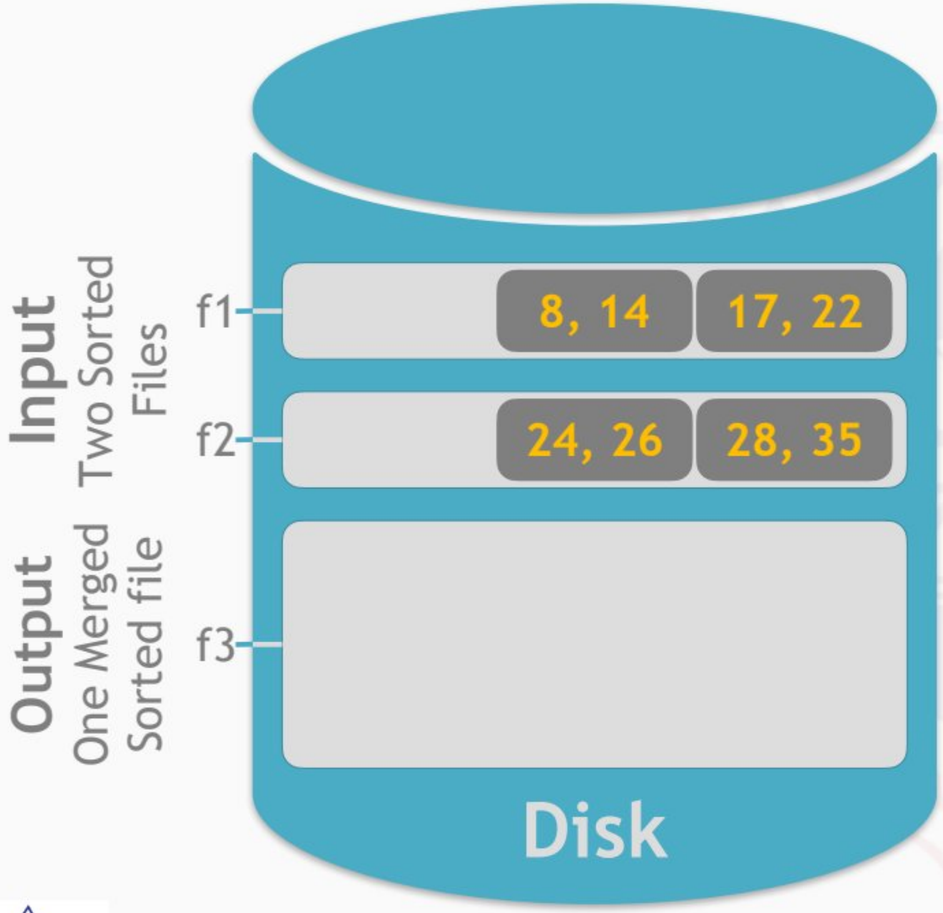
External Merge Algorithm (2 Way Sort)

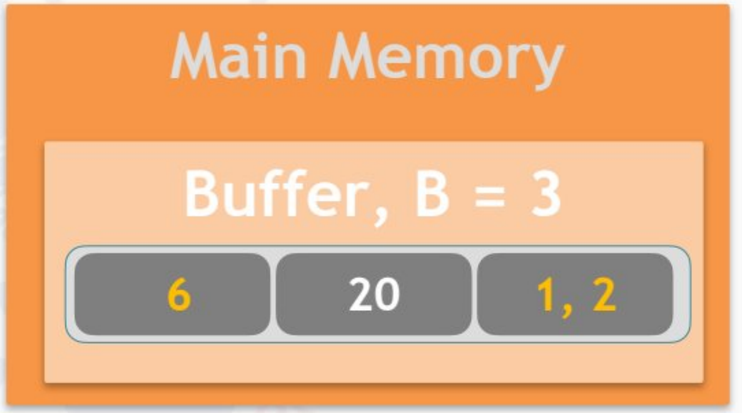




2-way merge takes place.

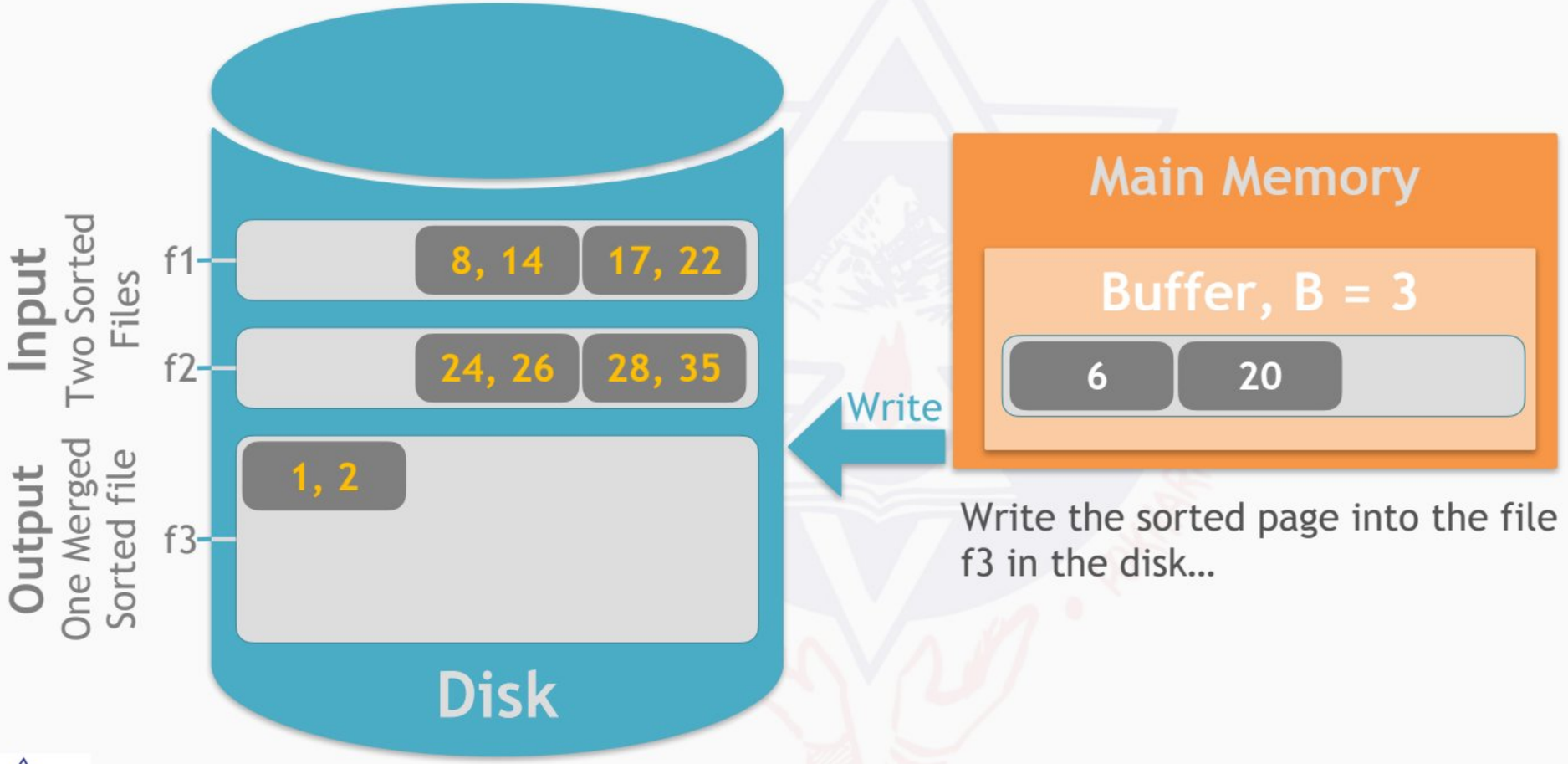
External Merge Algorithm (2 Way Sort)





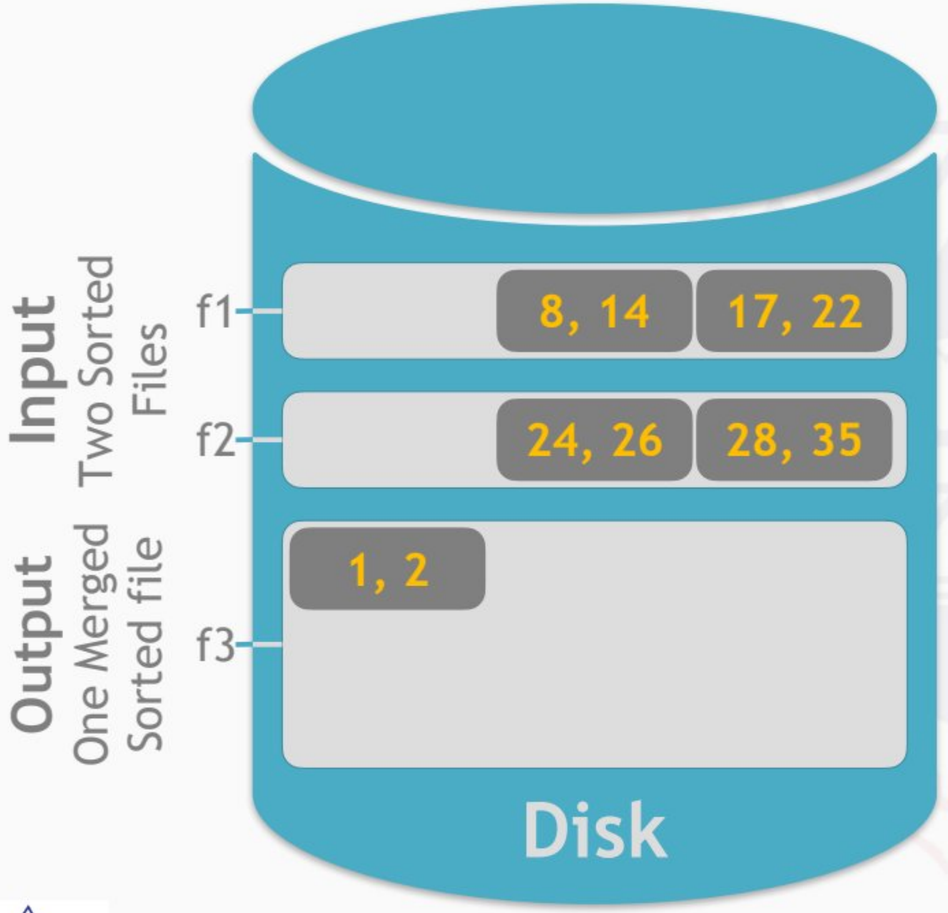
2-way merge takes place.

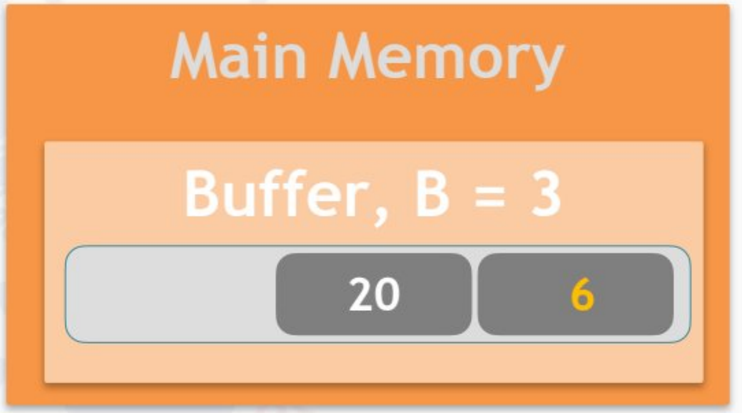
External Merge Algorithm (2 Way Sort)



CamScanner

External Merge Algorithm (2 Way Sort)

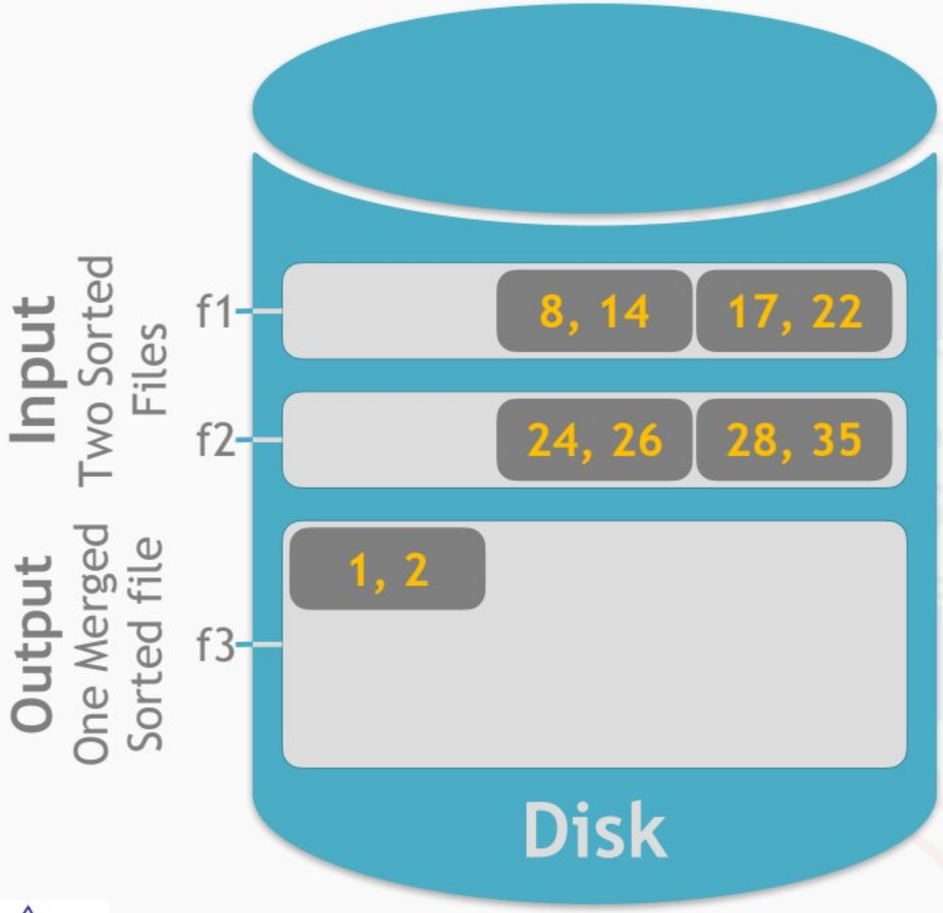


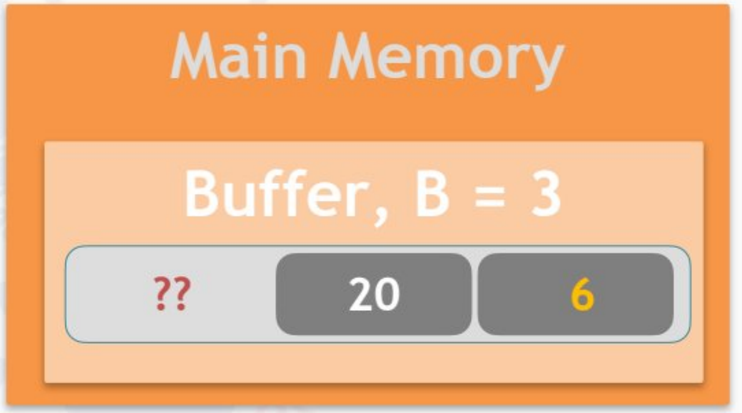


2-way merge takes place.
Keep merging until one buffer becomes empty...

CamScanner

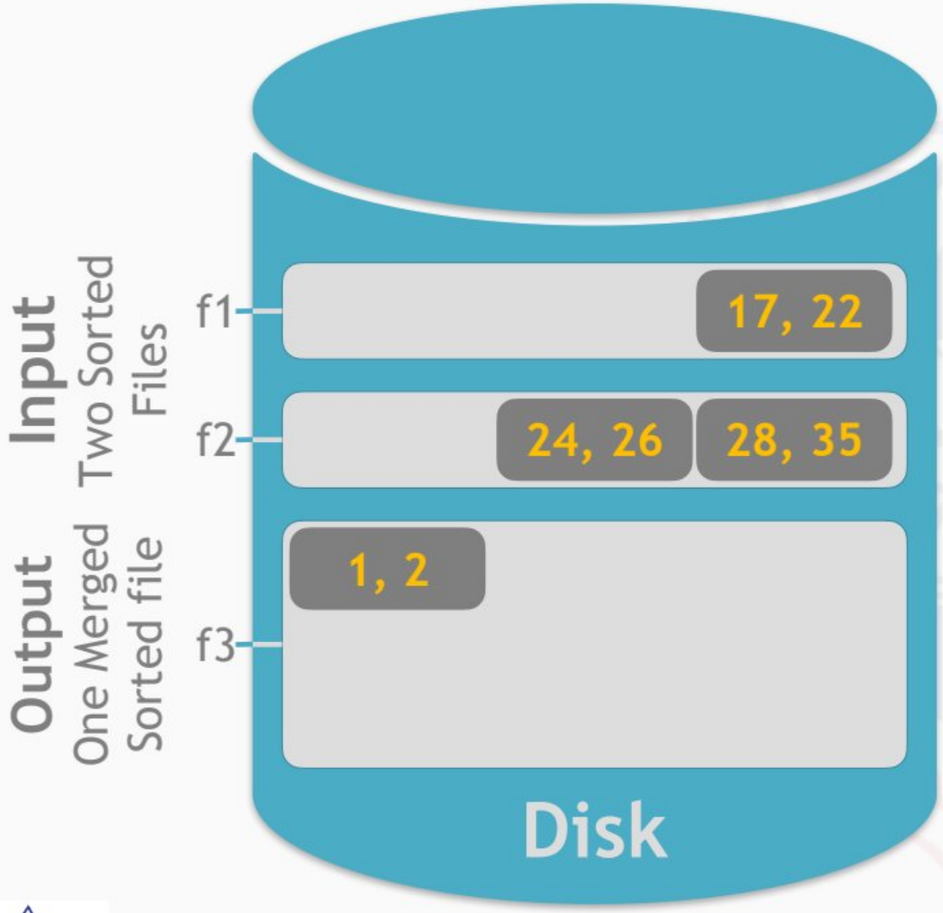
External Merge Algorithm (2 Way Sort)

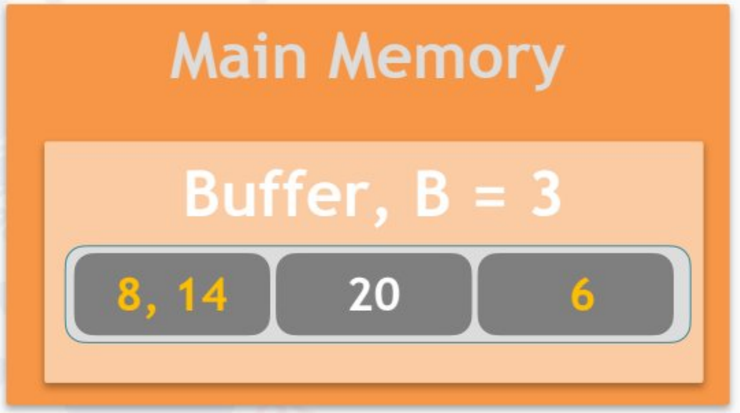




Which page should we read next from f1 or f2 ?

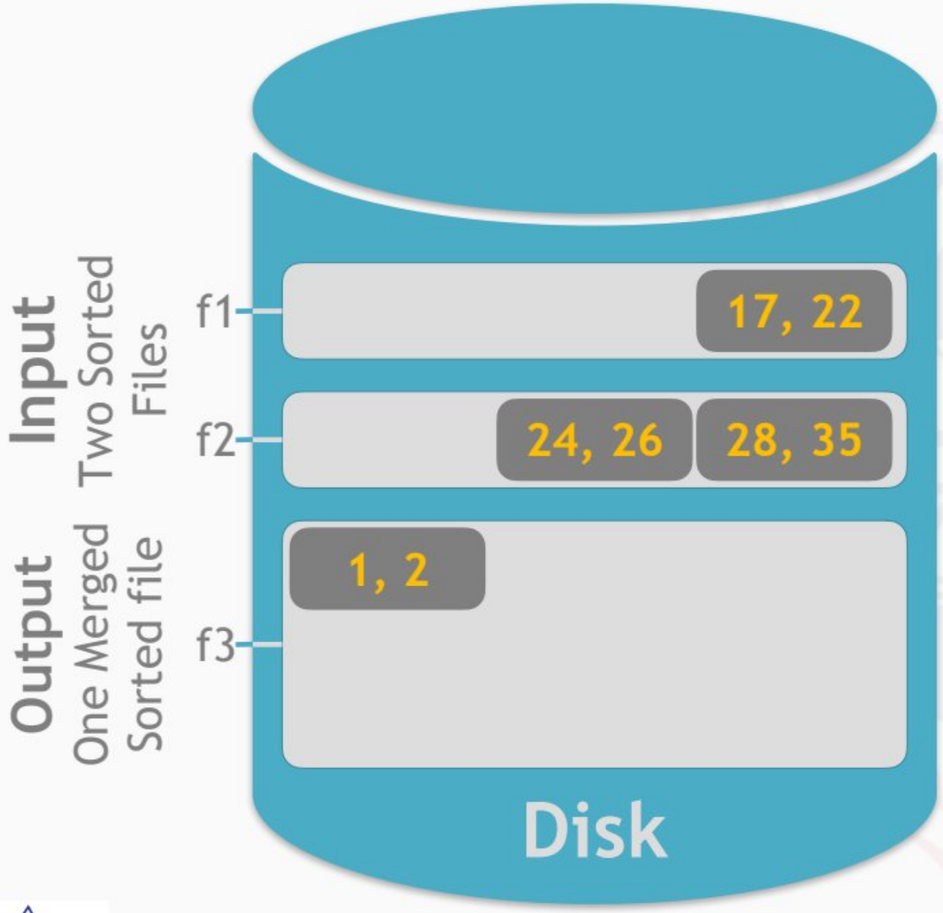
External Merge Algorithm (2 Way Sort)

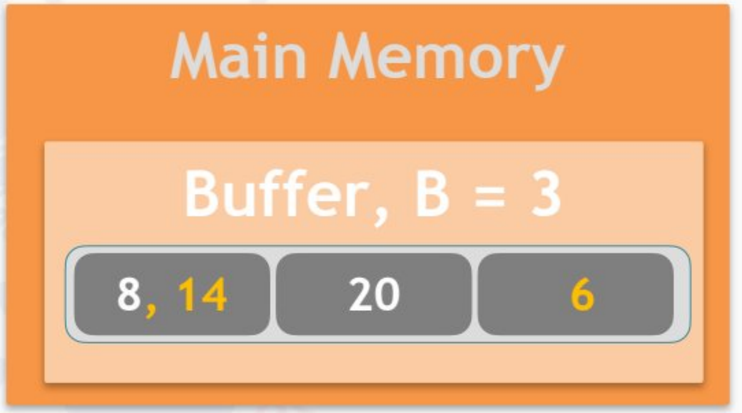




Which page should we read next from f1 or f2 ?
Since f2 contains all elements >= 20,
So, read from f1...

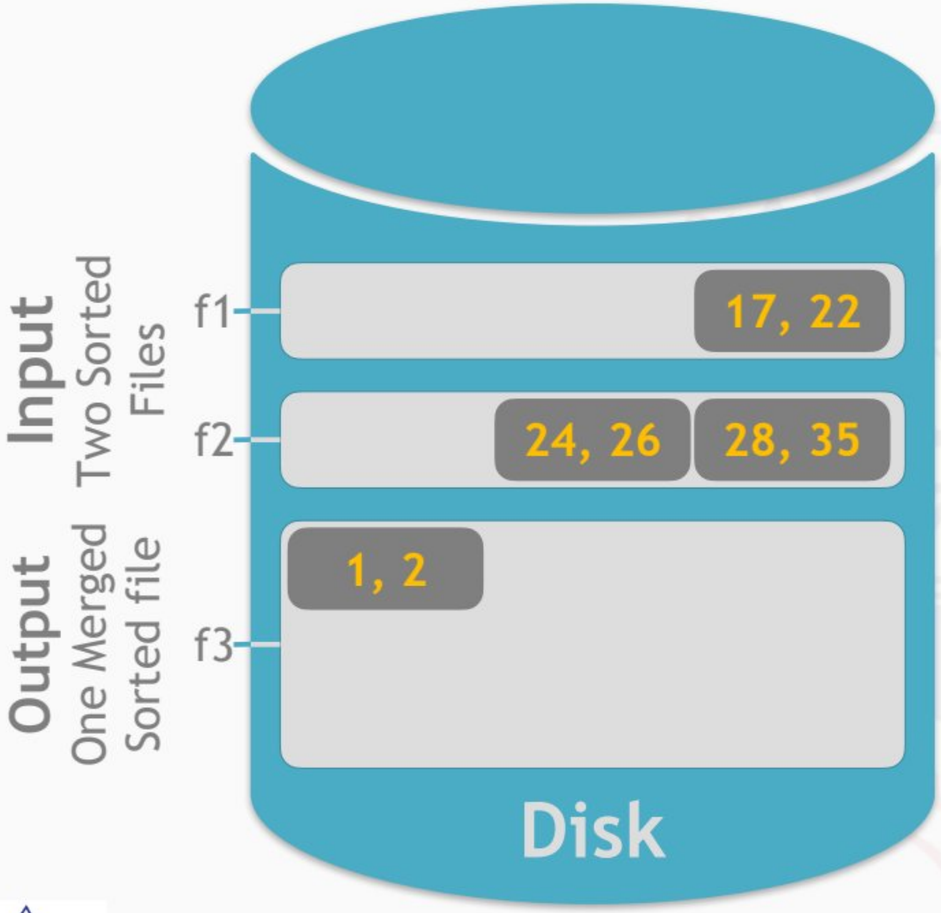
External Merge Algorithm (2 Way Sort)

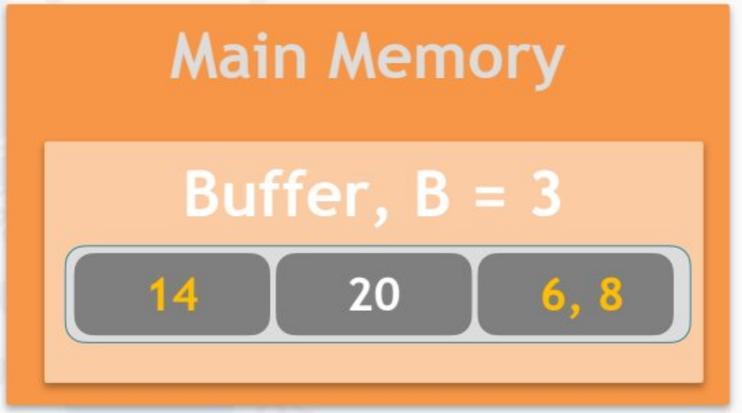




2-way merge takes place.

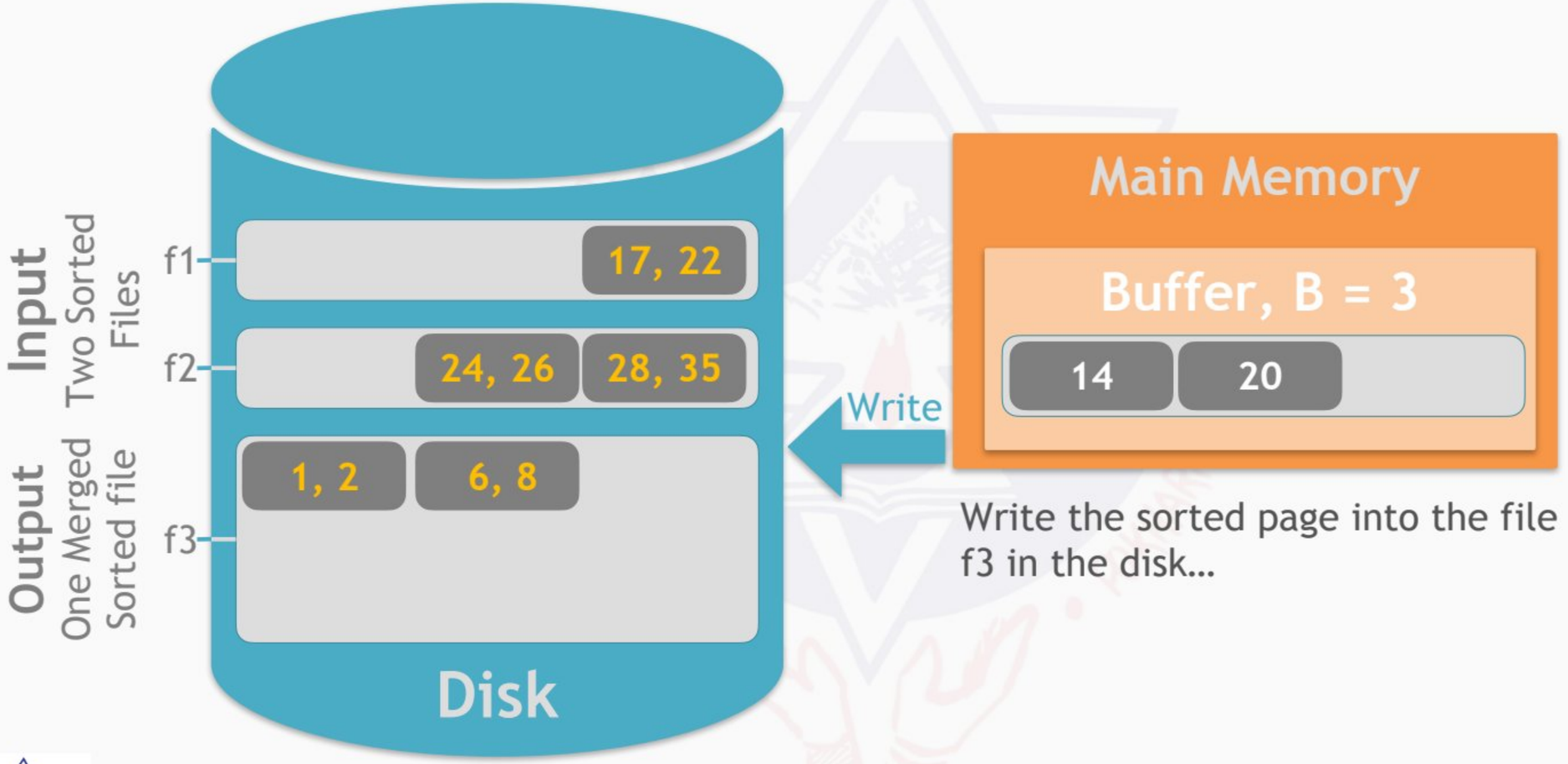
External Merge Algorithm (2 Way Sort)





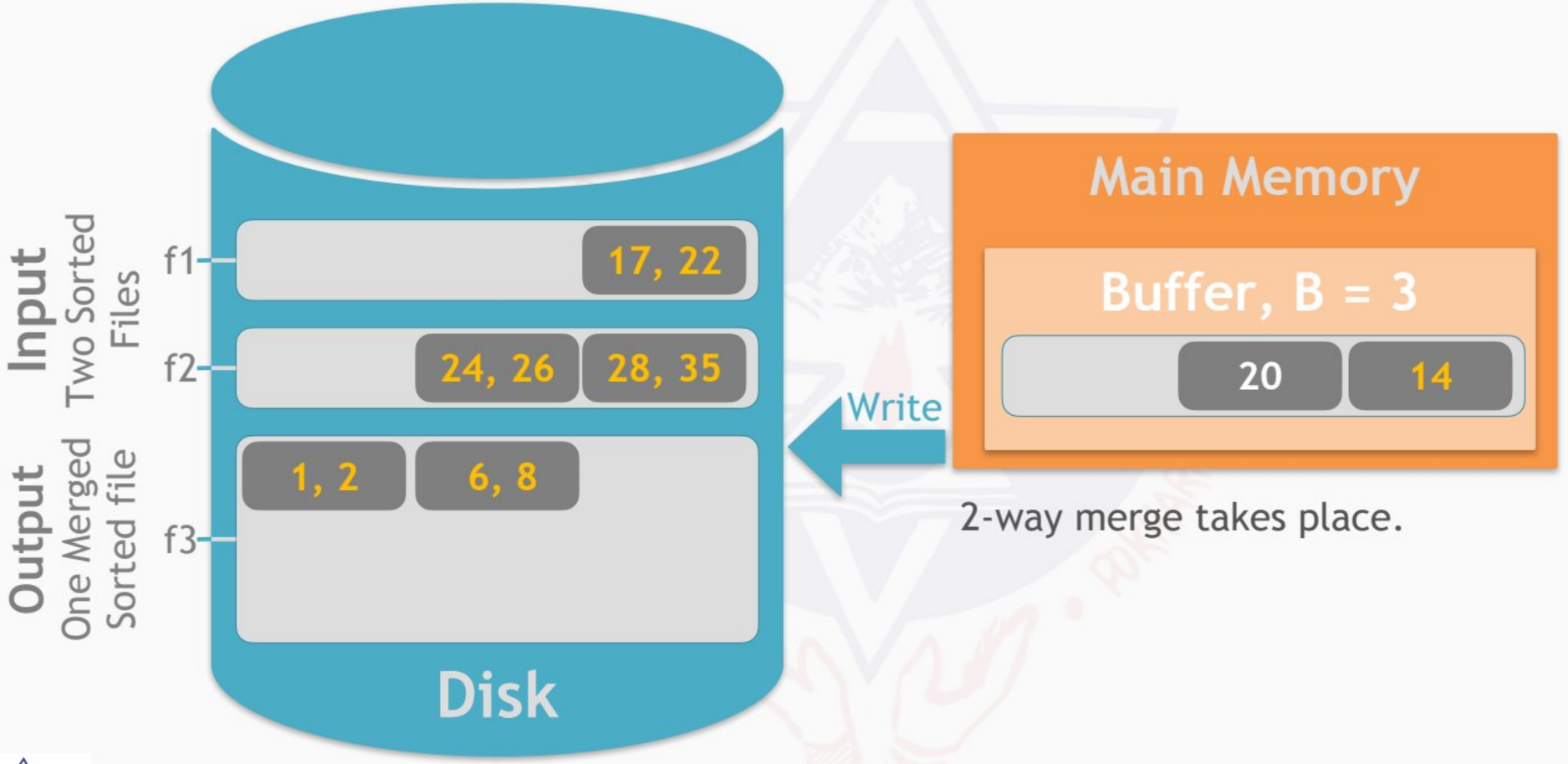
2-way merge takes place.

External Merge Algorithm (2 Way Sort)



CamScanner

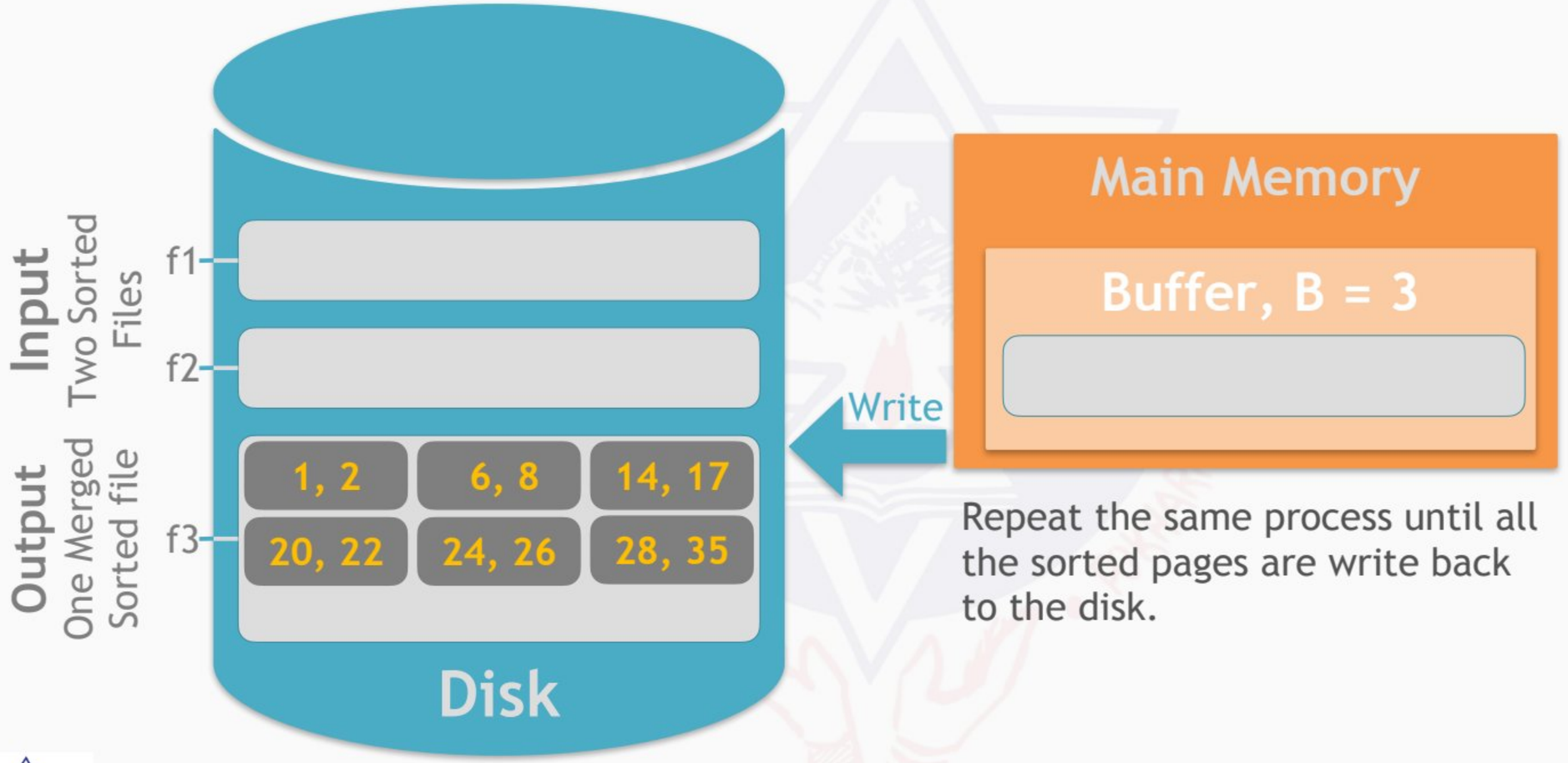
External Merge Algorithm (2 Way Sort)



2-way merge takes place.

CamScanner

External Merge Algorithm (2 Way Sort)



Repeat the same process until all the sorted pages are write back to the disk.

CamScanner

Analysis of External Merge Algorithm (2 Way Sort)

We can merge 2 lists of arbitrary length with *only* 3 buffer pages

Each page is read once and written once.

The size of list/file f1 = M

The size of list/file f2 = N

$\therefore$ the cost of the 2-Way External Merge = $2(M + N)\ I/Os$

So, when you have 2 sorted files larger than the size of your RAM, use the external merge algorithm to merge those file in $2(M + N)$ I/Os operations using 3 buffer pages

With B+1 buffer pages, can merge B lists with the same I/O cost. How?

CamScanner

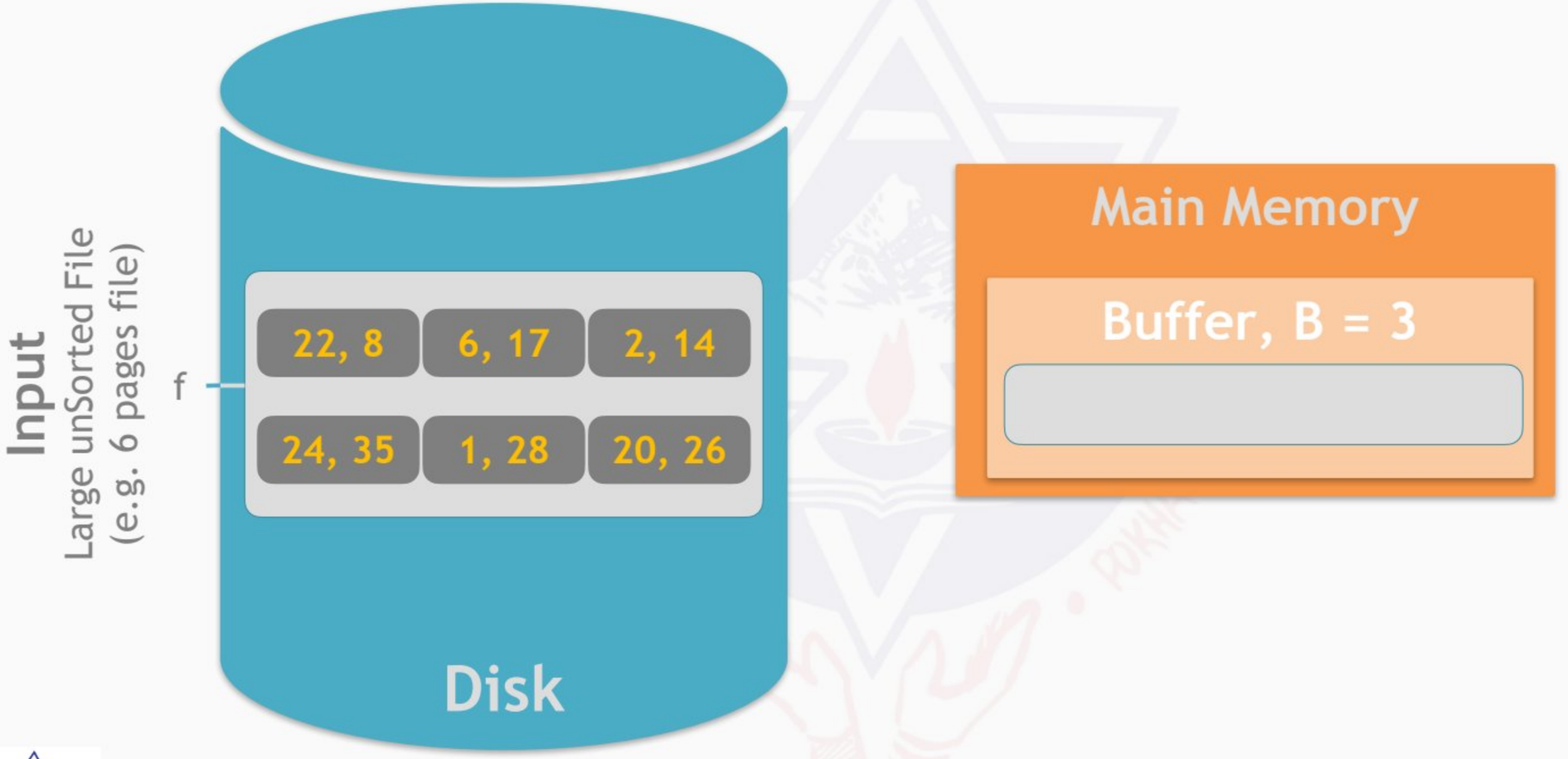
External Merge Sort

How do we sort large files?- (External Merge Sort)

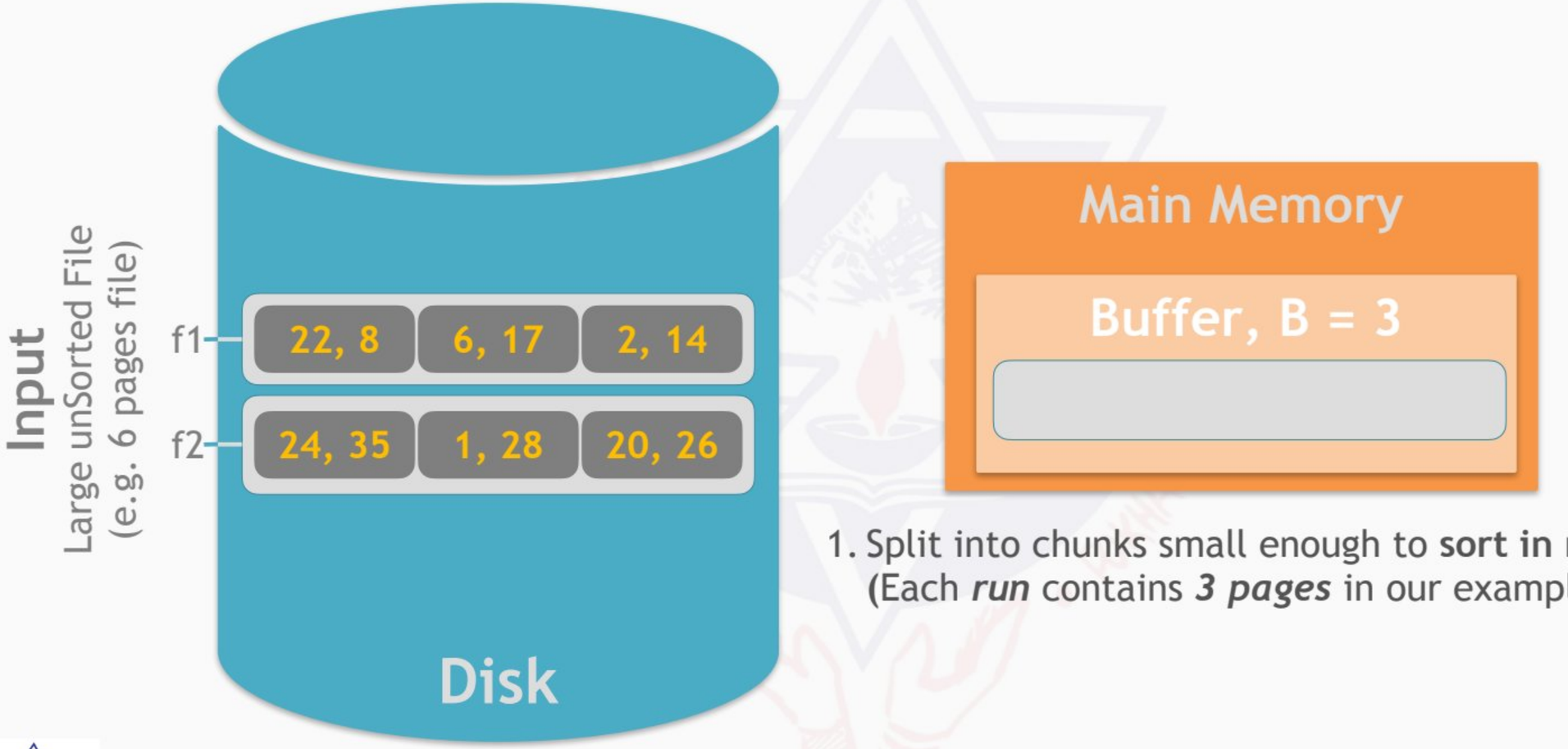
1. Split into chunks small enough to **sort in memory** *(*called **runs***)*
2. **Merge** groups of runs *using the* **external merge** algorithm
3. **Keep merging** the resulting runs until we get single sorted file of the original large file

CamScanner

External Merge Sort (2 Way Sort)



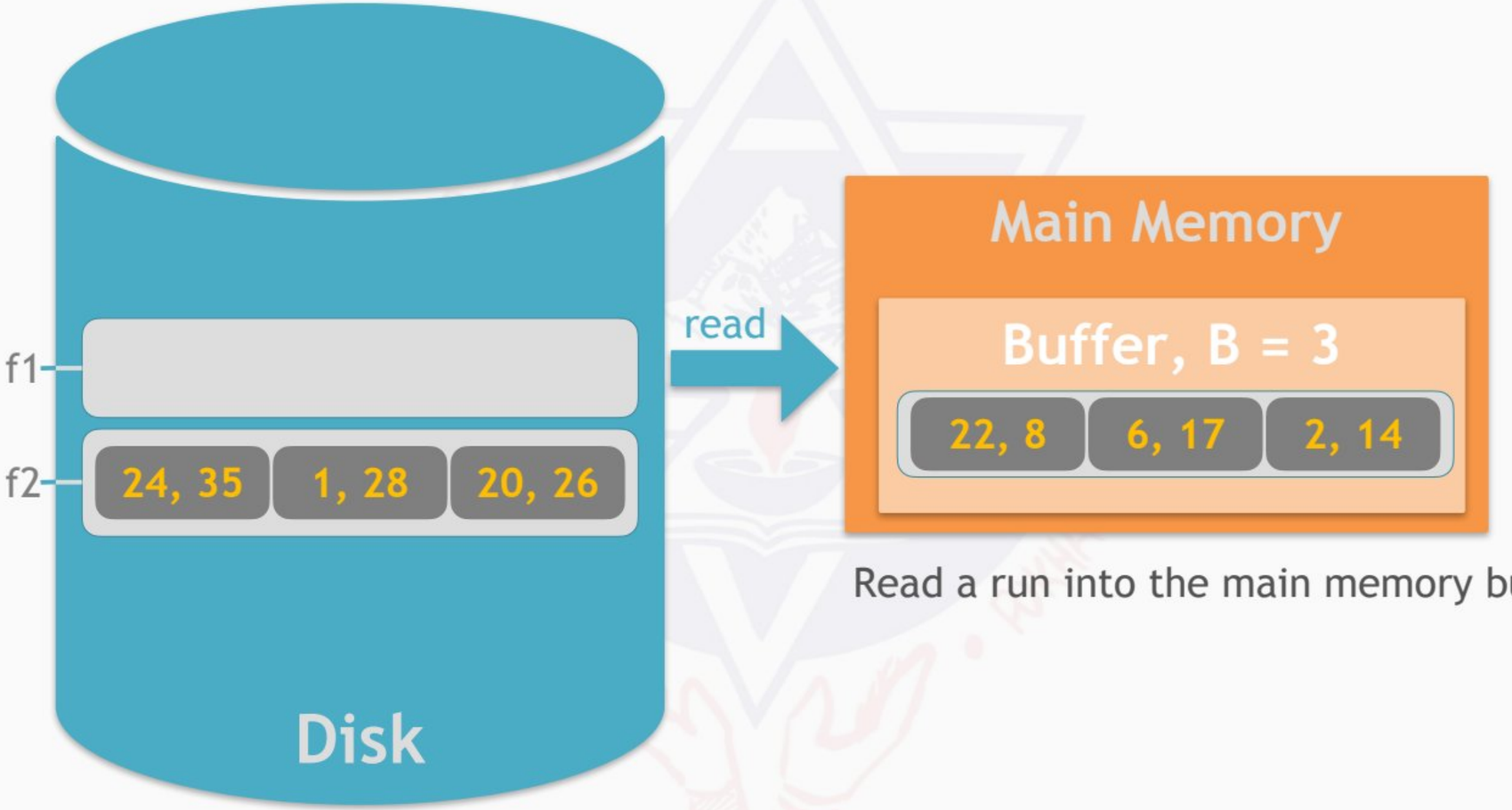
External Merge Sort (2 Way Sort)



1. Split into chunks small enough to **sort in memory (run)**
   (Each ***run*** contains ***3 pages*** in our example)

CamScanner

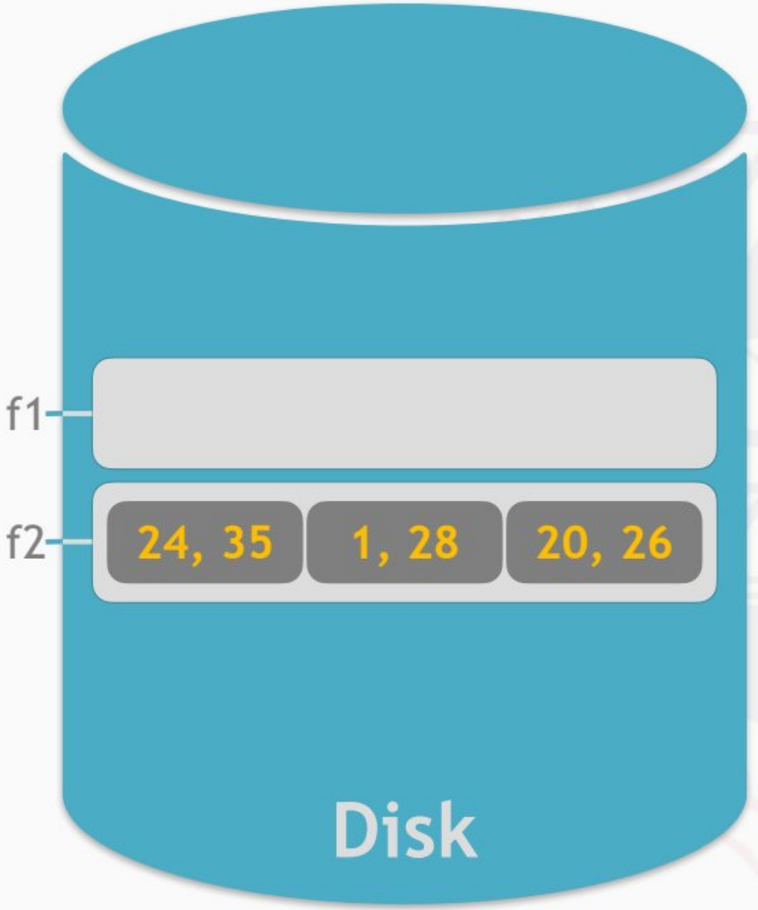
External Merge Sort (2 Way Sort)

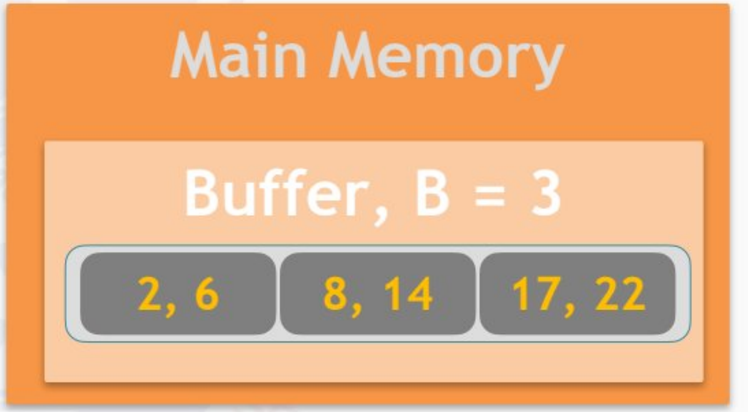


Read a run into the main memory buffer

CamScanner

External Merge Sort (2 Way Sort)

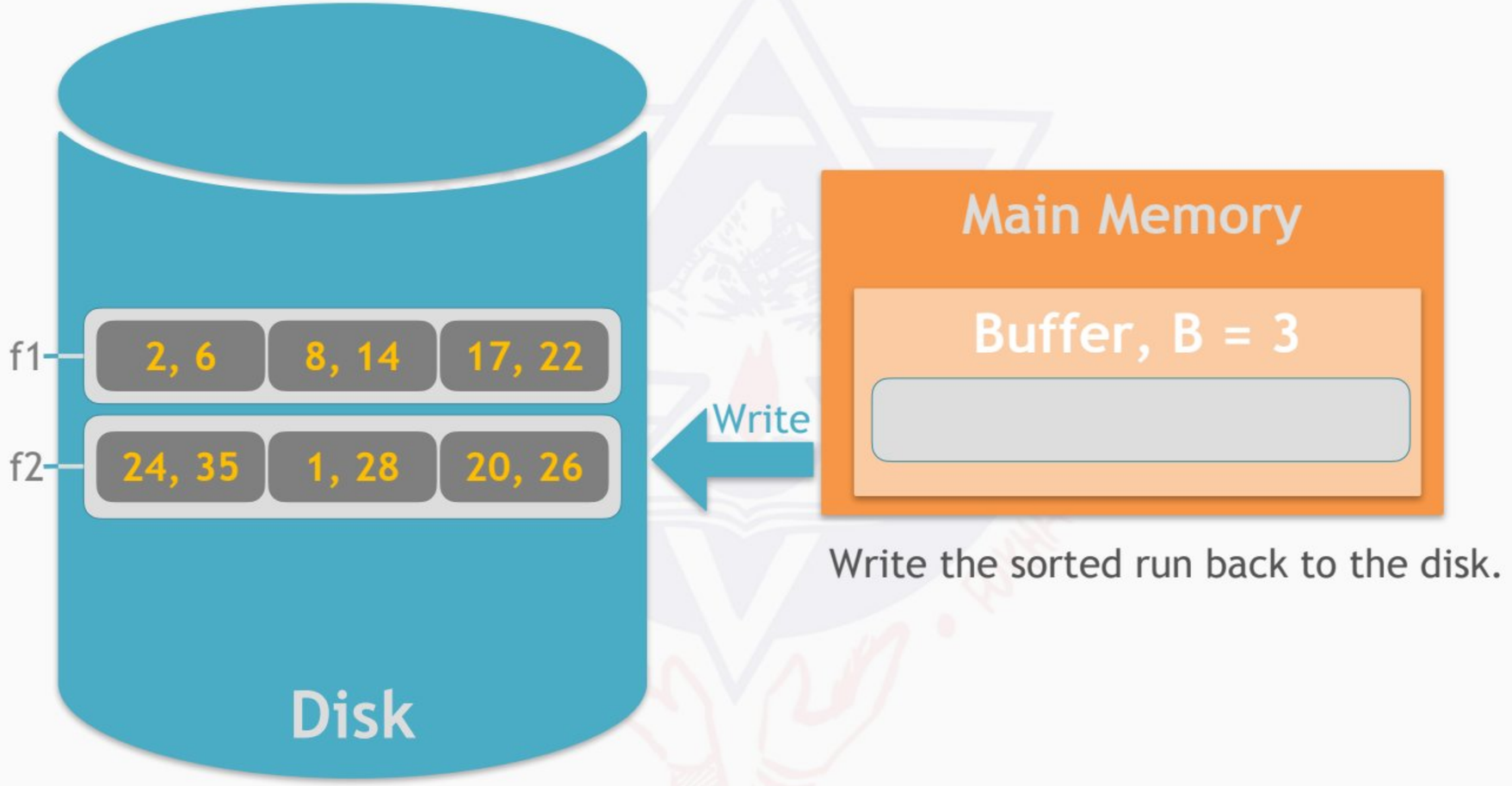




Sort the elements in memory
(e.g. quick sort, merge sort, or heap sort)

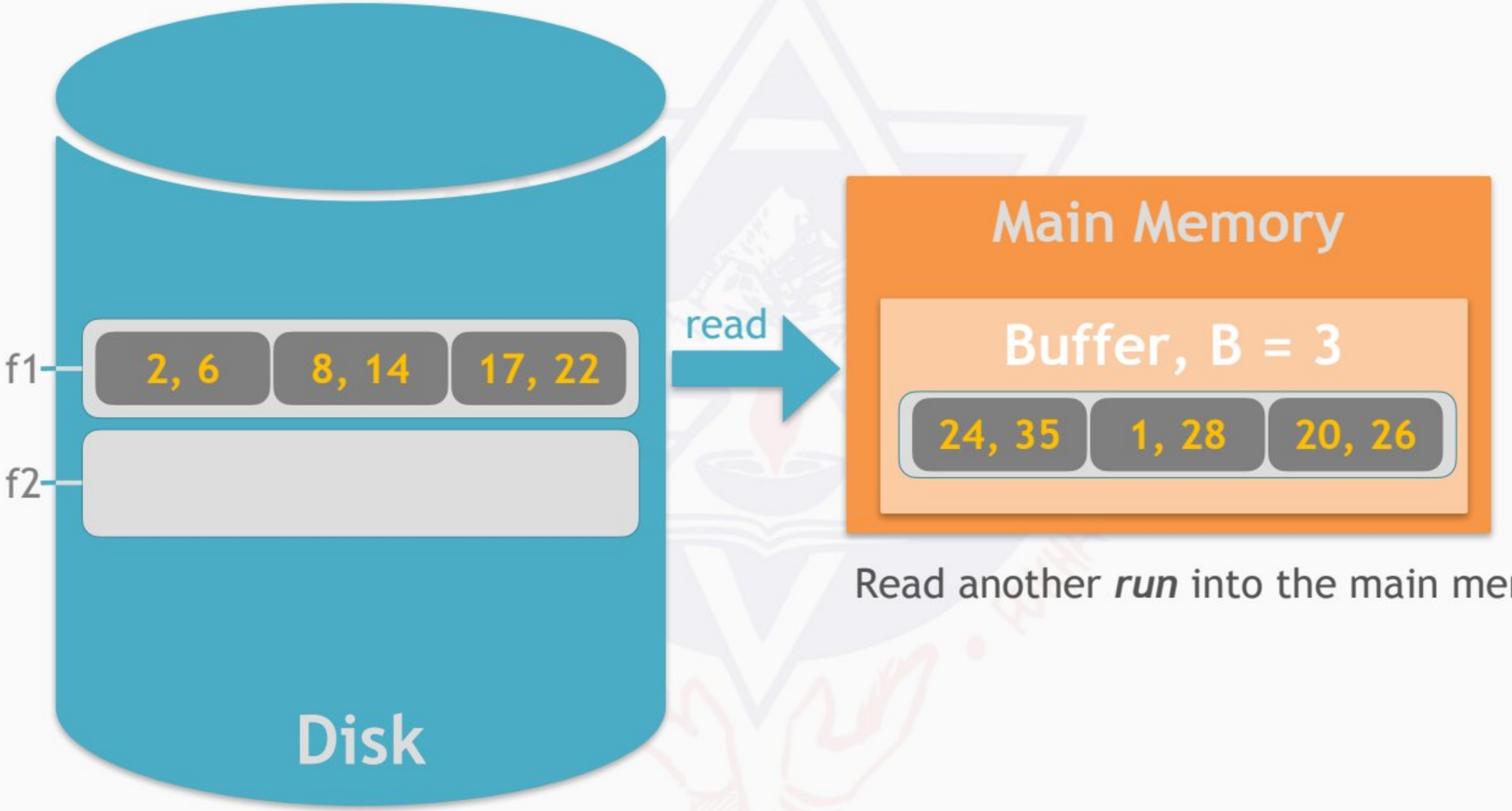
CamScanner

External Merge Sort (2 Way Sort)



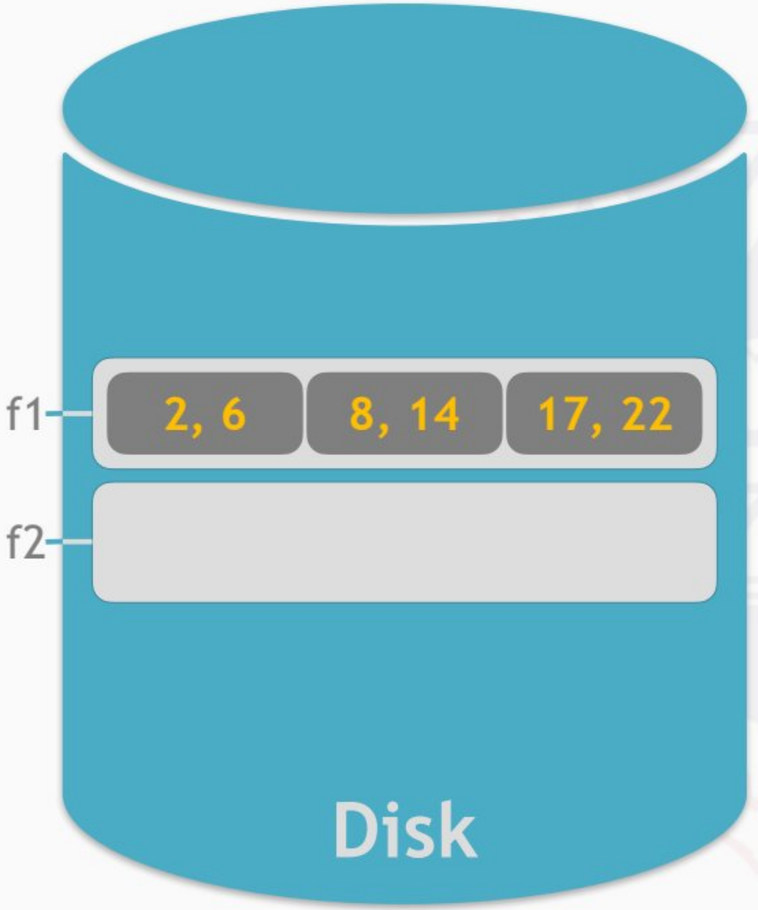
Write the sorted run back to the disk.

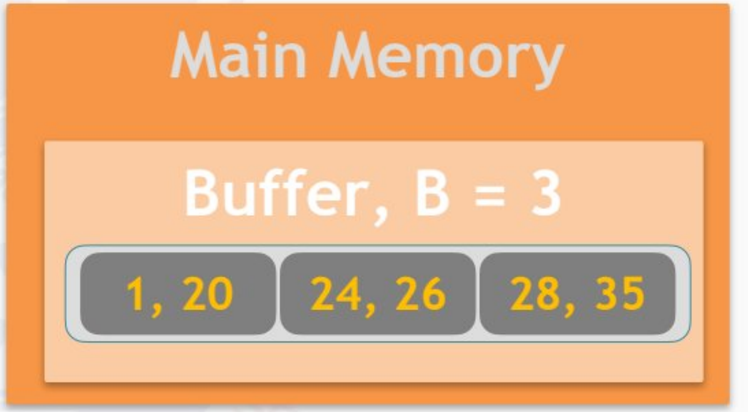
External Merge Sort (2 Way Sort)



Read another ***run*** into the main memory buffer

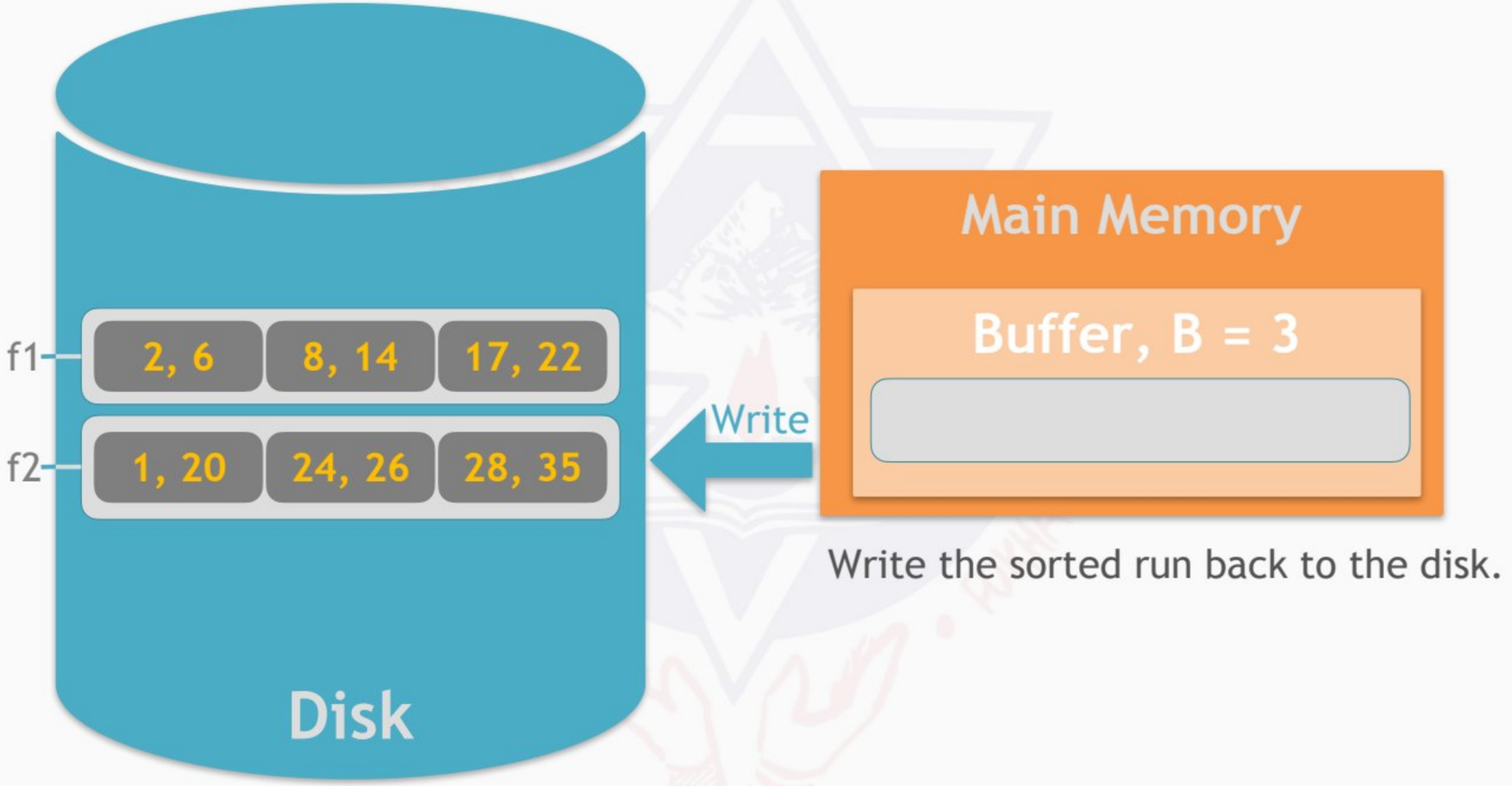
External Merge Sort (2 Way Sort)





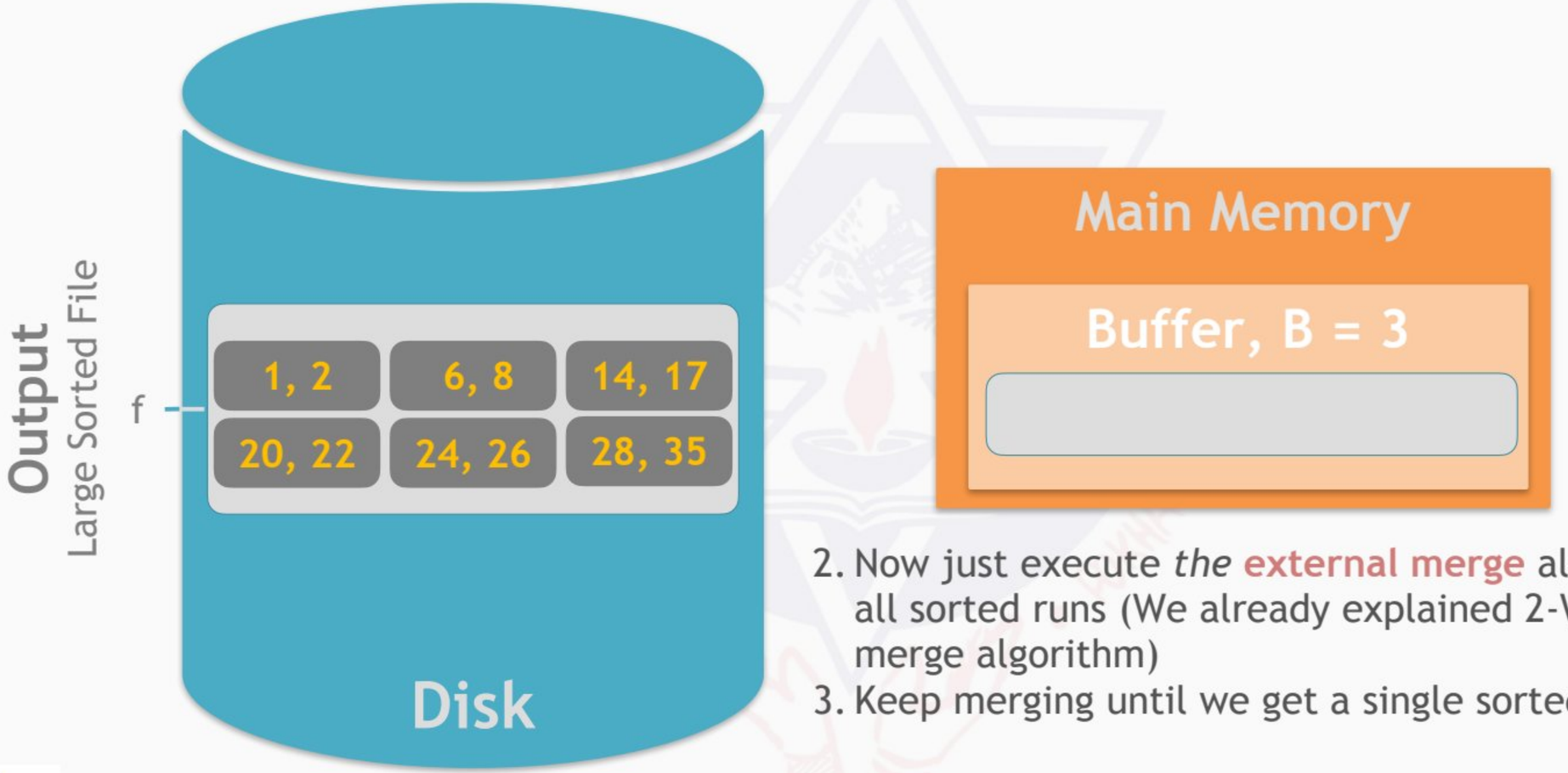
Sort the elements in memory

External Merge Sort (2 Way Sort)



Write the sorted run back to the disk.

External Merge Sort (2 Way Sort)



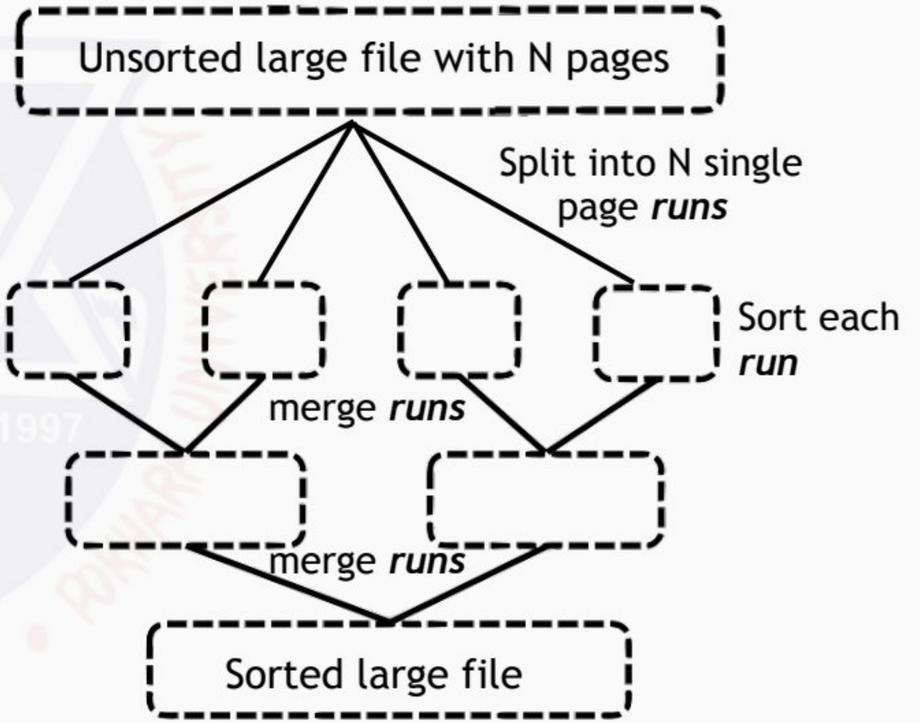
2. Now just execute *the* external merge algorithm to **Merge** all sorted runs (We already explained 2-Way external merge algorithm)
3. Keep merging until we get a single sorted file.

CamScanner

Analysis of External Merge Sort

- For simplicity, assume
  - The large file contains N pages
  - We split N-page file into N single page **runs**
- No of Passes
  - First pass: Merge $N/2$ ***pairs* of runs** each of length **1 page**
  - Second pass: Merge $N/4$ ***pairs* of runs** each of length 2
  - In general, for N pages, we do $\lceil log_2 N \rceil$ passes
    - $+1$ for the initial split & sort
  - Each pass involves reading in & writing out all the pages = $2N$ **I/Os operations**

$\therefore$ Total I/O Cost = $2N(\lceil log_2 N \rceil + 1)$

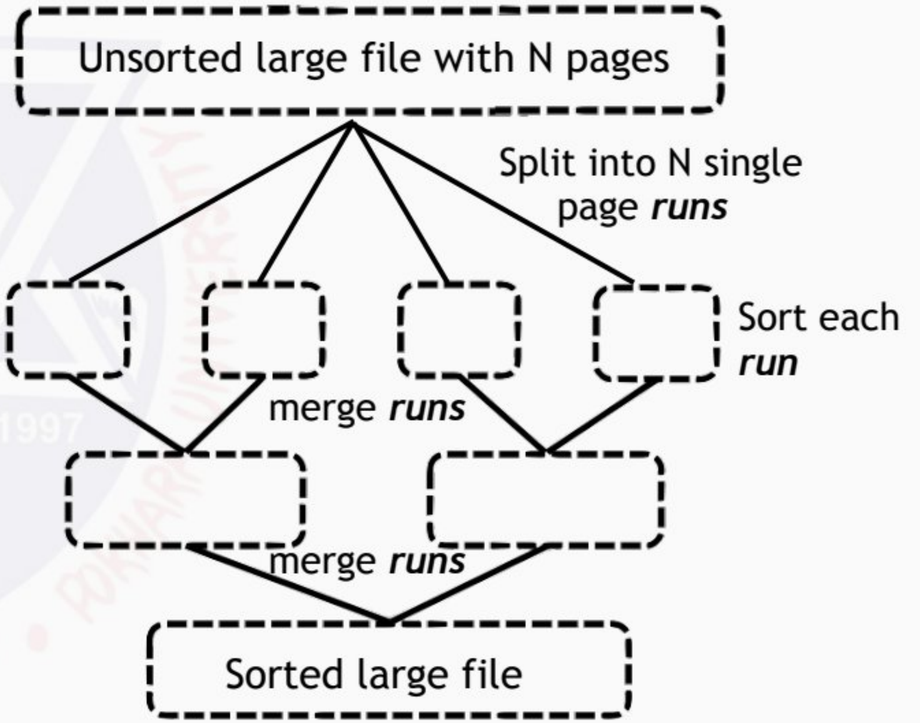


Analysis of External Merge Sort

- For simplicity, assume
  - The large file contains N pages
  - We split N-page file into N single page **runs**
- No of Passes
  - First pass: Merge $N/2$ ***pairs* of runs** each of length **1 page**
  - Second pass: Merge $N/4$ ***pairs* of runs** each of length 2
  - In general, for N pages, we do $\lceil log_2 N \rceil$ passes
    - $+1$ for the initial split & sort
  - Each pass involves reading in & writing out all the pages = $2N$ **I/Os operations**

$\therefore$ Total I/O Cost = $2N(\lceil log_2 N \rceil + 1)$

Can we do better?



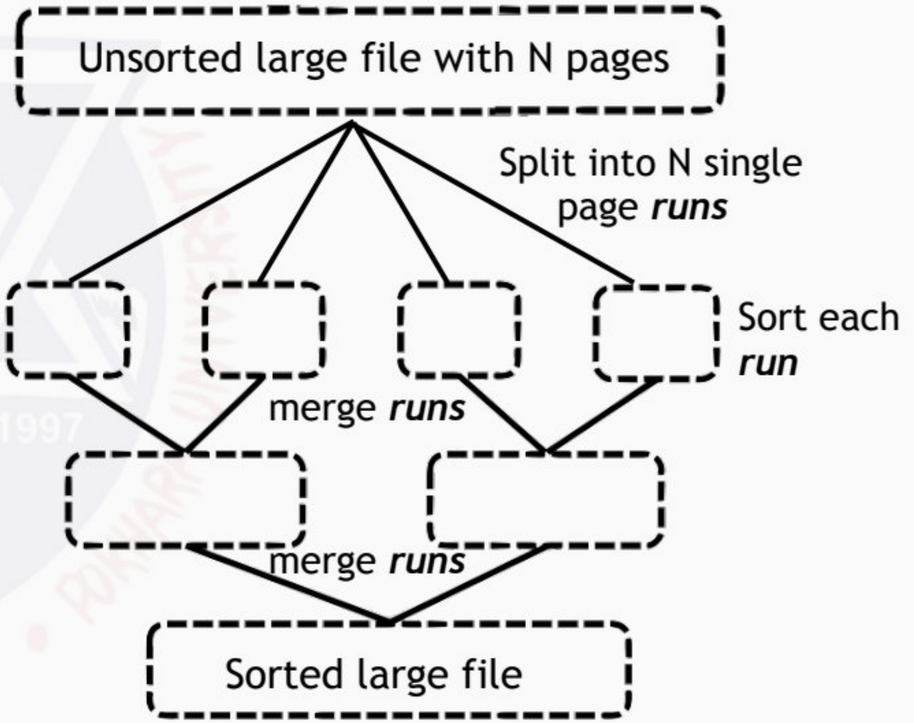
CamScanner

Analysis of External Merge Sort

- For simplicity, assume
  - The large file contains N pages
  - We split N-page file into N single page **runs**
- No of Passes
  - First pass: Merge $N/2$ ***pairs* of runs** each of length **1 page**
  - Second pass: Merge $N/4$ ***pairs* of runs** each of length 2
  - In general, for N pages, we do $\lceil log_2 N \rceil$ passes
    - $+1$ for the initial split & sort
  - Each pass involves reading in & writing out all the pages = $2N$ **I/Os operations**

$\therefore$ Total I/O Cost = $2N(\lceil log_2 N \rceil + 1)$

Yes, we can do!



CamScanner

Analysis of External Merge Sort

How!

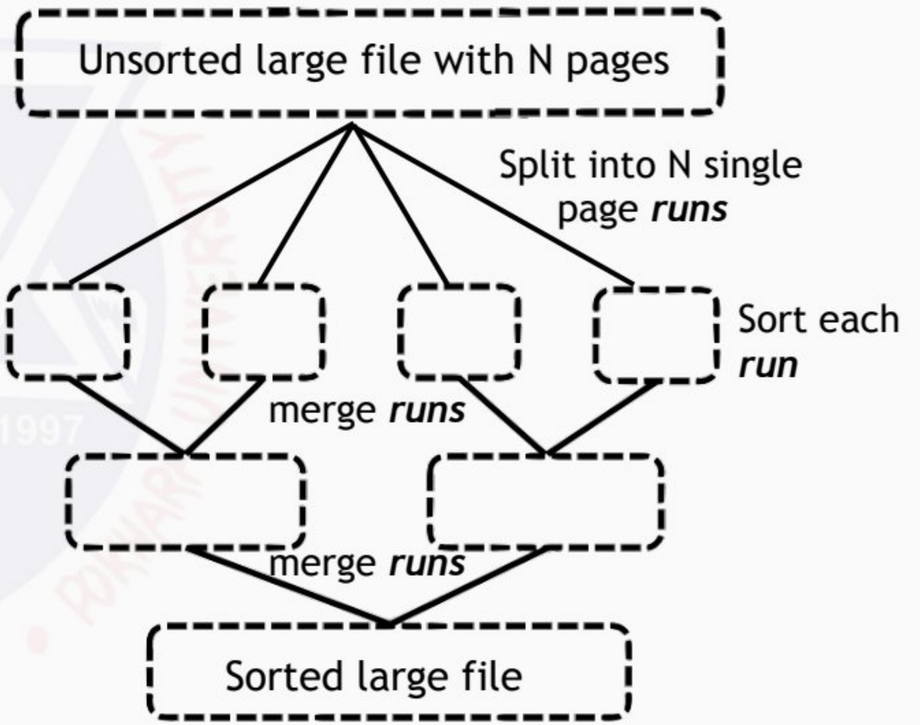
What We Did!

- 2-Way merge algorithm only uses 3 buffer pages.
- But we have more available memory!

Key Idea for betterment: use as much of the available memory as possible in every pass

- Increase the length of initial runs
- Increase the number of buffer for B-way merge

∴ this reduces the number of passes and hence reduces the I/O operations



CamScanner

Analysis of External Merge Sort

Suppose we have $B \geq 3$ buffer pages available

1. When initial length of each run = 1 page and

   buffer pages = B + 1 = 3 pages, then

   This is the 2-Way Merge

$$\therefore \text{I/O Cost} = 2N(\lceil log_2 N \rceil + 1)$$

CamScanner

Analysis of External Merge Sort

Suppose we have $B \geq 3$ buffer pages available

2. Increase the initial length of each run = B pages and

   buffer pages = B + 1 = 3 pages, then

   It is still 2-Way Merge but buffer size are 3 (reduces I/O operations).

$$\therefore \text{I/O Cost} = 2N(\lceil log_2 \frac{N}{B} \rceil + 1)$$

CamScanner

Analysis of External Merge Sort

Suppose we have $B \geq 3$ buffer pages available

3. Increase the initial length of each run = B pages ($B > 3$ pages)

   buffer pages = $B > 3$ pages, then merge B-1 runs at a time.

   It is still B-Way Merge (greatly reduces I/O operations).

$$\therefore \text{I/O Cost} = 2N(\lceil log_{B-1}\frac{N}{B}\rceil + 1)$$

CamScanner

Analysis of External Merge Sort

Number of Passes for different values of B

N	B=3	B=17	B=257
100	7	2	1
10,000	13	4	2
1,000,000	20	5	3
10,000,000	23	6	3
100,000,000	26	7	4
1,000,000,000	30	8	4

CamScanner

External Merge Sort (Pseudocode)

```
ExternalMergeSort(File inputFile, File outputFile, int MEMORY_SIZE):
    // Phase 1: Sorting & Partitioning
    Chunks ← SplitFileIntoChunks(inputFile, MEMORY_SIZE)
    SortedFiles ← []

    for each Chunk in Chunks:
        Sort(Chunk)  // Use an efficient internal sorting algorithm
        TempFile ← WriteToDisk(Chunk)  // Store sorted chunk
        Append TempFile to SortedFiles

    // Phase 2: K-Way Merging
    FinalSortedFile ← KWayMerge(SortedFiles)
    WriteToOutput(FinalSortedFile, outputFile)
```

CamScanner

External Merge Sort (Pseudocode)

```
SplitFileIntoChunks(File inputFile, int MEMORY_SIZE):
    Chunks ← []
    while inputFile is not empty:
        // Read a portion that fits into memory
        Chunk ← ReadNext(MEMORY_SIZE)
        Append Chunk to Chunks
    return Chunks

Sort(Chunk):
    Use an efficient internal sorting (QuickSort, MergeSort)
    return SortedChunk

WriteToDisk(SortedChunk):
    Create new temporary file TempFile
    Write SortedChunk to TempFile
    return TempFile
```

CamScanner

External Merge Sort (Pseudocode)

```
SplitFileIntoChunks(File inputFile, int MEMORY_SIZE):
    Chunks ← []
    while inputFile is not empty:
        // Read a portion that fits into memory
        Chunk ← ReadNext(MEMORY_SIZE)
        Append Chunk to Chunks
    return Chunks

Sort(Chunk):
    Use an efficient internal sorting (QuickSort, MergeSort)
    return SortedChunk

WriteToDisk(SortedChunk):
    Create new temporary file TempFile
    Write SortedChunk to TempFile
    return TempFile
```

CamScanner

External Merge Sort (Pseudocode)

```
KWayMerge(SortedFiles):
    MinHeap ← Empty Priority Queue

    // Step 1: Load first element from each sorted file into the heap
    for each File in SortedFiles:
        Element ← ReadNext(File)
        Insert (Element, File) into MinHeap

    MergedOutput ← Empty List

    // Step 2: Merge Process
    while MinHeap is not empty:
        SmallestElement, File ← ExtractMin(MinHeap)  // Get the smallest value
        Append SmallestElement to MergedOutput

        if File has more elements:
            NextElement ← ReadNext(File)
            Insert (NextElement, File) into MinHeap

    return MergedOutput
```

CamScanner

Optimizing External Merge Sort

Assignment:

1. Repacking
2. Replacement Sort.

CamScanner

Case Study

Assignment:

1. Application of external sorting on large-scale data Engineering.

CamScanner

THANK YOU

CamScanner